



ECOLE NATIONALE
SUPÉRIEURE
D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الاعلام الآلي سابقاً)
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Engineering Dissertation

For obtaining the State Engineer Diploma in Computer Science

Option: Computer Systems and Software (SL)

Theme:

Explainable Artificial Intelligence For
Deep Time Series Models

Completed by:

BENKHELIFA Bouchra

Supervised by:

Mrs. BOUZEGHOUB Amel (TSP)

Mrs. CHARABI Leila (ESI)

Mr. ZAOUIDI Mohamed (TSP)

Jury Members:

Mrs. YESSAD Lamia

Mrs. BESSAH Naima

Mrs. MAOUCHE Hanane

Host Organization : Samovar Télécom SudParis / Institut Polytechnique de Paris (TSP)

Promotion : 2025 / 2026

Acknowledgement

First and foremost, I would like to thank Allah, the Almighty, for granting me the strength, patience, and perseverance necessary to complete this project.

I would like to express my sincere gratitude to Mrs. Amel Bouzeghoub from the SAMOVAR laboratory at Télécom SudParis for her weekly guidance, continuous support, valuable advice, and insightful feedback throughout this project. Her encouragement and constructive recommendations played a significant role in helping me achieve the quality of work presented in this thesis.

I would also like to extend my sincere thanks to Mr. Mohamed Zaouidi from Télécom SudParis for his support, assistance, and valuable suggestions. Despite his busy schedule and research commitments, he was always willing to help and provide guidance whenever needed.

My heartfelt thanks go to Mrs. Leila Charabi, my academic supervisor, for her constant support, availability, careful reviews, valuable advice, and continuous feedback throughout the development of this project. Her guidance and encouragement were essential in helping me improve my work at every stage.

I would also like to thank all the professors and instructors who contributed to my education and provided me with the knowledge and skills necessary to undertake this project.

My sincere appreciation is extended to the members of the jury for accepting to evaluate this work and for their valuable time and consideration.

Finally, I would like to thank everyone who contributed, directly or indirectly, to the successful completion of this project.

إهداء

«وَجَاءَتْهُ الْبُشْرَى»

الحمد لله حمداً طيباً مباركاً على توفيقه ونعمه التي لا تُحصى، فقد أنعم علينا من العلم أحسنه، والصبر أعظمه، والنجاح أجمله. قبل أن أفتخر بما حققته اليوم، لا بد أن أعترف بفضل من سبقني تعباً وجهداً، ومن كان لهما الفضل الأكبر في كل خطوة وصلت إليها. إلى والديّ العزيزين، أمي وأبي، اللذين منحاني من الحب والدعم والثقة ما كان كفيلاً بأن يعيد إليّ الإيمان بنفسي في المحطات التي فقدته فيها، وساندتماني حين ضعفت، وقدمتم لي كل ما استطعتم، بل وحتى ما كان فوق استطاعتكما. إن هذا النجاح ليس ثمرة جهدي وحدي، بل هو حصاد سبعة عشر عاماً من عطاءكم وتضحياتكم وصبركم. اليوم أضع بين أيديكما شهادة الهندسة، فهنيئاً لكم ما زرعتما وما جنيتما. أسأل الله أن يجزيكما عني خير الجزاء وأن يديمكما نعمة وسنداً في حياتي. إلى أخواتي العزيزات، اللواتي كنّ لي السند والرفقة والطمأنينة، والكتف الذي أستند إليه، شكراً لكنّ على محبتكنّ ودعمكنّ الذي لا يُقدّر بثمن.

إلى أخي الصغير، أنت تراني قدوة، وأنا أرى فيك مستقبلاً يستحق أن يُحلم به. أسأل الله أن يسهل طريقك ويبلغك ما تمنى. إلى جدي الحبيب، الذي كان يتمنى أن يكون حاضراً في يوم تخرجي، لكن الأقدار شاءت أن يرحل قبل أن يشهد هذه اللحظة. ها أنا اليوم أخرج يا جدي العزيز، وأتمنى أن تكون غفوراً بي في جنات الرحمن، وأن يصلك مني هذا الإنجاز دعاءً ومحبةً ووفاءً لذكراك الطيبة. إلى عائلتي الكبيرة، شكراً لكم على محبتكم ودعمكم، الدائم. دمت لي سنداً وظهراً لا يميل، وبارك الله فيكم جميعاً.

Dedication

To the friends who made this journey lighter, warmer, and more meaningful.

To **Hidayat** and **Manar**, who walked beside me through every moment of this journey the doubt, the strength, the joy, and the accomplishment always present, always there.

To **Maroua**, who, though from a different field, always lent me a sincere ear, offered her unwavering support.

To **Hadjer**, genuine, always there, and never once hesitating to help.

To **Aida, Ahlam, and Melissa**, who, each in her own way, never stopped showing up through quiet check-ins, boundless optimism, and a friendship that distance could never fade.

There was a chapter far more precious childhood, where the truest bonds are born without even trying.

To **Douaa**, my childhood friend who always called me “*my smart girl*,” words that have carried me further than she may know.

To **Rana**, my oldest friend who never stopped praying for me and believing I would do my best .

To all of you, thank you for your love, your support, and for being part of every chapter of this journey.

Abstract

Deep learning models for time series forecasting including Transformer-based architectures such as iTransformer and PatchTST have achieved remarkable predictive accuracy on complex real-world benchmarks. However, their black-box nature severely limits their deployment in high-stakes domains where forecast justification is as critical as forecast accuracy. Existing Explainable AI (XAI) methods provide descriptive feature attributions but fail to prescribe actionable interventions. Counterfactual explanations offer a more actionable alternative, yet existing methods were designed for classification tasks and cannot be directly transposed to the continuous trajectory outputs of time series forecasting.

This dissertation proposes **RL-MCF** (Reinforcement Learning Masked Counterfactual Explanations For Time Series Forecasting), a principled and model-agnostic framework for generating counterfactual explanations for arbitrary deep time series forecasting models. RL-MCF operates in the structured latent space of a pre-trained TCN autoencoder, ensuring that generated counterfactuals remain on the data manifold and are therefore temporally coherent and distributionally realistic. A temporal masking mechanism enforces that perturbations are sparse and concentrated on the most recent, actionable timesteps of the input window. An actor-critic reinforcement learning agent learns an amortized counterfactual generation policy that jointly optimises validity and proximity via a composite reward function.

The framework is evaluated on three benchmark datasets ETTh1, ETTh2, and Weather across five forecasting architectures of fundamentally different designs. RL-MCF consistently outperforms the gradient-based baseline ForecastCF on almost all evaluation dimensions, achieving a validity ratio of up to 0.976, a compactness score of 0.803 versus 0.013 for the baseline, and a plausibility score of 0.170 versus 0.642, confirming that latent-space generation produces significantly more realistic counterfactuals. An ablation study validates each design choice independently.

Keywords: Explainable Artificial Intelligence, Counterfactual Explanations, Time Series Forecasting, Reinforcement Learning, Deep Learning.

Résumé

Les modèles d'apprentissage profond pour la prévision de séries temporelles notamment les architectures à base de Transformers telles que l'iTransformer et le PatchTST ont atteint des performances prédictives remarquables sur des benchmarks réels complexes. Cependant, leur nature de boîte noire limite considérablement leur déploiement dans les domaines critiques où la justification d'une prévision est aussi importante que sa précision. Les méthodes d'Intelligence Artificielle Explicable (XAI) existantes fournissent des attributions de caractéristiques descriptives, mais ne permettent pas de prescrire des interventions concrètes et actionnables. Les explications contrefactuelles répondent à la question « Qu'est-ce qui aurait dû être différent dans l'entrée pour que la prévision change ? », offrant ainsi une alternative plus opérationnelle, mais les méthodes actuelles ont été conçues pour des tâches de classification et ne peuvent pas être directement transposées aux sorties de trajectoires continues de la prévision de séries temporelles.

Ce mémoire propose **RL-MCF** (Reinforcement Learning Masked Counterfactual Explanations For Time Series Forecasting), un cadre principé et agnostique au modèle pour la génération d'explications contrefactuelles pour tout modèle de prévision de séries temporelles. RL-MCF opère dans l'espace latent structuré d'un auto-encodeur TCN pré-entraîné, garantissant que les contrefactuels générés restent sur le collecteur de données et sont donc temporellement cohérents et réalistes sur le plan distributionnel. Un mécanisme de masquage temporel force les perturbations à être éparées et concentrées sur les pas de temps les plus récents et actionnables de la fenêtre d'entrée. Un agent d'apprentissage par renforcement acteur-critique apprend une politique amortie de génération contrefactuelle qui optimise conjointement la validité et la proximité via une fonction de récompense composite.

Le cadre est évalué sur trois jeux de données de référence ETTh1, ETTh2 et Weather à travers cinq architectures de prévision de conceptions fondamentalement différentes. RL-MCF surpasse systématiquement la méthode de référence ForecastCF sur presque toutes les dimensions d'évaluation, atteignant un taux de validité allant jusqu'à 0,976, un score de compacité de 0,803 contre 0,013 pour la ligne de base, et un score de plausibilité de 0,170 contre 0,642, confirmant que la génération dans l'espace latent produit des contrefactuels significativement plus réalistes. Une étude d'ablation valide chaque choix de conception de manière indépendante.

Mots-clés : Intelligence Artificielle Explicable, Explications Contrefactuelles, Prévision de Séries Temporelles, Apprentissage par Renforcement, Apprentissage Profond.

ملخص

حققت نماذج التعلم العميق للتنبؤ بالسلاسل الزمنية — خاصةً البنى المبنية على المحولات مثل iTransformer و PatchTST — نتائج تنبؤية متميزة على مجموعات بيانات حقيقية معقدة. إلا أن طبيعتها كصندوق أسود تجعل توظيفها صعباً في المجالات الحساسة، حيث يصبح تفسير التنبؤ ضرورة لا تقل أهمية عن دقته. تقدم أساليب الذكاء الاصطناعي القابل للتفسير الموجودة حالياً توضيحات تصف أثر كل متغير، لكنها لا تقترح ما يجب تغييره فعلياً. أما التفسيرات المضادة للوقائع — التي تبحث عن أقل تعديل ممكن في البيانات المدخلة يؤدي إلى تغيير التنبؤ — فهي بديل أكثر فائدة من الناحية العملية، غير أن الطرق الموجودة صُممت للتصنيف ولا تصلح مباشرةً للتنبؤ بالسلاسل الزمنية ذات المخرجات المستمرة. تقترح هذه الأطروحة إطار **RL-MCF**، وهو إطار عام ومستقل عن النموذج لتوليد تفسيرات مضادة للوقائع لأي نموذج تنبؤ عميق للسلاسل الزمنية. يعمل RL-MCF داخل الفضاء الكامن لمشفر تلقائي من نوع TCN تم تدريبه مسبقاً، مما يضمن أن التفسيرات المولدة تبقى قريبة من البيانات الحقيقية وتحترم طابعها الزمني. تضمن آلية القناع الزمني أن التعديلات تكون محدودة ومركزة على أحدث نقاط الإدخال القابلة للتدخل. كما يتعلم عميل التعلم المعزز من نوع الممثل-الناقد سياسةً لتوليد التفسيرات تحقق توازناً بين الصحة والقرب من البيانات الأصلية. يُختبر الإطار على ثلاث مجموعات بيانات مرجعية — ETTh1 و ETTh2 و Weather — عبر خمس بنى تنبؤية مختلفة. يتفوق RL-MCF باستمرار على الطريقة المقارنة ForecastCF في جميع معايير التقييم، إذ يصل معدل الصحة إلى 0,976، ودرجة الإيجاز إلى 0,803 مقابل 0,013 للطريقة المقارنة، ودرجة المعقولة إلى 0,170 مقابل 0,642، مما يؤكد أن التوليد في الفضاء الكامن ينتج تفسيرات أكثر واقعية. كما تُثبت دراسة الاستئصال أهمية كل خيار في تصميم الإطار بشكل مستقل.

الكلمات المفتاحية: الذكاء الاصطناعي القابل للتفسير، التفسيرات المضادة للوقائع، التنبؤ بالسلاسل الزمنية، التعلم المعزز، التعلم العميق.

Contents

Acknowledgement	i
ii	إهداء
Dedication	iii
Abstract	iv
Résumé	v
vi	ملخص
List of Abbreviations	xvi
General Introduction	1
I State of the art	3
1 Time Series	4
1.1 Introduction	4
1.2 Definition	4
1.3 Types of Time Series	4
1.4 Components of Time Series	5
1.5 Time Series Decomposition Methods	5
1.6 Objectives and Applications of Time Series Analysis	6
1.7 Classical Forecasting Methods	7
1.8 Traditional Time Series Analysis Pipeline	8
1.9 The Temporal Data Manifold	9
1.10 Temporal Dependencies	10
1.11 Mutable and Immutable Observations	10
1.12 Conclusion	11
2 Deep Learning for Time Series	12
2.1 Introduction	12
2.2 Challenges in Time Series Modeling	12
2.3 Classical Deep Learning Architectures for Time Series	12
2.3.1 Multilayer Perceptrons (MLPs)	13
2.3.2 Recurrent Neural Networks (RNNs)	13
2.3.3 Convolutional Neural Networks (CNNs)	14

2.3.4	Graph Neural Networks (GNNs)	14
2.4	Advanced Architectures: Transformers for Time Series	16
2.4.1	Transformer Architecture	16
2.4.2	Tokenization Strategies in Transformer-Based Time Series Models	17
2.5	Advanced Transformer Architectures for Long-Term Time Series Forecasting	17
2.5.1	Informer	18
2.5.2	Autoformer	19
2.5.3	FEDformer	19
2.5.4	PatchTST	19
2.5.5	Crossformer	20
2.5.6	Inverted Transformer (iTransformer)	21
2.6	Synthesis and Comparison	21
2.7	Conclusion	23
3	Explainable Artificial Intelligence for Time Series Models	24
3.1	Introduction	24
3.2	Conceptual Definitions	25
3.2.1	Explainable Artificial Intelligence (XAI)	25
3.2.2	Explicability	25
3.2.3	Explainability	25
3.2.4	Objectives of XAI	25
3.3	Taxonomy of XAI Methods	26
3.3.1	Scale of Explanations	26
3.3.2	Stage of Explanations	26
3.3.3	Model Architecture Dependency	27
3.3.4	Evaluation of XAI Methods	29
3.4	Output-Based Taxonomy of XAI Methods for Time Series	29
3.5	XAI Methods for Time Series	31
3.5.1	Feature Importance Methods	31
3.5.2	Attention-Based Methods	32
3.5.3	Rule-Based Methods	34
3.5.4	Counterfactual Methods	34
3.6	Comparison of XAI Methods for Time Series	35
3.7	Evaluation Methods for XAI Explanations on Time Series	37
3.7.1	Perturbation-Based Methods	37
3.7.2	Sequence-Aware Methods	38
3.8	Conclusion	38
4	Counterfactual Explanations for Time Series	39
4.1	Introduction	39
4.2	Definition and Formulation of Counterfactual Explanations	40
4.3	Motivation of Counterfactual Explanations	40
4.4	Counterfactual Explanations for Time Series	40
4.4.1	Counterfactuals for Classification	41
4.4.2	Counterfactuals for Forecasting	42
4.5	Taxonomy of Counterfactual generation Methods	42
4.5.1	Optimization-Based Methods	42

4.5.2	Generative Model-Based Methods	43
4.5.3	Instance-Based Methods	43
4.5.4	Learning-Based Methods	43
4.6	Evaluation Metrics for Counterfactual Explanations	44
4.7	Challenges and Trade-offs in Counterfactual Explanations	45
4.8	Positioning of Counterfactual Explanations within XAI	45
4.9	Conclusion	46
II	Contribution	47
5	Framework Design	48
	Introduction	48
5.1	Problem Formulation	48
5.2	Baseline Approaches and Limitations	49
5.2.1	ForecastCF: Gradient-Based Counterfactual Forecasting	49
5.2.2	Reinforcement Learning for Counterfactual Classification	49
5.3	Contributions	50
5.4	Framework Overview	50
5.5	Framework Components	51
5.5.1	TCN Autoencoder	51
5.5.2	Black-Box Forecasting Model	52
5.5.3	Actor-Critic Agent	52
5.5.4	Temporal Masking Mechanism	53
5.5.5	Reward Function	54
5.6	Training and Inference Procedures	55
5.6.1	Training Procedure	55
5.6.2	Inference Procedure	55
5.7	Evaluation Metrics	56
5.7.1	Validity Ratio	56
5.7.2	Stepwise Validity AUC	56
5.7.3	Proximity	57
5.7.4	Compactness	57
5.7.5	Temporal Consistency	57
5.7.6	Plausibility	57
5.8	Conclusion	58
6	Implementation and Experimental Setup	59
6.1	Introduction	59
6.2	Development Environment	59
6.3	Datasets	60
6.3.1	ETTh1	60
6.3.2	ETTh2	61
6.3.3	Weather	61
6.3.4	Synthesis	63
6.3.5	Data Preparation	63
6.4	Forecasting Model Training	65
6.4.1	Model Overview	65

6.4.2	Observation Window and Forecasting Horizon Selection	65
6.4.3	Configuration	67
6.4.4	Implementation	67
6.4.5	Results	68
6.4.6	Reproducing the Results	68
6.5	TCN Autoencoder Training	68
6.5.1	Architecture and Implementation	69
6.5.2	Configuration	69
6.5.3	Results	70
6.5.4	Reproducing the Results	70
6.6	Anomaly Detector Training	70
6.6.1	Motivation and Role in the Framework	70
6.6.2	Implementation	70
6.6.3	Configuration	71
6.6.4	Baseline Plausibility Scores	71
6.7	RL Agent Training and Counterfactual Generation	72
6.7.1	Architecture	72
6.7.2	Full Generation Pipeline	73
6.7.3	Configuration	73
6.7.4	Hyperparameter Selection	74
6.7.5	Reproducing the Results	76
6.8	Conclusion	76
7	Experiments and Results	77
7.1	Introduction	77
7.2	Experimental Setup	77
7.2.1	Baselines	77
7.2.2	Evaluation Protocol	78
7.3	Individual Component Evaluation	78
7.3.1	Forecasting Model Performance	78
7.3.2	Autoencoder Reconstruction Quality	79
7.3.3	RL Agent Performance	80
7.4	Hyperparameter Sensitivity	80
7.4.1	Sensitivity to Band Width f_r	80
7.4.2	Sensitivity to Gap Parameter ρ	81
7.4.3	Reward Weight Sensitivity	82
7.5	Comparison with Baselines	83
7.5.1	Discussion of Results	84
7.6	Model-Agnostic Generalization	86
7.6.1	Discussion of Results	86
7.7	Ablation Study	88
7.7.1	Discussion of Results	88
7.8	Qualitative Evaluation Across Datasets	89
7.8.1	ETTh1	89
7.8.2	ETTh2	90
7.8.3	Weather	91
7.9	Conclusion	92

General Conclusion	93
Bibliography	95

List of Figures

1.1	Time Series Components (from Google Images)	5
1.2	Comparison of Additive and Multiplicative Time Series Decomposition(from Google Images).	6
1.3	Stationary vs. non-stationary time series (from Google Images).	8
1.4	Time Series Analysis Pipeline.	9
1.5	The temporal data manifold $\mathcal{M} \subset \mathbb{R}^L$	9
1.6	Mutable and immutable regions of a look-back window of length L	10
2.1	MLP Architecture for Time Series	13
2.2	RNN Architecture for Time Series	13
2.3	CNN Architecture for Time Series	14
2.4	TCN architectural details Wan et al. (2019).	15
2.5	GNN Architecture for Time Series	15
2.6	The Transformer architecture Vaswani et al. (2017).	17
2.7	Tokenization strategies for Transformer-based time series models.	18
2.8	Informer architecture H. Zhou et al. (2021).	18
2.9	Autoformer architecture Wu et al. (2021).	19
2.10	FEDformer architecture T. Zhou et al. (2022).	19
2.11	PatchTST architecture Nie et al. (2023).	20
2.12	Crossformer architecture Yunhao Zhang and J. Yan (2023).	20
2.13	iTransformer architecture Y. Liu et al. (2023).	21
2.14	Performance comparison of iTransformer against baseline models Y. Liu et al. (2023).	21
3.1	Knowledge graph relating all the purposes of explainability methods (Rojat et al. 2021).	26
3.2	Multi-Criteria Taxonomy of XAI Methods.	28
3.3	Taxonomy of evaluation methods for Explainable Artificial Intelligence (Lopes et al. 2022).	29
3.4	Output-based taxonomy of XAI methods for time series, adapted from Bodria et al. (2023).	30
3.5	LRP with Virtual Inspection Layer Vielhaben et al. (2024).	32
3.6	ShapTST architecture Cheng et al. (2025).	33
3.7	Architecture of the Neuro-Symbolic Time Series Classification (NSTSC) framework.	34
3.8	Time series Evaluation Strategies	37
4.1	Example of a counterfactual explanation for time series classification. The original signal (blue) is modified to generate a counterfactual instance (Red) that changes the predicted class (Delaney, Greene, and Keane (2021)).	41
4.2	Counterfactual forecasting under bounded target constraints	42
5.1	Overview of the RL-MCF framework	51
5.2	Temporal ramp mask illustration	53

6.1	STL decomposition of the ETTh1 target variable (OT, period = 24 h).	61
6.2	STL decomposition of the ETTh2 target variable (OT, period = 24 h).	62
6.3	STL decomposition of the Weather target variable.	63
6.4	Autocorrelation function (ACF) of the target variable for each dataset.	66
6.5	Plausibility score distributions on ETTh1 (IF, LOF, OC-SVM).	72
6.6	RL-MCF training curves on ETTh1 (iTransformer backbone).	74
7.1	iTransformer forecasting results on the test set.	78
7.2	TCN autoencoder reconstruction on the test set.	79
7.3	Counterfactual example on ETTh1 ($\rho = 0.2$, $f_r = 0.5$, $k = 12$).	80
7.4	Validity Ratio vs. f_r (ETTh1, iTransformer); star marks $f_r = 0.5$ (VR=0.976).	81
7.5	Plausibility vs. ρ ; star marks $\rho = 0.2$.	81
7.6	Validity-Proximity trade-off vs. ρ ; $\rho = 0.2$ lies at the knee of the curve.	81
7.7	Reward weight sensitivity on ETTh1.	82
7.8	Qualitative example on a single ETTh1/iTransformer test sample ($\rho=0.2$, $f_r=0.5$). Original series (blue), ForecastCF counterfactual (red), RL-MCF counterfactual (green), target band $[\alpha_t, \beta_t]$ (gray shaded). For this sample: RL-MCF achieves VR = 96% vs. ForecastCF VR = 46%. Dataset-level averages are reported in Table 7.6.	84
7.9	Validity vs. Plausibility Trade-off across datasets (iTransformer).	85
7.10	Compactness across methods and datasets.	85
7.11	Temporal Consistency across methods and datasets (iTransformer).	85
7.12	Validity Ratio heatmap across datasets and architectures. Green ≥ 0.9 ; yellow-red indicates lower validity.	87
7.13	t-SNE of the TCN latent space on ETTh1: original (blue), RL-MCF counterfactuals (red), random perturbations (grey).	87
7.14	Qualitative comparison of counterfactual series generated by RL-MCF, RLP, and the w/o-mask ablation on a representative ETTh1 test sample (iTransformer backbone).	88
7.15	Qualitative examples on ETTh1. Per-sample validity: 0.94, 0.98, 1.00, 0.94.	89
7.16	ETTh1: RL-MCF across 5 forecasting architectures.	89
7.17	Qualitative examples on ETTh2. Per-sample validity: 0.96, 0.88, 0.69, 0.17.	90
7.18	ETTh2: RL-MCF across 5 forecasting architectures.	90
7.19	Qualitative examples on Weather. Per-sample validity: 1.00, 1.00, 1.00, 1.00.	91
7.20	Weather: RL-MCF across 5 forecasting architectures.	91

List of Tables

2.1	Comparison of Deep Learning Architectures for Time Series	16
2.2	Comparative analysis of Transformer architectures for time series forecasting.	22
3.1	Comparative overview of XAI methods for time series.	36
6.1	Libraries and frameworks used throughout the project.	60
6.2	Summary of datasets used in the experiments.	60
6.3	Synthesis of STL decomposition results and modeling implications.	63
6.4	Dataset split sizes after windowing ($L = 96$, $H = 48$).	64
6.5	Summary of the five forecasting architectures trained in this work.	65
6.6	Observation window and forecasting horizon for each dataset.	66
6.7	Shared training configuration for all forecasting models.	67
6.8	Anomaly detector configuration.	71
6.9	RL-MCF configuration for ETTh1 with iTransformer.	74
6.10	Grid search results on ETTh1 (RL counterfactual generation).	75
7.1	iTransformer forecasting performance on the test set.	78
7.2	TCN autoencoder reconstruction performance on the test set.	79
7.3	RL-MCF results on ETTh1 (iTransformer backbone).	80
7.4	Impact of w_{val} ($w_{\text{prox}} = 1.5$ fixed, ETTh1).	82
7.5	Impact of w_{prox} ($w_{\text{val}} = 3.5$ fixed, ETTh1).	82
7.6	Quantitative comparison across datasets, methods and forecasting architectures. Bold = best per column per dataset. $\downarrow$ lower is better (Prox., Plaus.); $\uparrow$ higher is better (Valid., AUC, Comp., T-Cons.).	83
7.7	Performance of RL-MCF across datasets and forecasting architectures. Bold indicates best per dataset.	86
7.8	Ablation study: quantitative comparison on ETTh1 (iTransformer backbone). Best values in bold.	88

List of Algorithms

1	RL-MCF: Training Procedure	55
2	RL-MCF: Inference Procedure	55

List of Abbreviations

XAI	Explainable Artificial Intelligence
RL	Reinforcement Learning
TCN	Temporal Convolutional Network
MLP	Multilayer Perceptron
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
CNN	Convolutional Neural Network
GNN	Graph Neural Network
VAE	Variational Autoencoder
GAN	Generative Adversarial Network
SHAP	SHapley Additive exPlanations
LRP	Layer-wise Relevance Propagation
LIME	Local Interpretable Model-agnostic Explanations
STL	Seasonal-Trend decomposition using Loess
ACF	Autocorrelation Function
IF	Isolation Forest
LOF	Local Outlier Factor
AUC	Area Under the Curve
MSE	Mean Squared Error
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
TS	Time Series
CF	Counterfactual
AE	AutoEncoder

RevIN Reversible Instance Normalization

ARIMA AutoRegressive Integrated Moving Average

DFT Discrete Fourier Transform

STFT Short-Time Fourier Transform

RL-MCF Reinforcement Learning Masked Counterfactual Explanations For Time Series Forecasting

General Introduction

Time series data are at the heart of modern science and industry. From electricity consumption and financial markets to patient vital signs and climate variables, a vast number of real-world systems produce sequences of observations that evolve over time sequences whose accurate forecasting carries direct operational and economic consequences.

For decades, this challenge was addressed by classical statistical models ARIMA, exponential smoothing, and their variants built on explicit assumptions of linearity and stationarity. These methods are interpretable and well understood, but they were designed for simple, well-behaved series. Real-world temporal data is noisy, non-stationary, high-dimensional, and governed by complex nonlinear dynamics that no linear model can adequately capture.

The rise of deep learning fundamentally changed what is achievable. Architectures such as Temporal Convolutional Networks and Transformer-based models Informer, PatchTST, iTransformer learn hierarchical temporal representations directly from data, capturing long-range dependencies and multivariate interactions that classical methods systematically miss. These models now define the state of the art on every major forecasting benchmark. Yet this gain in predictive power comes at a fundamental cost: these models operate as black boxes, producing forecasts without any transparent justification. In high-stakes environments, a prediction that cannot be explained cannot be trusted, and a prediction that cannot be trusted cannot be acted upon.

Explainable AI (XAI) attempts to restore this transparency. Existing methods SHAP, LRP, attention-based approaches identify which features or timesteps influenced a prediction. However, these descriptive explanations remain incomplete for practitioners: knowing that a timestep was important does not tell an operator what to do. The actionable question is different: *what minimal, realistic modification to the observed input would have been sufficient to produce a different, more desirable forecast?* This is the question that counterfactual explanations are designed to answer and it is the question this dissertation addresses.

Generating valid, plausible, and compact counterfactual explanations for deep time series forecasting models is far from straightforward, and no existing method simultaneously satisfies these requirements in the forecasting setting. This gap is the central motivation of this work.

Structure of the dissertation. This dissertation is organised into two parts. Part I surveys the fundamental concepts and related work necessary to understand and position our contribution. Chapter 1 introduces the structural properties of time series data — components, decomposition methods, temporal dependencies, and the mutable/immutable distinction — alongside classical modelling approaches, whose limited capacity to capture complex non-linear patterns motivates the shift towards deep learning, discussed in Chapter 2. The latter traces the evolution of deep learning architectures for time series, from foundational families (MLPs, RNNs, CNNs, GNNs) to advanced Transformer-based models, which have yielded state-of-the-art forecasting performance at the cost of opacity — an inherent black-box nature that motivates the need for dedicated explainability methods, addressed in Chapter 3. The latter surveys the landscape of Explainable AI through a multi-criteria taxonomy, revealing that virtually all

existing approaches remain strictly descriptive and fall short of supporting actionable decision-making — a limitation that motivates a turn towards counterfactual explanations, examined in depth in Chapter 4. The latter covers their formulation, generation methods, evaluation criteria, and open challenges, culminating in a precise identification of five nested research gaps that no existing method simultaneously addresses. Part II presents our contribution. Chapter 5 introduces RL-MCF (Reinforcement Learning for Counterfactual Explanations in Time Series Forecasting), a principled and model-agnostic framework that addresses each identified limitation through three complementary components: a TCN-based autoencoder that confines perturbations to the learned data manifold, a temporal masking mechanism that enforces sparse and localized modifications, and an actor-critic agent that learns an amortized counterfactual policy enabling single-pass inference. Chapter 6 details the full implementation of the framework across three benchmark datasets — ETTh1, ETTh2, and Weather — covering the independent training and validation of each pipeline component, the hyperparameter selection procedure, and complete reproduction guidelines. Chapter 7 presents the empirical validation: RL-MCF is evaluated against three baselines of increasing sophistication, across five forecasting architectures of fundamentally different designs, and subjected to a structured ablation study isolating the contribution of each design choice. RL-MCF consistently outperforms the gradient-based baseline ForecastCF on all evaluation dimensions, achieving a validity ratio of up to 0.976, a compactness score of 0.803 versus 0.013 for the baseline, and a plausibility score of 0.170 versus 0.642, confirming that latent-space generation produces significantly more realistic counterfactuals — while generating them in a constant 2 ms via a single forward pass, up to $3000\times$ faster than gradient-based baselines. An ablation study validates each design choice independently. The dissertation closes with a General Conclusion that summarises the contributions, discusses the limitations of the current framework, and outlines promising directions for future work, including extension to multivariate settings, adaptive temporal masking, and integration of domain knowledge constraints.

Part I

State of the art

Chapter 1

Time Series

1.1 Introduction

Time series data are central to a wide range of scientific and industrial domains, from energy management and financial forecasting to healthcare monitoring and climate analysis. Understanding, modelling, and forecasting such data requires a careful account of the structural properties that make temporal data fundamentally different from tabular or image data.

This chapter lays the conceptual groundwork for the rest of the dissertation. It opens with the classical foundations of time series analysis: definitions, components, decomposition methods, and the statistical forecasting models that have long dominated the field. Their limitations are then examined, revealing why modern deep learning architectures became necessary. Building on this, the chapter introduces deeper structural constraints of real-world series that classical methods fail to capture, and that any model operating on temporal data must respect.

1.2 Definition

A time series is a sequence of observations collected over time, typically at regular intervals such as hourly or daily. As defined by Alsudani and J. Liu (2017), it captures the temporal evolution of one or more variables, and its analysis aims to model, understand, and forecast that evolution. Unlike tabular data, where observations are assumed to be independent, time series observations are intrinsically ordered and statistically dependent on one another. This temporal structure is not a nuisance to be corrected — it is the defining characteristic of the data, and any model must account for it.

1.3 Types of Time Series

As discussed by Chudo and Terdik (2025), time series can be classified according to their dimensionality:

- **Univariate time series:** a single variable is measured over time. The analysis focuses on its internal temporal structure — trend, seasonality, and autocorrelation.
- **Multivariate time series:** multiple variables are tracked simultaneously. Beyond the temporal structure of each individual series, the analysis must also account for cross-variable dependencies, which may be nonlinear, time-varying, or lagged.

1.4 Components of Time Series

Time series data are commonly decomposed into constituent components that reveal the underlying structure of the signal :

- **Trend:** the long-term movement of the series, reflecting a persistent increase or decrease over time (yellow line in the figure 1.1).
- **Seasonality:** repeating patterns at fixed, known intervals, such as daily or yearly cycles (green line).
- **Residual (Noise):** the random variation that remains after the trend and seasonal components have been accounted for (red line).

Figure 1.1 illustrates the three components on a representative series. Understanding these components

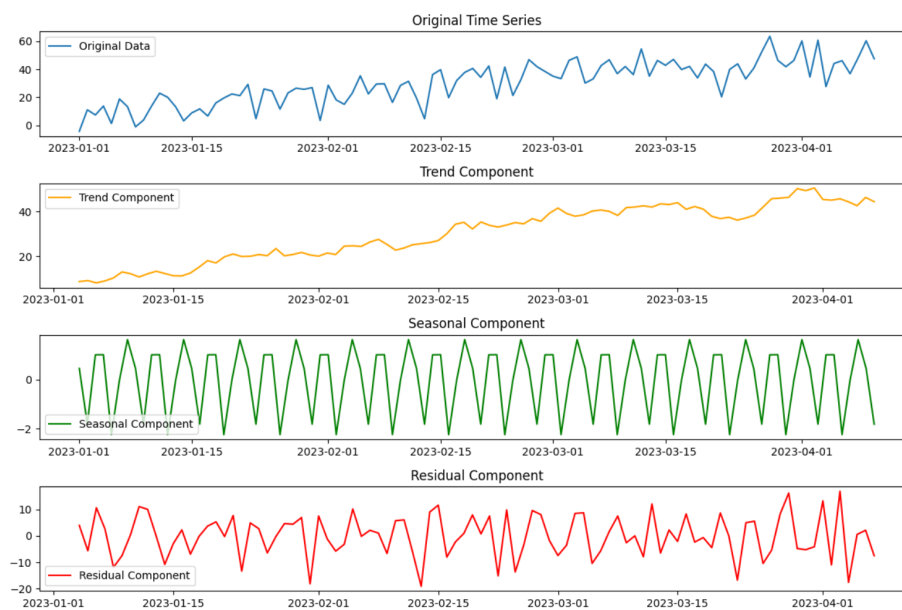


Figure 1.1: Time Series Components (from Google Images)

is the first step toward selecting appropriate modeling strategies, as different forecasting methods are designed to capture different subsets of these patterns.

1.5 Time Series Decomposition Methods

Time series decomposition separates a series into its trend, seasonal, and residual components. Two classical approaches are widely used: additive and multiplicative decomposition. The choice between them depends on whether seasonal fluctuations remain approximately constant in magnitude or scale proportionally with the level of the series.

In **additive decomposition**, components combine as:

$$y_t = T_t + S_t + R_t$$

A well-known implementation of additive decomposition is **STL** (Seasonal and Trend decomposition using Loess), which estimates each component using locally weighted regression (Loess). Unlike classical

additive decomposition, STL handles any seasonality period, is robust to outliers, and allows the seasonal component to evolve over time — making it particularly suitable for real-world series with irregular dynamics. In **multiplicative decomposition**, they combine as:

$$y_t = T_t \times S_t \times R_t$$

where T_t , S_t , and R_t denote the trend, seasonal, and residual components at time t , respectively. Additive models are appropriate when seasonal effects remain roughly constant regardless of the level of the series. As established by Kreuzer, Zdravkovic, and Papapetrou (2025), multiplicative models are preferred when the amplitude of seasonal fluctuations grows with the trend, as is common in economic or energy data.

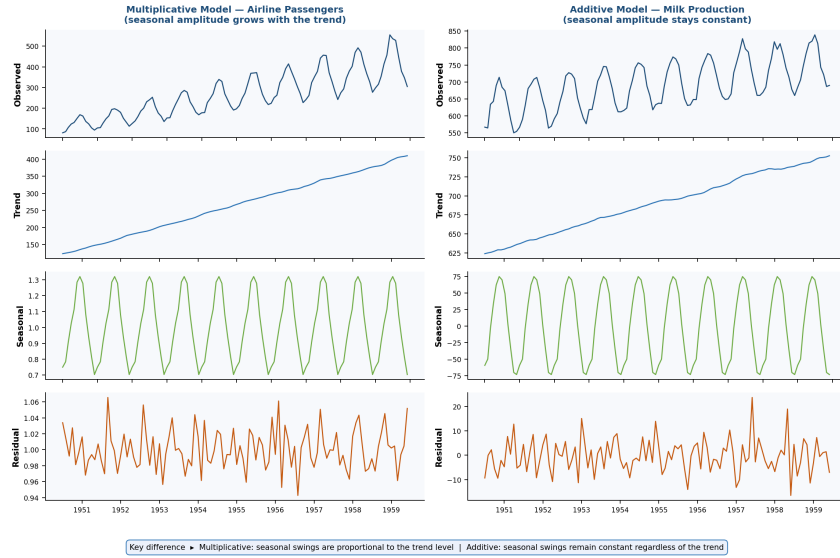


Figure 1.2: Comparison of Additive and Multiplicative Time Series Decomposition(from Google Images).

As shown in Figure 1.2, the additive model maintains a constant seasonal amplitude, while the multiplicative model scales it with the trend. These decomposition frameworks are foundational to time series analysis, and their logic is embedded as inductive biases in several deep learning architectures reviewed in Chapter 2

1.6 Objectives and Applications of Time Series Analysis

The analysis of time series serves three principal objectives in practice:

1. **Classification:** assigning an input sequence to a discrete category based on learned temporal patterns. Given a sequence $\mathbf{x} \in \mathbb{R}^L$, a classifier $g : \mathbb{R}^L \rightarrow \{1, \dots, C\}$ maps it to a label.
Example: classifying heart rate patterns as normal or arrhythmic, recognizing daily activity patterns from wearable sensors.
2. **Anomaly Detection:** identifying observations or subsequences that deviate significantly from the expected statistical structure of the series. A sequence is flagged as anomalous when it falls outside the distributional boundaries learned from normal behavior.
Example: detecting a sudden drop in server performance, spotting unusual spikes in bank transactions.

3. **Forecasting:** predicting future values from past observations. A forecasting model $f : \mathbb{R}^{L \times M} \rightarrow \mathbb{R}^{H \times M}$ maps a look-back window $\mathbf{x} = (x_{t-L+1}, \dots, x_t) \in \mathbb{R}^{L \times M}$ of length L over M variables to a predicted trajectory of H future values:

$$\hat{\mathbf{y}} = f(\mathbf{x}), \quad \hat{\mathbf{y}} \in \mathbb{R}^{H \times M}$$

where L is the look-back window length, H is the forecast horizon, and M is the number of input variables ($M = 1$ for univariate forecasting). *Example:* estimating next week’s electricity demand from historical usage, forecasting next quarter’s sales.

Among these objectives, forecasting is the most demanding. Unlike classification, which reduces a sequence to a single discrete label, forecasting requires the model to produce a coherent future trajectory — a continuous, multi-step output that must remain physically consistent with the observed past. This consistency is only possible because real-world series are not arbitrary sequences: they are constrained by the underlying physical process that generated them, a property examined in depth later in this chapter.

1.7 Classical Forecasting Methods

Before deep learning, time series forecasting was dominated by statistical models that rely on explicit assumptions about the data structure. The main families are:

- **Moving Average (MA):** smooths observations by averaging a fixed window of recent values, suppressing short-term noise. Effective only for series without strong trend or seasonality.
- **ARIMA(p, d, q):** combines autoregressive terms (dependence on past values), differencing (to enforce stationarity), and moving-average error terms. The dominant classical model for univariate forecasting.
- **SARIMA(p, d, q)(P, D, Q) $_s$:** extends ARIMA with seasonal autoregressive, integration, and moving-average terms at period s , handling recurring seasonal cycles.
- **Exponential Smoothing (SES, Holt, Holt-Winters):** produces forecasts as exponentially weighted averages of past observations. Holt-Winters additionally models trend and seasonality explicitly.

Stationarity: A Prerequisite for Classical Methods

Classical forecasting methods cannot operate on raw time series data without a key condition being met: **stationarity:** a time series is weakly stationary if its mean, variance, and autocovariance are time-invariant:

$$\mathbb{E}(x_t) = \mu, \quad \text{Var}(x_t) = \sigma^2, \quad \text{Cov}(x_t, x_{t-\tau}) = c_\tau$$

If statistical properties drift over time, past behaviour provides an unreliable basis for prediction. Classical models such as ARIMA and exponential smoothing are only valid when applied to a stationary series — or one that has been transformed into one. Real-world series are rarely stationary. Standard remedies include:

- **Differencing:** $Y'_t = Y_t - Y_{t-1}$ removes linear trends by computing consecutive differences.
- **Detrending:** subtracting a fitted polynomial function of time from the series.

- **Variance-stabilising transformations:** logarithmic or square-root scaling to stabilise heteroscedastic variance.

Figure 1.3 illustrates the contrast between a stationary and a non-stationary series. Once stationarised,

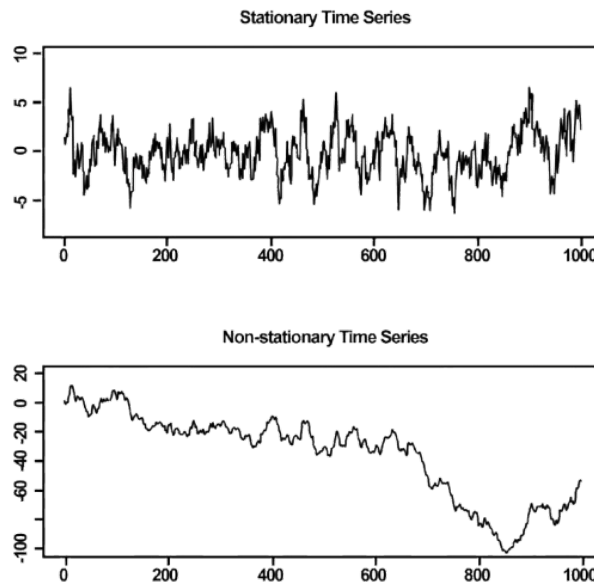


Figure 1.3: Stationary vs. non-stationary time series (from Google Images).

the series is modelled using the appropriate classical method, and a back-transformation is applied to restore the original scale. While these approaches are interpretable and computationally inexpensive, they operate under two restrictive assumptions: linearity and stationarity. These assumptions make them ill-suited to the complex, high-dimensional, and non-stationary datasets encountered in modern forecasting tasks, as discussed in the following section.

1.8 Traditional Time Series Analysis Pipeline

The classical analysis pipeline organises the steps described above into a structured sequence, illustrated in Figure 1.4:

1. **Exploration / Inspection:** visual inspection of the series using time plots and summary statistics such as mean, variance, and autocorrelation. This step identifies the presence of trend, seasonality, and potential anomalies, and informs all subsequent modelling choices.
2. **Decomposition:** the series is separated into its structural components — trend, seasonal, and residual — using either additive or multiplicative decomposition, depending on the characteristics identified in the inspection step.
3. **Stationarity Check and Transformation:** formal tests such as the Augmented Dickey-Fuller test, introduced by Dickey and Fuller (1979), are applied to verify stationarity.
4. **Modelling and Forecasting:** a suitable forecasting model is selected and fitted to the transformed series. The fitted model is then used to generate predictions for future time points, and the output is back-transformed to the original scale.

This pipeline is interpretable, computationally inexpensive, and well understood theoretically. However, it treats time series as simple mathematical sequences governed by linear and stationary dynamics,

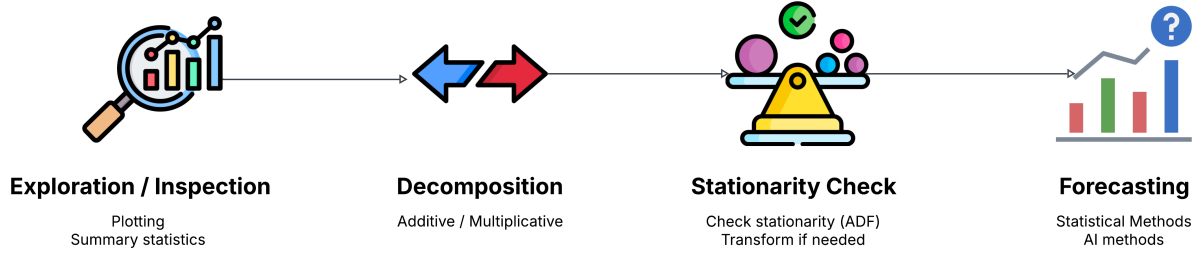


Figure 1.4: Time Series Analysis Pipeline.

rather than as realisations of complex physical systems. As a consequence, it fails to capture the non-linear, long-range, and multivariate dependencies that characterise real-world data. These limitations motivate the shift to deep learning, examined in Chapter 2. Before that, however, it is necessary to understand the deeper structural properties of time series data that any model classical or deep must respect.

1.9 The Temporal Data Manifold

Time series generated by real-world physical systems do not occupy the full observation space $\mathbb{R}^L$ arbitrarily. As noted by Rojat et al. (2021), they reside on a structured subspace — defined by the latent dynamics, transition laws, and statistical dependencies of the underlying generative process referred to as the **temporal data manifold** $\mathcal{M}$.

$$\mathbf{x} \in \mathcal{M} \subset \mathbb{R}^L, \quad \dim(\mathcal{M}) \ll L \quad (1.1)$$

This low-dimensional structure imposes two critical constraints on any meaningful manipulation of a series, as illustrated in Figure 1.5:

- **Temporal coherence.** Each observation x_t is intrinsically coupled with its predecessors and successors through the autocorrelation structure of the process. A modified sequence that ignores this coupling may be mathematically feasible yet physically nonsensical it leaves $\mathcal{M}$.
- **Structural plausibility.** A valid series must respect the distributional boundaries of $\mathcal{M}$: its amplitude, variance, and local dynamics must remain consistent with the patterns observed in the training distribution.

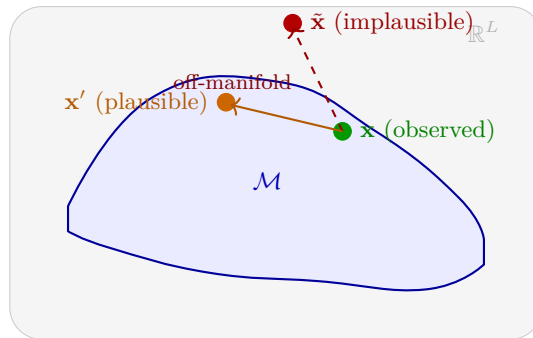


Figure 1.5: The temporal data manifold $\mathcal{M} \subset \mathbb{R}^L$.

These constraints have a direct practical consequence: any method that generates a new series from an observed one must ensure that the result remains on $\mathcal{M}$. Methods that operate directly in the raw input

space $\mathbb{R}^L$ provide no such guarantee, frequently producing sequences that are temporally incoherent and that domain experts cannot interpret or act upon. This confinement to $\mathcal{M}$ has a concrete manifestation in the data: observations are not independent. The nature of these statistical links and the scales at which they operate is what the following section examines.

1.10 Temporal Dependencies

A defining characteristic of time series is that observations are not statistically independent: x_t depends on its past. This dependence is quantified by the autocorrelation function:

$$\rho(\tau) = \frac{\text{Cov}(x_t, x_{t-\tau})}{\text{Var}(x_t)}, \quad \tau = 1, 2, \dots \quad (1.2)$$

Real-world series exhibit dependencies at multiple temporal scales simultaneously, as characterized by Y. Liu et al. (2023):

- **Short-term dependencies** ($\tau \leq$ a few steps): local continuity and smooth transitions between consecutive observations. Any realistic series must preserve these.
- **Long-term dependencies** ($\tau \gg 1$): periodic patterns such as daily and weekly cycles in energy data, sustained trends, and regime changes spanning dozens or hundreds of timesteps.
- **Inter-variate dependencies** (multivariate series): cross-channel correlation structures — for instance, the relationship between load and temperature in the ETTh datasets — that must be modelled jointly across all variates.

These multi-scale dependencies reveal an asymmetry that is unique to time series and absent from tabular or image domains: within a look-back window, not all observations carry the same practical status with respect to what can be known or changed.

1.11 Mutable and Immutable Observations

A critical distinction in time series one that has no direct equivalent in tabular or image domains concerns the **actionability** of individual observations within a look-back window $\mathbf{x} = (x_1, \dots, x_L)$, as illustrated in Figure 1.6:

- **Historically fixed observations** $x_1, \dots, x_{L-k}$ record events that have already occurred. They are *immutable* in an operational sense: no present action can alter them. As noted by Z. Wang et al. (2023), a counterfactual that modifies these positions may remain statistically plausible, but describes an intervention that is no longer realisable and therefore not actionable.
- **Recent observations** $x_{L-k+1}, \dots, x_L$ correspond to the immediate past, where an operator or system may still intervene. These are *mutable*: modifying them describes a scenario that is both physically realisable and temporally consistent with the causal structure of the process.

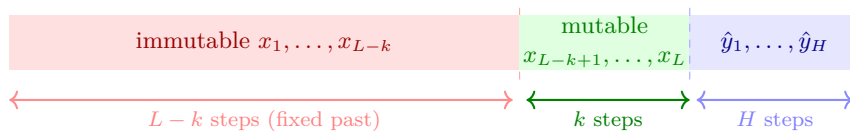


Figure 1.6: Mutable and immutable regions of a look-back window of length L .

This distinction has a direct implication for how a meaningful alternative series should be constructed: modifications must be **sparse** (few timesteps altered) and **localised** (concentrated in the recent, mutable region). A modified series that satisfies these two properties is both physically interpretable and practically useful — it prescribes precisely what to change and when, without altering a past that can no longer be modified.

1.12 Conclusion

Time series analysis has evolved from rigid statistical frameworks toward a richer and more principled understanding of temporal dynamics. The classical pipeline decomposition, stationarity transformation, and statistical modelling remains a useful reference, but its assumptions are too restrictive for the complexity of modern data.

The structural properties introduced in the second part of this chapter manifold confinement, multi-scale temporal dependencies, and the mutable/immutable distinction go beyond what classical methods can capture or enforce. They define the deeper constraints that any model operating on temporal data must respect, and that will guide the design choices presented throughout this dissertation.

The following chapter examines how deep learning architectures address the modelling limitations of classical methods, by learning hierarchical temporal representations directly from data.

Chapter 2

Deep Learning for Time Series

2.1 Introduction

Following the evolution of time series analysis from classical statistical foundations, recent years have witnessed a major paradigm shift toward deep learning-based approaches. Traditional models such as ARIMA, Exponential Smoothing, and VAR rely on assumptions of linearity and stationarity, which limit their ability to capture the complex, nonlinear, and long-range dependencies characteristic of real-world time series. Deep learning models overcome these limitations by learning hierarchical and expressive temporal representations directly from data, making them particularly suitable for high-dimensional, noisy, and non-stationary sequences.

This chapter summarizes key advances in deep learning for time series, tracing the evolution from classical families MLPs, RNNs, CNNs, and GNNs to advanced Transformer-based architectures. While each family offers distinct trade-offs, classical architectures exhibit significant limitations with complex, long-range, and multivariate data. Transformer-based models address many of these shortcomings through self-attention and parallel processing, becoming the dominant paradigm for modern time series tasks. Nevertheless, interpretability the ability to understand and trust model predictions remains a critical shared challenge, especially in high-stakes domains such as healthcare, finance, and energy forecasting.

2.2 Challenges in Time Series Modeling

As identified by Y. Wang et al. (2024), time series data present two fundamental challenges. First, **temporal dependencies**: capturing multi-scale dynamics across short-term patterns, long-term trends, and irregular fluctuations is difficult, especially for long sequences where conventional models suffer from vanishing gradients or limited receptive fields. Second, **variate correlations**: in multivariate settings, interactions among variables may be nonlinear, time-varying, or misaligned, requiring expressive mechanisms beyond simple linear assumptions.

2.3 Classical Deep Learning Architectures for Time Series

Deep learning has introduced a wide range of architectures capable of modeling the nonlinear, multi-scale, and high-dimensional characteristics of time series data. According to the survey in Y. Wang et al. (2024), from which the following descriptions and figures are drawn, existing approaches can be grouped into four major architectural families :

2.3.1 Multilayer Perceptrons (MLPs)

MLP-based models treat forecasting as direct mappings from historical inputs to future outputs. Although simple, they are highly efficient and surprisingly competitive for many benchmarks. Models such as N-BEATS and N-HiTS use deep fully connected stacks and basis expansions to capture multi-scale temporal patterns, while lightweight architectures like DLinear demonstrate strong performance even with linear layers. The Figure 2.1 illustrates how an MLP processes time-series inputs to generate predictions.

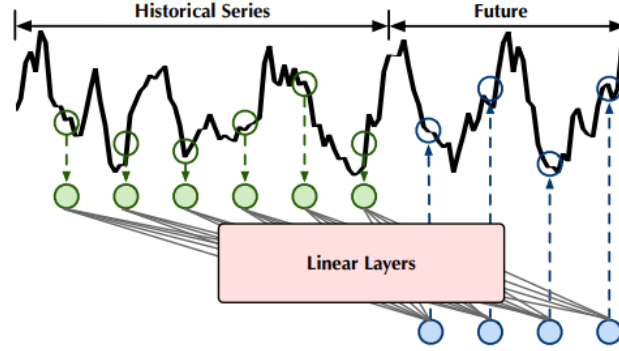


Figure 2.1: MLP Architecture for Time Series

However, MLPs lack inherent mechanisms to model long-range dependencies, requiring fixed-length input windows and treating time steps as independent features rather than sequential observations.

2.3.2 Recurrent Neural Networks (RNNs)

RNNs, including LSTM and GRU variants, naturally model sequential dependencies by maintaining hidden states over time. They were early dominant architectures in time series forecasting, imputation, and generative modeling. Extensions such as DeepAR, recurrent-skip connections (LSTNet), and state-space models (e.g., S4, Mamba) improve long-range modeling and temporal stability. The Figure 2.2 provides an illustration of how an RNN consumes a time series sequentially to update hidden representations and produce predictions.

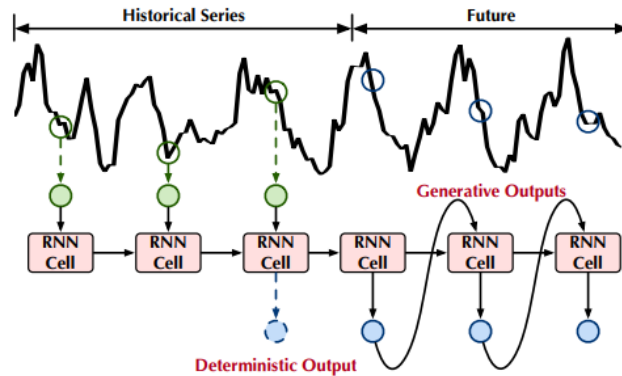


Figure 2.2: RNN Architecture for Time Series

However, RNNs suffer from vanishing gradient problems in very long sequences, inherently sequential computation that prevents parallelization during training, and difficulty capturing long-range dependencies.

2.3.3 Convolutional Neural Networks (CNNs)

CNN-based models capture local temporal patterns efficiently through 1D convolutions and dilated filters. They scale well to long sequences and large datasets. Architectures such as TCN, WaveNet, SCINet, and TimesNet enhance temporal modeling using hierarchical, dilated, or multi-period convolutional designs. The Figure 2.3 illustrates the principle of applying temporal convolutions to a time series.

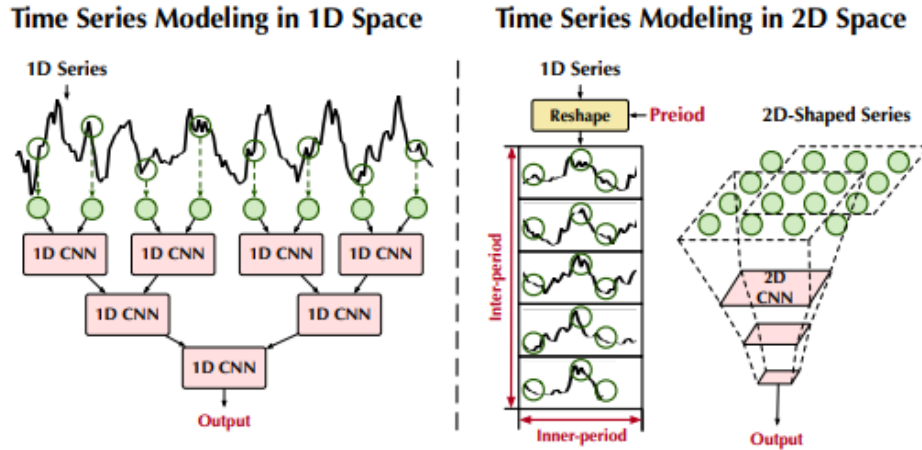


Figure 2.3: CNN Architecture for Time Series

Among CNN-based architectures, **Temporal Convolutional Networks (TCN)** Wan et al. (2019) have emerged as a particularly robust variant, overcoming the limitations of RNNs through three core mechanisms: **causal convolutions**, which ensure that the output at time t depends strictly on inputs from 0 to t , respecting the arrow of time; **dilated convolutions**, which grow the receptive field exponentially with network depth without increasing parameters; and **residual connections**, which stabilize deep network training and prevent information degradation. Unlike RNNs, TCNs process long sequences in parallel during training, combining a stable gradient flow with a large effective receptive field. The architectural details are illustrated in Figure 2.4.

However, CNNs in general — including TCNs — have limited receptive fields that require deep stacking or large dilation rates to capture very long-range dependencies, and their fixed kernel sizes may not adapt well to time series with varying temporal scales or irregular sampling patterns.

2.3.4 Graph Neural Networks (GNNs)

GNNs are designed for multivariate time series where variables exhibit structured or latent relationships. They model interactions among variables through graph edges while learning temporal evolution at each node. Examples include DCRNN, STGCN, Graph WaveNet, AGCRN, and StemGNN, which combine graph convolutions with recurrent or spectral components. The Figure 2.5 shows how a time series with multiple correlated variables can be structured as a graph for spatiotemporal modeling.

However, GNNs require predefined or learned graph structures that may not generalize across different datasets or domains, and they primarily focus on spatial relationships among variables rather than capturing complex temporal patterns within individual series.

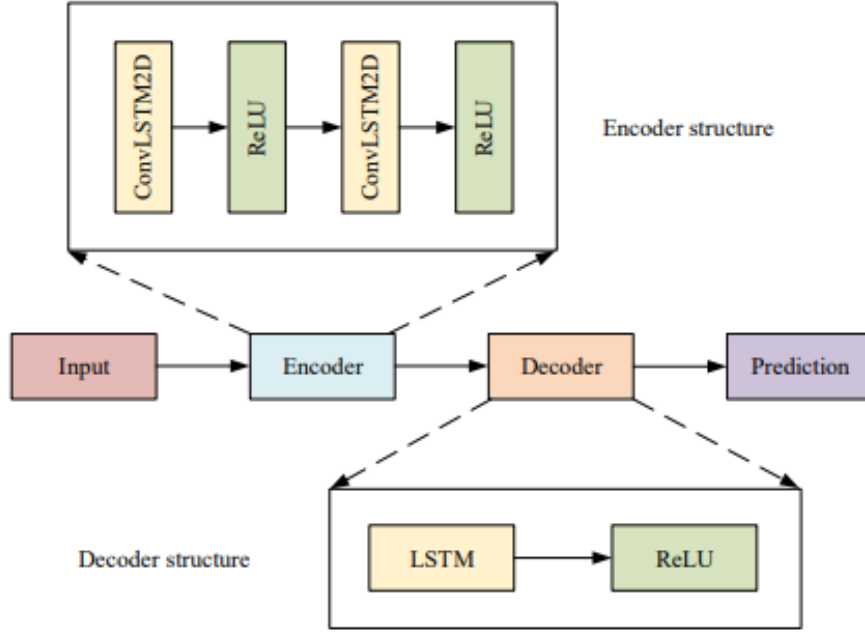


Figure 2.4: TCN architectural details Wan et al. (2019).

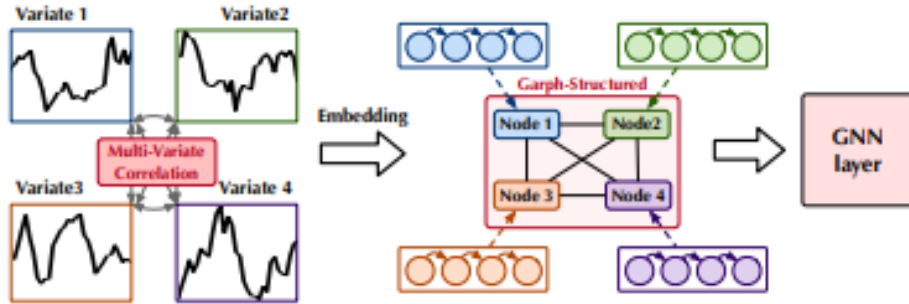


Figure 2.5: GNN Architecture for Time Series

Architectural Trade-offs

Table 2.1 summarizes the key trade-offs across these architectural families, synthesized from Y. Wang et al. (2024).

While MLPs, RNNs, CNNs, and GNNs offer specific advantages for time series modeling, they exhibit significant limitations when applied to complex, multivariate, and long-range temporal data. These observations highlight that simpler architectures alone are insufficient for capturing the intricate patterns present in real-world time series. **Transformers, with their attention mechanisms and parallel processing capabilities, provide the flexibility and expressive power necessary for modern time series tasks.**

Architecture	Advantages	Limitations	Examples
MLP	Simple, fast, effective for short and linear series	Cannot capture temporal order, relies on fixed input window	N-BEATS, N-HiTS, DLinear
RNN / LSTM / GRU	Sequential modeling, captures local and long-term dependencies	Vanishing gradients, sequential $\rightarrow$ slow training, hard to parallelize	DeepAR, LSTNet
CNN / TCN	Captures local patterns, parallelizable, efficient for long series	Limited receptive field, requires deep stacking for long-term dependencies	WaveNet, SCINet, TimesNet, TCN
GNN	Models inter-variable relationships, suitable for multivariate series	Requires a graph structure, less effective for intra-series temporal patterns	DCRNN, STGCN, AGCRN

Table 2.1: Comparison of Deep Learning Architectures for Time Series

2.4 Advanced Architectures: Transformers for Time Series

2.4.1 Transformer Architecture

The Transformer architecture, introduced by Vaswani et al. (2017), is a sequence modeling framework that relies entirely on attention mechanisms, replacing recurrent and convolutional networks. Its ability to process sequences in parallel and capture long-range dependencies makes it particularly suitable for time series forecasting.

The main components of a Transformer layer are: **Self-Attention**, which computes relevance scores between all positions in the sequence; **Multi-Head Attention**, which runs multiple attention heads in parallel to capture different types of relationships; **Positional Encoding**, which injects order information since Transformers have no inherent sequential bias; **Feed-Forward Networks (FFN)**, applied position-wise for nonlinear transformation; and **Residual Connections with Layer Normalization** to stabilize training and facilitate gradient flow. The overall Transformer architecture is depicted in Figure 2.6.

The core operation is the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.1)$$

where Q, K, V are the query, key, and value matrices, and d_k is the key dimension used for numerical stability. Multi-head attention extends this by running h parallel attention heads over different projection subspaces:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2.2)$$

This mechanism allows the model to simultaneously attend to different aspects of the sequence, making it powerful for capturing both local and long-range temporal dependencies.

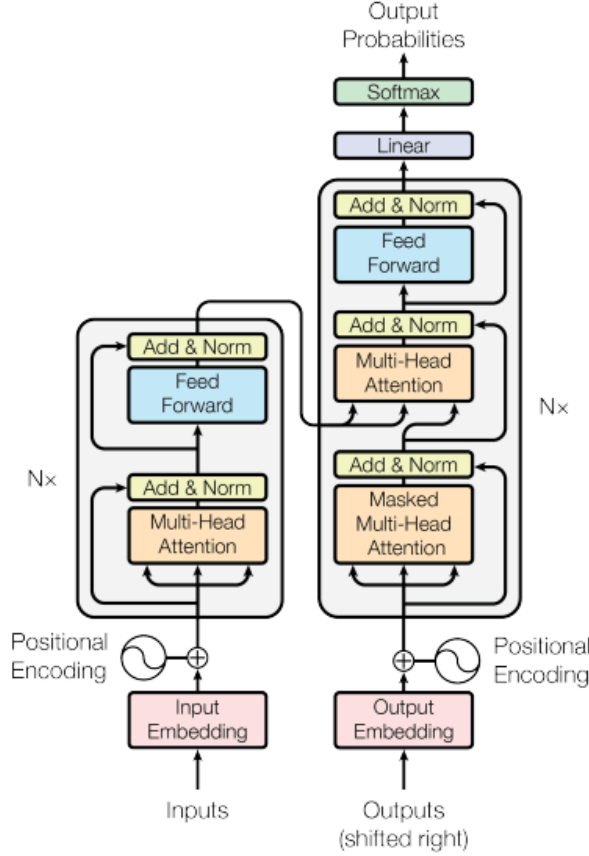


Figure 2.6: The Transformer architecture Vaswani et al. (2017).

2.4.2 Tokenization Strategies in Transformer-Based Time Series Models

A key design choice in Transformer-based time series models is how the input sequence is divided into tokens fed to the attention mechanism. As illustrated in Figure 2.7, three main strategies exist:

- **Point-wise tokens:** each time step is a separate token. Used by the vanilla Transformer and Informer — captures fine-grained temporal correlations but scales poorly with sequence length.
- **Patch-wise tokens:** contiguous segments of each variate form a token. Used by PatchTST and Crossformer — reduces the number of tokens while preserving local temporal structure.
- **Series-wise tokens:** the entire sequence of a single variate forms one token. Used by iTransformer — attention operates across variates rather than time steps, making it particularly effective for multivariate forecasting.

This tokenization choice directly determines what relationships the model can learn: point-wise captures temporal order, patch-wise balances local and global patterns, and series-wise prioritises inter-variate dependencies.

2.5 Advanced Transformer Architectures for Long-Term Time Series Forecasting

Building on the standard Transformer framework, several specialized architectures have been proposed to address its scalability and accuracy limitations for long-term time series forecasting, as reviewed by

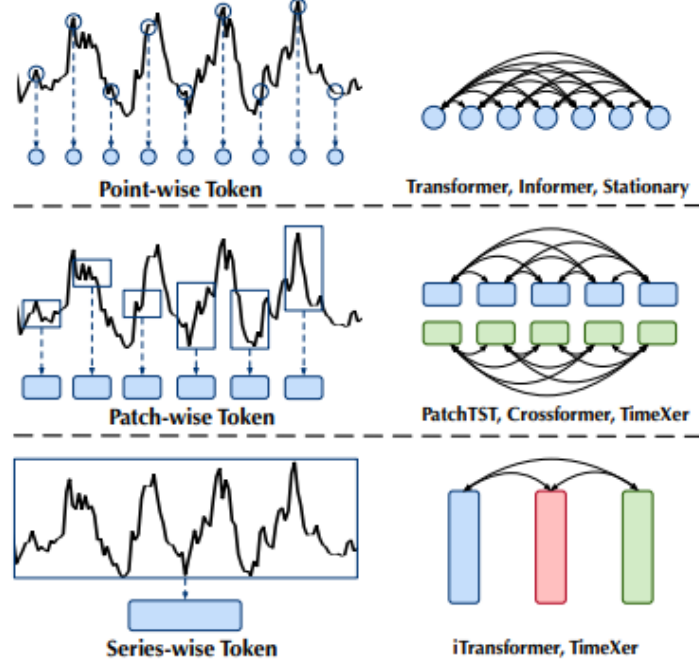


Figure 2.7: Tokenization strategies for Transformer-based time series models.

Wen et al. (2022). Each variant targets a specific bottleneck computational complexity, seasonality modeling, frequency-domain representation, or multivariate correlation and proposes a targeted architectural innovation. The following sections survey the most representative models.

2.5.1 Informer

H. Zhou et al. (2021) tackles the quadratic complexity of standard self-attention by introducing a ProbSparse mechanism that retains only the most informative queries, reducing complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$. A distilling operation further compresses encoder representations across layers, and a generative decoder predicts the full horizon in a single forward pass, avoiding error accumulation typical of auto-regressive decoding. Figure 2.8 summarises the overall design.

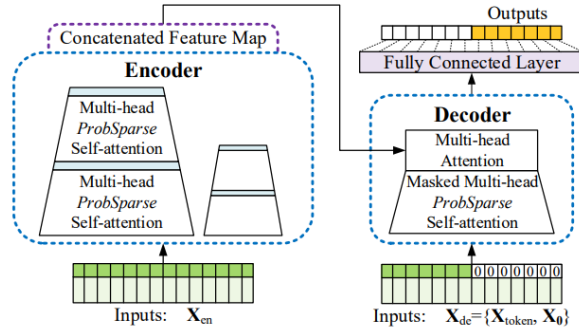


Figure 2.8: Informer architecture H. Zhou et al. (2021).

Best suited for This architecture is particularly suited for series exhibiting long sequences with point-wise temporal correlations, where computational efficiency is the primary bottleneck.

2.5.2 Autoformer

Wu et al. (2021) replaces standard attention with an auto-correlation mechanism that identifies repeating periodic patterns by measuring similarity between a series and its lagged versions. A series decomposition block explicitly separates the input into trend and seasonal components at each encoder and decoder layer, allowing the model to handle long-range dependencies while preserving temporal structure. Figure 2.9 provides an overview of the model.

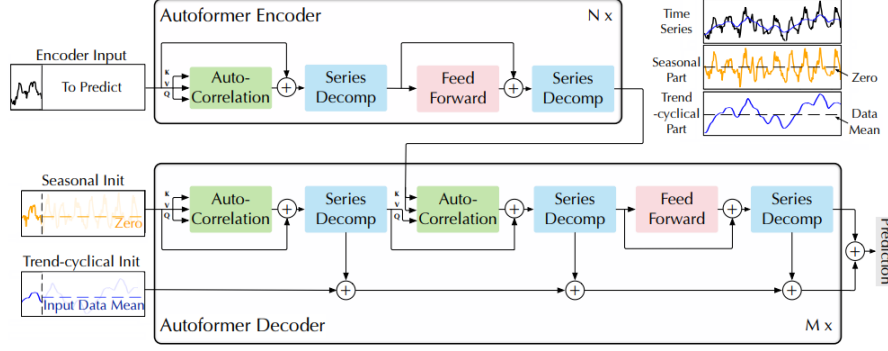


Figure 2.9: Autoformer architecture Wu et al. (2021).

Best suited for Its decomposition block makes it naturally adapted to series with strong seasonal and trend components, as introduced in Section 1.5.

2.5.3 FEDformer

T. Zhou et al. (2022) operates in the frequency domain rather than the time domain. Frequency Enhanced Blocks replace self-attention with Fourier or Wavelet transforms over a randomly selected subset of frequency modes, achieving linear complexity $\mathcal{O}(L)$. A Mixture-of-Experts decomposition block adaptively extracts multi-scale trend components, complementing the frequency-domain seasonal modeling. The overall design is illustrated in Figure 2.10.

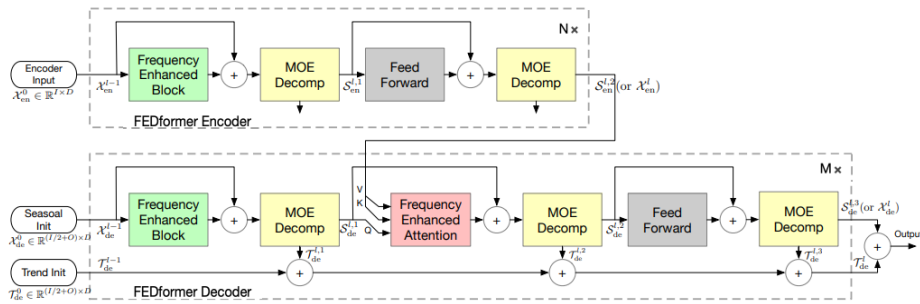


Figure 2.10: FEDformer architecture T. Zhou et al. (2022).

Best suited for This design is most effective when the series exhibits dominant periodic patterns in the frequency domain, such as strong seasonality with stable amplitude.

2.5.4 PatchTST

Nie et al. (2023) introduces a patch-based tokenization strategy that fundamentally rethinks how time series data is fed into the Transformer. Instead of treating each individual time step as a token, PatchTST

divides each univariate time series into non-overlapping or overlapping *patches* (contiguous segments) that serve as tokens. This design reduces the number of tokens by a factor equal to the patch length, dramatically lowering memory and computational requirements.

Two key design principles distinguish PatchTST. First, **channel independence**: each variate is processed independently by the same Transformer, allowing the model to capture intra-series temporal patterns without interference from other variables. Second, **patch-level semantic richness**: each token represents a local temporal segment rather than a single point, providing the attention mechanism with richer contextual information per token. The architecture is depicted in Figure 2.11.

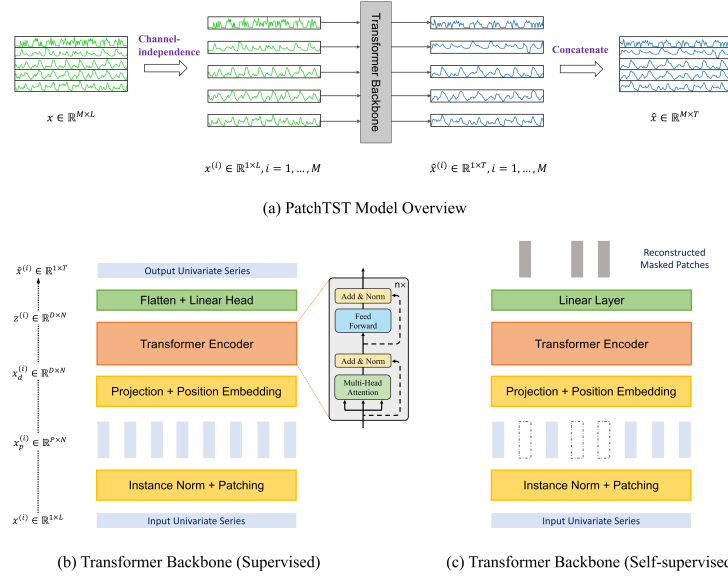


Figure 2.11: PatchTST architecture Nie et al. (2023).

Best suited for By grouping consecutive time steps into patches, this architecture is well-suited for univariate series with local temporal structure and long-range dependencies

2.5.5 Crossformer

Yunhao Zhang and J. Yan (2023) introduces a dual-attention mechanism that explicitly captures dependencies across both the *time dimension* and the *variate dimension* simultaneously. The **Cross-Time Stage** segments the input sequence into patches and applies self-attention within each variate to capture local and global temporal patterns. The **Cross-Dimension Stage** then applies attention across all variates at each patch position, modeling inter-variable correlations that evolve over time. Figure 2.12 illustrates this dual-attention design.

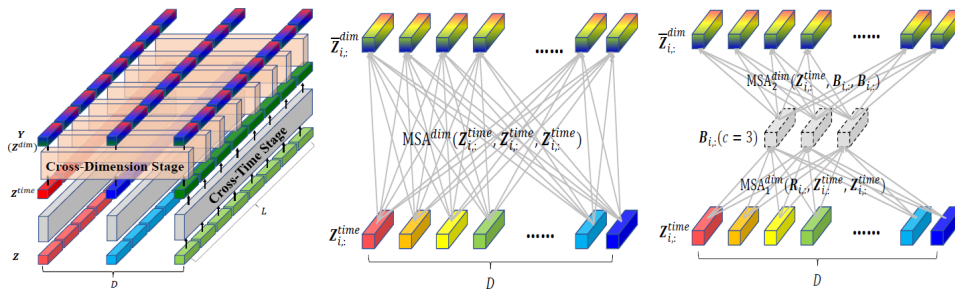


Figure 2.12: Crossformer architecture Yunhao Zhang and J. Yan (2023).

Best suited for This dual-attention design targets multivariate series where both temporal dynamics

and inter-variable correlations evolve jointly over time.

2.5.6 Inverted Transformer (iTransformer)

Y. Liu et al. (2023) proposed the *iTransformer*, an alternative formulation of Transformer-based models for time series forecasting. Unlike conventional Transformers that treat each timestamp as a token, iTransformer adopts an inverted representation where each variate is considered as a token. This design allows the self-attention mechanism to capture inter-variable correlations, while feed-forward networks model temporal dynamics within each variate. This simple yet effective reformulation achieves state-of-the-art results on multiple benchmarks, as illustrated in Figure 2.13.

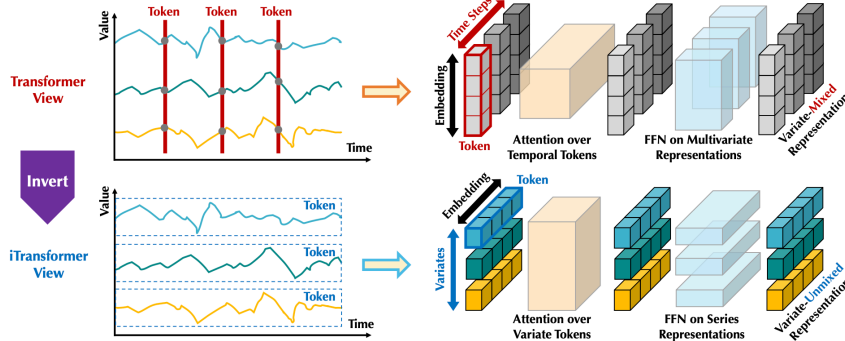


Figure 2.13: iTransformer architecture Y. Liu et al. (2023).

The inverted architecture manages to handle long-term dependencies much better than point-wise Transformers, which often suffer from performance degradation as the lookback window increases, positioning the iTransformer among the strongest forecasting architectures currently available for multivariate time series. Figure 2.14 summarises these gains across benchmarks.

Models	iTransformer (Ours)	RLinear (2023)	PatchTST (2023)	Crossformer (2023)	TiDE (2023)	TimesNet (2023)	DLinear (2023)	SCINet (2022)	FEDformer (2022)	Stationary (2022)	Autoformer (2021)
Metric	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE
Merchant	0.072 0.147	0.152 0.247	<u>0.084</u> <u>0.171</u>	0.117 0.181 0.187 0.289	0.093 0.184 0.110 0.206	0.316 0.385 0.296 0.401	0.172 0.277 0.494 0.531				
Wealth	0.345 0.289	0.585 0.461	0.394 0.326 0.429 0.288	0.595 0.481 0.360 0.318	0.501 0.412 0.660 0.514	0.625 0.543 0.499 0.415	0.772 0.612				
Finance	0.184 0.216	0.395 0.336 0.231 0.248	5.333 0.618 0.987 0.442	0.516 0.308 0.765 0.372	2.817 0.734 1.621 0.569	1.368 0.643 1.872 0.681					
Terminal	0.065 0.150	0.180 0.286 0.077 0.179 0.071 0.162	0.216 0.311 0.080 0.179 0.106 0.210	0.280 0.360 0.295 0.403	0.180 0.296 0.518 0.547						
Payment	0.072 0.144	0.143 0.245 0.084 0.174	0.207 0.179 0.208 0.278	0.105 0.182 0.116 0.200	0.288 0.322 0.300 0.373	0.166 0.271 0.417 0.460					
Customer	0.094 0.150	0.214 0.261 0.118 0.180	0.309 0.194 0.308 0.307	0.142 0.191 0.184 0.219	0.461 0.385 0.350 0.391	0.242 0.301 0.669 0.593					

Figure 2.14: Performance comparison of iTransformer against baseline models Y. Liu et al. (2023).

Best suited for This inverted formulation is most effective for high-dimensional multivariate series where cross-variable dependencies dominate over fine-grained temporal patterns.

2.6 Synthesis and Comparison

The six Transformer variants surveyed in this chapter each address a distinct bottleneck of the vanilla Transformer: computational efficiency (Informer), seasonal decomposition (Autoformer), frequency-domain modeling (FEDformer), patch-based tokenization (PatchTST), joint temporal and cross-variate attention (Crossformer), and inter-variate correlation (iTransformer). Table 2.2 synthesises these architectural choices across six evaluation criteria, from computational complexity and long-range modeling to practical guidance on when each variant is most appropriate.

Criteria	Transformer	Informer	Autoformer	FEDformer	PatchTST	Crossformer	iTransformer
Computational Complexity	$\mathcal{O}(N^2)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(L)$ (linear)	$\mathcal{O}(N/P)^2$ (P = patch size)	$\mathcal{O}(N \cdot M)$	$\mathcal{O}(M^2)$ (M = variates)
Long-range Modeling	Full self-attention (all-to-all)	ProbSparse attention (dominant queries)	Auto-correlation (periodic patterns)	Frequency-domain representation	Patch-level temporal attention	Dual attention: time + variate	Variate-wise attention (inter-series)
Seasonality Handling	Positional encoding only	Explicit timestamp encoding	Series decomposition (trend + seasonal)	Frequency analysis (Fourier/Wavelet)	Implicit via patch context	Implicit via patch context	Not explicitly modeled
Data Adaptability	Requires careful positional encoding	Needs timestamp tuning per dataset	Sensitive to decomposition parameters	Random frequency selection	Sensitive to patch size choice	Sensitive to patch and variate alignment	Best suited for multivariate series
Strengths	Theoretical expressivity	Efficient long sequences	Explicit seasonal modeling	Linear complexity, interpretable	Efficient, strong univariate forecasting	Joint temporal + cross-variate modeling	Strong multivariate forecasting
Limitations	Quadratic cost, no inductive bias for TS	Query selection sensitivity	Decomposition quality dependent	Random selection sensitivity	Channel-independent, ignores inter-variate	Higher complexity, harder to tune	Less effective for univariate series
When to Use	General baseline, short sequences	Very long univariate sequences	Series with strong seasonality	Global trend + periodic patterns	Univariate or channel-independent forecasting	Multivariate with temporal and cross-variate dependencies	High-dimensional multivariate forecasting

Table 2.2: Comparative analysis of Transformer architectures for time series forecasting.

2.7 Conclusion

The architectures reviewed in this chapter reflect a clear progression in the treatment of time series data: from simple fixed-window mappings in MLPs, through sequential state propagation in RNNs and local pattern extraction in CNNs, to parallel attention-based modeling in Transformers. Each family addresses specific limitations of its predecessors, yet introduces new trade-offs.

Among all architectures surveyed, Transformer-based models stand out for their unique ability to simultaneously capture long-range dependencies and model complex multivariate interactions in parallel. However, this expressive power comes at a fundamental cost: the internal mechanisms of these models remain largely opaque to end users. Attention maps, while sometimes informative, do not directly translate into actionable explanations they indicate which variates or time steps the model attended to, but not *why*. This opacity is not merely a technical inconvenience; in high-stakes domains such as healthcare, finance, or energy, the inability to justify a prediction can undermine trust and limit deployment. This fundamental limitation directly motivates the study of dedicated Explainable AI methods, which form the focus of the next chapters.

Chapter 3

Explainable Artificial Intelligence for Time Series Models

3.1 Introduction

As established in the previous chapter, deep learning models have set new standards for forecasting accuracy. However, their multi-layered non-linear architectures function as “black boxes,” providing predictions without transparent reasoning. In practice, this opacity poses critical challenges: a model predicting a patient’s cancer risk must justify its decision to the clinician, a model refusing a bank loan must provide a legal explanation under European regulation, and a model forecasting an electrical overload must indicate which variables to act upon and when. In such high-stakes contexts, a forecast without an explanation is often unusable, as stakeholders require not only accuracy but also trust and accountability.

This chapter explores the field of Explainable Artificial Intelligence (XAI), which aims to demystify these models. We present a multi-criteria taxonomy of existing XAI techniques and analyze their limitations when applied to the temporal and multivariate nature of time series. These limitations ultimately motivate the need for more suitable explanation approaches ones that go beyond feature attribution to deliver actionable insights: identifying not just *what* influenced a prediction, but *when*, *why*, and *what corrective action* should follow.

3.2 Conceptual Definitions

3.2.1 Explainable Artificial Intelligence (XAI)

Explainable Artificial Intelligence (XAI) is a subfield of AI that aims to ensure that the reasoning behind model decisions is understandable by human users. The need for XAI arises from the increasing complexity of modern AI systems, particularly deep learning models, which often operate as "black boxes". By providing explanations for predictions, XAI enhances transparency, fosters user trust, facilitates model debugging, and supports accountability in critical applications such as healthcare, finance, and autonomous systems.

3.2.2 Explicability

As defined by Bruckert, Finzel, and Schmid (2020), explicability refers to the property of an AI system that allows its structure, parameters, or components to be examined and inspected. It focuses on making the internal mechanisms of the model accessible and understandable to experts, without necessarily providing context or reasoning for individual decisions. Explicability is considered a prerequisite for explainability, as a model must first be inspectable before its decisions can be meaningfully interpreted.

3.2.3 Explainability

Going further, Bruckert, Finzel, and Schmid (2020) define explainability as involving contextual information about how a model arrives at its outputs, why certain predictions are made, and which features contribute most to a given decision. Explainability is critical for non-expert users, regulators, and stakeholders who need to understand and trust AI decisions, especially in high-stakes domains.

3.2.4 Objectives of XAI

As outlined by Rojat et al. (2021), and as summarized in Figure 3.1, the effectiveness of XAI is assessed through several key properties: **explainability** (meaningful insight into the decision process), **interpretability** (comprehensibility of input-output mappings), **trustworthiness** (confidence that the model behaves as intended), **stability** (robustness of explanations to small input perturbations), and **reproducibility** (consistency of explanations across runs).



Figure 3.1: Knowledge graph relating all the purposes of explainability methods (Rojat et al. 2021).

3.3 Taxonomy of XAI Methods

Following the classification proposed by Schwalbe and Finzel (2023), XAI methods can be systematically organized along three complementary dimensions, as illustrated in Figure 3.2:

3.3.1 Scale of Explanations

As discussed by Rojat et al. (2021), XAI methods provide either **local** or **global** explanations, depending on the scope of their outputs:

Local explanations

Local explanations focus on individual predictions and are valid for a specific sample. They aim to answer questions such as “Why was this particular example classified as a X ?” or “Why did the model make this decision for this input?”. Local methods evaluate the contribution of each feature to a single prediction, making them particularly useful for debugging, trust assessment, or justifying specific decisions. Examples of techniques producing local explanations include LIME, SHAP applied per instance.

Global explanations

Global explanations characterize the overall behavior of a model across the entire dataset or a subset of inputs. They address questions such as “How does the model generally arrive at its conclusions ?” or “Which features are most influential across all predictions ?”. Some local methods can be extended to generate global insights, for example by aggregating SHAP values across the dataset or using clustering techniques to summarize patterns. Global explanations allow a broader understanding of complex models and facilitate model validation, regulatory compliance, and high-level interpretability.

3.3.2 Stage of Explanations

Another important dimension in the taxonomy of explainable AI methods concerns the stage at which interpretability is achieved. As discussed by Schwalbe and Finzel (2023) and TransferLab AppliedAI

Institute (2025), this distinguishes techniques integrated into the model design from those applied after training:

Ante-hoc Methods

Ante-hoc methods, also referred to as intrinsic explainability, incorporate transparency directly into the model's architecture or learning process. These methods aim to make the decision logic understandable by design, rather than through external analysis after training. By constraining the model structure or embedding interpretable components into its architecture.

Post-hoc Methods

In contrast, post-hoc explainability refers to methods that are applied after a complex model has been trained. These techniques aim to produce explanations for the behavior of black-box models without altering their internal structure or parameters. Common examples include feature attribution methods (such as LIME and SHAP). Post-hoc explainers are widely used in practice due to their flexibility and broad applicability, but they do not change the model itself; instead, they derive interpretable insights by analyzing the trained model's outputs or internal activations.

3.3.3 Model Architecture Dependency

Model Agnostic

As described by Arrieta et al. (2020), model-agnostic post-hoc techniques are designed to be applied to any machine learning model with the goal of extracting interpretable information from its predictions. These methods do not require access to the internal structure of the model and often rely on three main strategies: simplification, feature relevance estimation, and visualization:

- **Simplification:** Create interpretable proxies, such as rule-based or linear models (e.g., LIME, G-REX), to approximate complex models and make their predictions understandable.
- **Feature Relevance:** Quantify the contribution of input features to predictions using techniques like SHAP, local gradients, or influence functions, providing local and global insights.
- **Visualization:** Represent model behavior or feature importance graphically through ICE plots, saliency maps, or cluster-based visualizations, often combined with feature relevance for clearer interpretation.

These approaches collectively enable interpretability for any model while remaining independent of its architecture, supporting better understanding and trust in machine learning systems.

Model Specific

Model-specific explainability methods are tailored to a particular type of model and exploit its internal structure. They are divided into two main categories:

- **Shallow Machine Learning** Shallow ML models include a variety of supervised learning approaches. Some models, such as KNN and single decision trees, are inherently interpretable. More sophisticated models, such as tree ensembles and Support Vector Machines (SVMs), require post-hoc explainability techniques to make their predictions understandable. Techniques commonly used :

- **Explanation by Simplification:** Rule extraction from support vectors (e.g., sequential covering algorithms, fuzzy rules, Hyper-rectangle Rule Extraction) and clustering prototype vectors to approximate the decision boundary.
 - **Local Explanations:** Highlight individual predictions and their relation to nearby support vectors or margins.
 - **Visualization Techniques:** Visualizing kernel matrices, heatmaps, or weight vectors to understand feature contributions and decision margins.
 - **Bayesian Approaches:** Interpreting SVMs as Maximum A Posteriori (MAP) solutions with Gaussian Process priors, providing probabilistic predictions and hyperparameter transparency.
- **Deep Learning Models** Deep learning models, including MLPs, CNNs, and RNNs, are highly expressive but often considered black boxes. Model-specific explainability techniques leverage the architecture and internal representations:
- **MLPs:** Use model simplification (e.g., DeepRED, distillation methods) and feature relevance (e.g., Deep Taylor decomposition, DeepLIFT, Integrated Gradients) to interpret hidden layers.
 - **CNNs:** Employ visual explanation techniques such as saliency maps and activation visualizations to interpret hierarchical features.
 - **RNNs:** Use feature relevance (LRP, finite-horizon n-grams) and architectural modifications (attention mechanisms, interpretable latent vectors) to understand sequential dependencies.
 - **Hybrid Methods:** Combine black-box DNNs with transparent models (Decision Trees, kNN, rule-based systems) or integrate domain knowledge to improve interpretability while preserving performance.

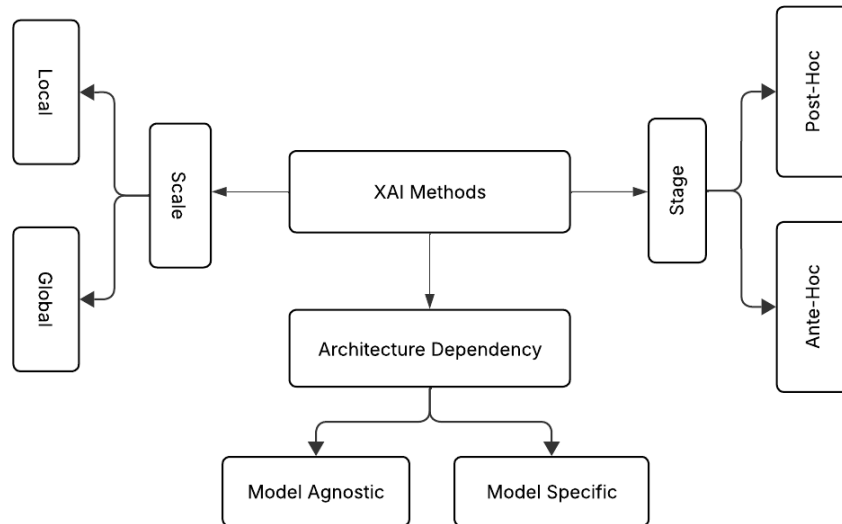


Figure 3.2: Multi-Criteria Taxonomy of XAI Methods.

3.3.4 Evaluation of XAI Methods

As proposed by Lopes et al. (2022), evaluating XAI methods requires both technical and human-centered perspectives:

Computer-centered metrics assess explanation quality formally: *fidelity* measures how accurately the explanation reflects the true model behavior (faithfulness); *clarity* measures the absence of ambiguity; and *completeness* evaluates whether all relevant factors influencing a prediction are captured.

Human-centered metrics assess real-world utility: *understandability* measures whether users form accurate mental models; *trust* evaluates users' confidence in the system; and *usefulness* assesses whether explanations support effective decision-making.

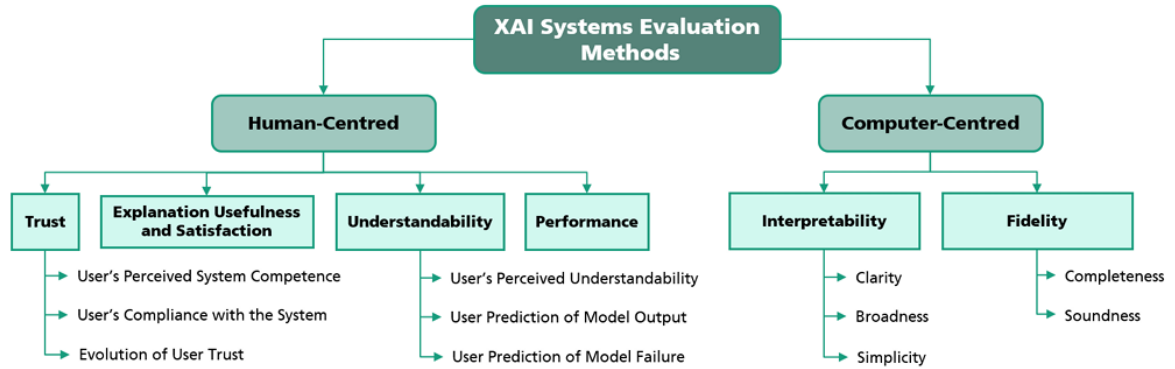


Figure 3.3: Taxonomy of evaluation methods for Explainable Artificial Intelligence (Lopes et al. 2022).

No single XAI method is universally optimal: the appropriate choice depends on the audience (developers, end-users, decision-makers), the required scale (local vs. global), the stage (ante-hoc vs. post-hoc), and the specific forecasting context. This taxonomy provides the conceptual basis for understanding the methods surveyed in the rest of this chapter.

3.4 Output-Based Taxonomy of XAI Methods for Time Series

Following the explanation-based taxonomy proposed by Bodria et al. (2023), XAI methods can be systematically organised according to the *type of explanation they produce*, independently of their internal mechanism. This perspective is particularly suitable for practitioners, who first identify the kind of insight they need before selecting a method. Applied to the time series setting, four output families emerge, as illustrated in Figure 3.4:

- **Feature Importance (FI).** These methods assign a scalar relevance score to each input feature or timestep, producing importance maps or attribution vectors that highlight which parts of the input influenced the prediction most. Representative methods include LIME, SHAP, ShapTime, Dynamic Mask, LRP, and LRP-VIL.
- **Attention-Based Explanations.** These methods derive explanations from the internal attention weights of Transformer architectures, either directly or by propagating them across layers. The output is a token- or timestep-level importance map grounded in the model's own attention mechanism. Representative methods include Attention Rollout, Attention Flow, and ShapTST.
- **Rule-Based Explanations (RB).** These methods extract symbolic rules or logic programs from the model's behaviour, producing human-readable conditions that describe when and why a certain

prediction is made. In the time series setting, NeuroSymbolic approaches fall into this category, combining neural representations with symbolic reasoning to produce interpretable classification rules.

- **Counterfactual Explanations (CF).** Rather than assigning importance scores to existing inputs, these methods generate a *modified* input sequence $\mathbf{x}'$ such that the model produces a desired output — answering the actionable question “*what would need to change?*” rather than the descriptive question “*what mattered?*”. ForecastCF Z. Wang et al. (2023) is the sole existing method in this category designed specifically for time series *forecasting*.

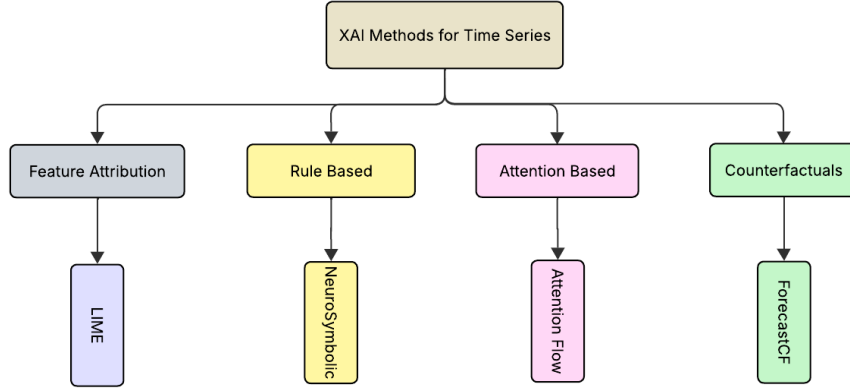


Figure 3.4: Output-based taxonomy of XAI methods for time series, adapted from Bodria et al. (2023).

This taxonomy reveals a fundamental asymmetry: the first three families — feature importance, attention-based, and rule-based methods — are all *descriptive*. They explain why a prediction was made, but offer no guidance on how to intervene. Counterfactual explanations are the only family that is inherently *prescriptive*: their output directly encodes an actionable intervention. This distinction, and the near-absence of counterfactual methods in the forecasting setting, constitutes the central motivation for the contribution presented in Part II.

3.5 XAI Methods for Time Series

Following the output-based taxonomy introduced in Section 3.4, the methods reviewed in this section are organised into four families according to the type of explanation they produce Bodria et al. (2023).

3.5.1 Feature Importance Methods

Feature importance methods assign a relevance score to each input feature or timestep, producing importance maps that highlight which parts of the input most influenced the prediction.

LIME

Local Interpretable Model-agnostic Explanations (LIME) Ribeiro, Singh, and Guestrin (2016) is a post-hoc, model-agnostic method that explains individual predictions of any black-box model by fitting a simple interpretable surrogate — such as a linear model — in the local neighborhood of the input instance. Perturbed samples are generated around the instance, weighted by proximity, and used to train the surrogate, whose coefficients reveal each feature’s contribution to the prediction. In time series analysis, LIME can be adapted to perturb temporal segments or individual time steps, identifying the intervals most influential for a given prediction. However, temporal correlations between observations may be disrupted during perturbation, leading to unrealistic samples and potentially misleading explanations.

SHAP

SHapley Additive exPlanations (SHAP) Lundberg and Lee (2017) is a unified framework grounded in cooperative game theory that assigns each feature a theoretically consistent contribution to the model’s prediction. SHAP values are additive — their sum equals the difference between the model output and the expected baseline — providing both local (per-instance) and global (dataset-level) explanations. In time series forecasting, SHAP identifies which time steps, features, or lags are most influential for a prediction, making it particularly helpful for models with complex dependencies like Transformers.

ShapTime

ShapTime Yuyi Zhang et al. (2024) is a model-agnostic XAI approach specifically designed for time series forecasting, extending Shapley values from cooperative game theory to the temporal dimension. Rather than attributing importance to individual features, it divides the input sequence into contiguous groups of time points called *super-times* and computes the marginal contribution of each segment to the model’s prediction. This temporal granularity makes ShapTime particularly interpretable for forecasting tasks, where understanding which historical intervals drive future predictions is more meaningful than feature-level attribution. However, the method becomes computationally expensive as the number of segments grows, and its effectiveness remains tied to the quality of the underlying forecasting model.

Dynamic Mask

Dynamic Mask Crabbé and Van Der Schaar (2021) is a post-hoc, model-agnostic technique that provides local interpretability for time series models by identifying the contribution of individual features and time steps to a given prediction. The method optimizes a mask matrix M applied to the input sequence X , producing a modified input X_{mask} whose prediction remains as close as possible to the original output. The resulting mask highlights the most influential temporal segments and features without requiring any modification to the underlying model. While effective for multivariate and high-dimensional sequences,

Dynamic Mask is computationally intensive for long series and relies on the quality of the perturbation strategy, which may introduce artifacts that distort the explanation.

Layer-Wise Relevance Propagation (LRP)

Layer-Wise Relevance Propagation (LRP) is a post-hoc technique that decomposes a model’s output into relevance scores assigned to each input feature by propagating the prediction backward through the network layers. A conservation constraint ensures that relevance is neither created nor destroyed across layers — neurons contributing more strongly to a given output receive a proportionally larger share of its relevance. The resulting relevance map highlights which input features support or inhibit the prediction. However, when applied directly to raw time series, LRP often produces diffuse and semantically unclear explanations, as individual time steps lack intrinsic meaning and spectral patterns remain hidden.

LRP with Virtual Inspection Layer (VIL)

To address this limitation, the Virtual Inspection Layer (VIL) Vielhaben et al. (2024) introduces an invertible linear transform — either a Discrete Fourier Transform (DFT) or a Short-Time Fourier Transform (STFT) — as a conceptual layer placed before the model input. LRP relevance is then propagated through both the model and this transform, expressing relevance in the frequency or time-frequency domain rather than the raw time domain. DFT-VIL reveals which global frequency components support or inhibit the prediction, while STFT-VIL produces a time-frequency relevance map that additionally captures *when* those frequencies matter. Both variants preserve LRP’s conservation principle and expose spectral decision strategies invisible to standard LRP. Figure 3.5 illustrates how the virtual inspection layer redirects relevance propagation.

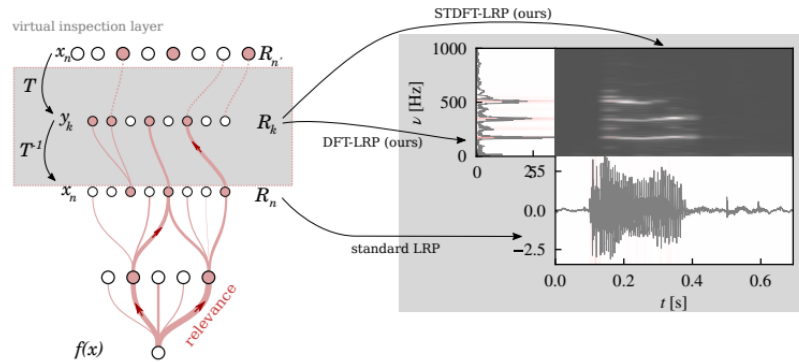


Figure 3.5: LRP with Virtual Inspection Layer Vielhaben et al. (2024).

3.5.2 Attention-Based Methods

Attention-based methods derive explanations directly from the attention weights computed within Transformer architectures, either by reading them layer-by-layer or by propagating them globally across the network.

Attention Rollout

Standard attention weights in Transformers are limited for interpretability, as they provide only local, layer-wise information and lose token identity in deeper layers. Consequently, they cannot reliably reflect the true contribution of input tokens to model predictions Abnar and Zuidema (2020). To address

this, Attention Rollout is proposed as a post-hoc global method that estimates token contributions by propagating attention across layers while incorporating residual connections. The augmented attention matrix combines attention weights with identity connections to preserve original token information. The cumulative attention is computed recursively across layers, capturing all possible paths from input tokens to higher-level representations. Attention Rollout is particularly useful for time series, as it captures long-range temporal dependencies and identifies influential time steps. However, it relies on a linear propagation assumption, may lose fine-grained details due to head averaging, and does not provide causal explanations.

Attention Flow

Attention Flow extends Attention Rollout by addressing its limitation in handling multiple overlapping paths. It models attention as a flow network, where tokens are nodes and attention weights (including residual connections) define edge capacities Abnar and Zuidema (2020). The method computes the maximum flow from output tokens to input tokens, ensuring that all possible paths are considered without underestimating their contributions. This results in a more robust and distributed estimation of token importance compared to Rollout. While Attention Flow provides improved interpretability and better alignment with attribution methods, it is computationally more expensive and, like Rollout, does not guarantee causal explanations.

ShapTST

ShapTST Cheng et al. (2025) is an ante-hoc explanation method that integrates interpretability directly into the Transformer architecture through a dedicated *explainer head* — a Shapley-based module that computes feature and temporal importance simultaneously with predictions, without any post-hoc overhead. This tight coupling allows the model to jointly optimize predictive accuracy and explanation fidelity, providing both local and global attributions in a single forward pass. Benchmark evaluations show that ShapTST outperforms or matches ShapTime in computational efficiency, accuracy, and explanation faithfulness. Unlike post-hoc methods, however, ShapTST is model-specific and requires retraining the Transformer with the explainer head, limiting its applicability to black-box or already-trained models. Figure 3.6 illustrates the ShapTST architecture with its integrated explainer head.

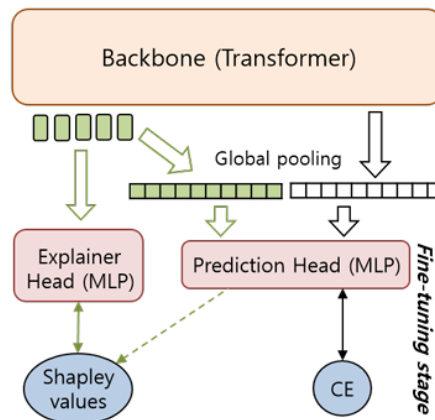


Figure 3.6: ShapTST architecture Cheng et al. (2025).

3.5.3 Rule-Based Methods

Rule-based methods produce symbolic rules or logic programs extracted from the model’s behaviour, offering human-readable conditions that describe when and why a prediction is made.

NeuroSymbolic Time Series Classification

A neuro-symbolic framework is proposed that integrates symbolic reasoning with deep learning for interpretable time series analysis. The approach embeds **Signal Temporal Logic (STL)** into neural architectures and employs a **Multi-View Representation** of time series data, enabling rule extraction while maintaining competitive predictive performance R. Yan et al. (2022).

As illustrated in Figure 3.7, the framework comprises two key components:

- **Node Phase Classifiers (NPCs)**: Binary classifiers that correspond to neural equivalents of STL formulas. Each NPC outputs a *truth degree* indicating the degree of satisfaction of a temporal property. NPCs operate on a multi-view representation—composed of raw, spectral, and derivative features—to capture complementary temporal characteristics.
- **Tree Phase Classifier (TPC)**: For multi-class problems, NPCs are hierarchically combined into a TPC. This structure enables structured and interpretable decision-making through recursive aggregation of NPC outputs, forming a transparent decision tree.

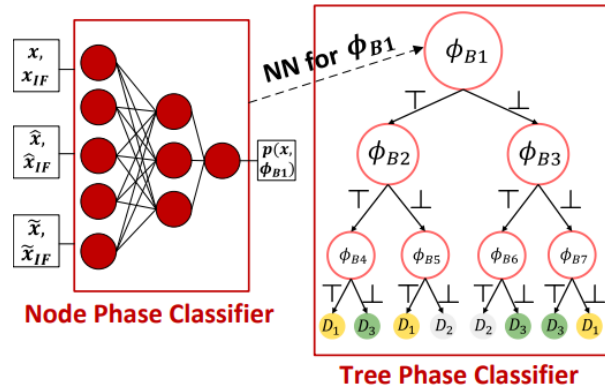


Figure 3.7: Architecture of the Neuro-Symbolic Time Series Classification (NSTSC) framework.

3.5.4 Counterfactual Methods

Unlike the three families above, counterfactual methods do not assign importance scores to the existing input. Instead, they generate a modified input sequence $\mathbf{x}'$ such that the model produces a desired output, answering the actionable question “*what would need to change?*” rather than the descriptive question “*what mattered?*”. Within the output-based taxonomy of Bodria et al. (2023), counterfactual explanations constitute a distinct family characterised by their prescriptive nature.

ForecastCF

ForecastCF, introduced by Z. Wang et al. (2023), is the first method to adapt counterfactual explanations specifically to time series forecasting. Unlike feature attribution methods, ForecastCF generates actionable modifications of the input sequence that lead a forecasting model to produce a desired future trajectory. The approach applies gradient-based perturbations to the original time series, iteratively

adjusting it until the model’s predictions satisfy user-defined upper and lower bound constraints over the forecasting horizon. Evaluated on six real-world datasets (CIF2016, NN5, Tourism, M4-Finance, SP500, MIMIC) across four architectures (GRU, Seq2seq, WaveNet, N-BEATS), ForecastCF consistently achieves validity rates above 75% while producing more compact and interpretable modifications than nearest-neighbor and naive shifting baselines. However, the method operates directly in the input space without any plausibility constraint, which can lead to unrealistic or temporally incoherent counterfactuals. Furthermore, its instance-specific optimisation is computationally demanding, limiting scalability to large datasets, while the absence of sparsity constraints results in dense modifications that are difficult to interpret and act upon. These limitations are examined in depth in Chapter 4.

3.6 Comparison of XAI Methods for Time Series

Explainable Artificial Intelligence (XAI) methods provide different ways to interpret model predictions in time series analysis. These methods vary in terms of required inputs, produced outputs, and the type and scale of explanations they provide. We summarize in Table 3.1 commonly used XAI methods, their main inputs, output forms, and the nature of the explanations they produce, as reviewed throughout this chapter. This comparison is purely descriptive and does not aim to evaluate or rank the methods.

Method	Input	Output	XAI Scale	XAI Stage	Architecture Dependency
SHAP	Input + model outputs	Feature contributions	Local + Global	Post-hoc	Model-agnostic
ShapTime	Time series input + model predictions	Temporal feature importance	Local + Global	Post-hoc	Model-agnostic
LRP	Model weights + activations	Relevance maps	Local + Global	Post-hoc	Model-specific
Dynamic Mask	Time series input (masking perturbation)	Feature importance (temporal attribution)	Local	Post-hoc	Model-agnostic
Attention Rollout	Multi-layer attention + residuals	Global token importance	Global	Post-hoc	Model-specific
Attention Flow	Attention graph representation	Information flow paths	Local + Global	Post-hoc	Model-specific
ShapTST	Transformer-based time series input	Attention-aware SHAP explanations	Local + Global	Post-hoc	Model-specific
Neuro-Symbolic Models	Time series input + logical rules	Symbolic rules + temporal explanations	Local + Global	Post-hoc	Model-specific
ForecastCF	Time series forecasting input	Counterfactual explanations	Local	Post-hoc	Model-agnostic

Table 3.1: Comparative overview of XAI methods for time series.

3.7 Evaluation Methods for XAI Explanations on Time Series

This section summarizes the verification methodology for evaluating explainable AI (XAI) methods on time series data, as proposed by Schlegel et al. in Schlegel et al. (2019) and shown in the figure 3.8. The core idea is to perturb the input time series based on relevance scores from an XAI method and measure the impact on model performance. If the explanation correctly identifies important features, perturbing them should degrade performance more than perturbing irrelevant or random features.

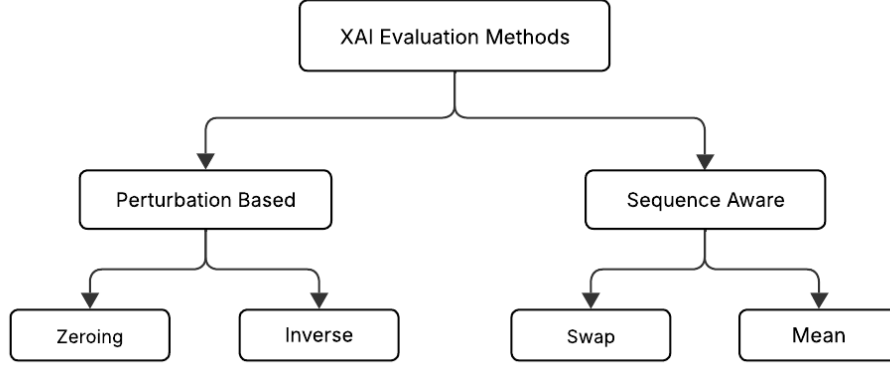


Figure 3.8: Time series Evaluation Strategies

Two main categories of evaluation methods are introduced: A **random baseline** is used as a control to ensure effects are due to relevance, not noise.

3.7.1 Perturbation-Based Methods

Given a time series t and relevance vector r , points with relevance above a threshold e are modified.

Zeroing:

Set relevant points to zero. Tests dependence on magnitude.

$$t_i^{\text{zero}} = \begin{cases} 0 & \text{if } r_i > e \\ t_i & \text{otherwise} \end{cases}$$

If the explanation is correct, setting relevant points to zero should cause a noticeable drop in model performance (e.g., accuracy). This drop should be larger than when zeroing random points (random baseline), indicating that the model indeed relies on the magnitude of those specific time points.

Inverse:

Replace with inverse value. Tests sensitivity to direction.

$$t_i^{\text{inverse}} = \begin{cases} \max(t) - t_i & \text{if } r_i > e \\ t_i & \text{otherwise} \end{cases}$$

If the explanation correctly identifies directional features, inverting relevant points should degrade performance more than random inversion. This suggests the model is sensitive to the polarity or shape direction (e.g., peaks vs. troughs) of the signal.

3.7.2 Sequence-Aware Methods

Modify contiguous subsequences starting from high-relevance points.

Swap:

Reverse the subsequence. Tests importance of temporal order.

$$t_{\text{sub}}^{\text{swap}} = (t_{i+n_s}, \dots, t_{i+1}, t_i)$$

If the explanation captures meaningful temporal patterns, reversing relevant subsequences should lead to a significant performance decrease, more so than reversing randomly selected subsequences. This indicates the model relies on the original temporal ordering (e.g., trends, slopes, or patterns).

Mean:

Flatten subsequence to its mean. Tests reliance on local variability.

$$t_{\text{sub}}^{\text{mean}} = (\mu, \dots, \mu), \quad \mu = \frac{1}{n_s} \sum_{k=i}^{i+n_s} t_k$$

If the explanation identifies regions where local variability matters, flattening those subsequences should harm performance more than flattening random subsequences.

3.8 Conclusion

The comparative analysis of XAI methods for time series reveals a consistent and fundamental tension. Methods ranging from gradient-based saliency (LRP, Integrated Gradients) to attention-based approaches (Attention Rollout, Attention Flow), model-agnostic perturbations (SHAP, ShapTime, Dynamic Mask), and neuro-symbolic models provide valuable insights into feature importance, yet they face compounding limitations in the temporal domain either computationally prohibitive for long-lookback architectures, or unable to respect the physical and temporal correlations of multivariate data, producing static importance maps that lack the dynamic context required for complex forecasting.

Beyond these practical constraints lies a deeper, structural limitation: virtually all methods reviewed remain strictly *descriptive*. They identify which timestamps or variables influenced a forecast, but provide no *actionable* guidance. In critical scenarios energy management, predictive maintenance, clinical forecasting knowing that a variable was “important” is insufficient; stakeholders need to know *how to intervene*. LIME, SHAP, LRP, and Attention Rollout all remain confined to explaining existing predictions rather than guiding intervention. ForecastCF represents the closest step toward actionability, yet its instance-specific optimisation in raw input space and absence of distributional constraints expose fundamental limitations in plausibility, sparsity, and scalability.

This limitation marks the boundary of traditional feature attribution. Accordingly, the following chapter delves into counterfactual explanations in depth covering their foundations, existing approaches, and specific challenges when applied to time series laying the groundwork for the unified framework proposed in Part II.

Chapter 4

Counterfactual Explanations for Time Series

4.1 Introduction

In recent years, explainable artificial intelligence (XAI) methods have been widely used to improve the interpretability of machine learning models. These approaches provide valuable insights into model behavior by identifying the features or temporal patterns that influence predictions. However, most of these methods remain descriptive, focusing on explaining why a prediction is made rather than providing guidance on how to change it.

In many real-world applications, particularly in time series analysis, decision-making requires actionable insights rather than descriptive explanations. In this context, counterfactual explanations offer a complementary perspective by identifying the minimal changes needed in the input to achieve a desired outcome.

This chapter focuses on counterfactual explanations for time series. It first introduces their definition and underlying principles, then reviews existing methods for both classification and forecasting tasks. Finally, it discusses their limitations and open challenges, from which we identify the key research gaps that directly motivate the framework proposed in Part II.

4.2 Definition and Formulation of Counterfactual Explanations

According to Wachter, Mittelstadt, and Russell (2017), a counterfactual explanation is defined as the minimal change to an input instance that results in a change in the model’s prediction to a desired target outcome. These explanations are model-agnostic and provide actionable insights by focusing on how to alter the input rather than explaining the internal mechanisms of the model.

4.3 Motivation of Counterfactual Explanations

Counterfactual explanations have gained increasing attention in explainable artificial intelligence due to their intuitive and actionable nature. Instead of explaining a prediction by assigning importance scores to input features, counterfactual explanations focus on identifying how an input should be minimally modified to obtain a different prediction.

This type of explanation directly answers a contrastive question: *what should be different for the model to produce another outcome?* Such reasoning closely matches how humans naturally explain decisions, often by comparing the observed situation with alternative scenarios.

Given an input X and a predictive model $f(\cdot)$, a counterfactual explanation consists of a modified input X_{cf} such that the model output changes, while keeping the modification as small as possible. The goal is not only to change the prediction, but also to ensure that the resulting instance remains realistic and interpretable.

Compared to attribution-based methods such as feature importance scores, counterfactual explanations provide more actionable insights. They explicitly indicate which changes in the input would lead to a different prediction, rather than simply highlighting influential features.

However, counterfactual explanations describe the behavior of the model rather than true causal relationships. They should therefore be interpreted as local, model-dependent explanations.

Overall, counterfactual explanations offer a complementary perspective in XAI by focusing on minimal and meaningful transformations of the input that lead to different outcomes.

4.4 Counterfactual Explanations for Time Series

Counterfactual explanations aim to provide actionable insights by identifying how an input should be modified in order to achieve a desired outcome. Unlike traditional XAI methods that focus on explaining model predictions, counterfactual approaches explicitly define an objective and search for minimal changes to the input that satisfy this objective.

In the context of time series, defining such objectives is more complex due to the sequential nature of the data and the dependencies between time steps. The type of objective depends on the learning task, and different formulations arise for classification and forecasting problems.

In the following, we distinguish between counterfactual objectives for classification and forecasting, as they involve fundamentally different constraints and challenges :

4.4.1 Counterfactuals for Classification

Counterfactual explanations for classification tasks are relatively well studied compared to other domains. In this setting, a counterfactual explanation aims to determine how an input time series should be modified so that the predicted class changes from an original class c to a target class c' . Formally, given a time series T_q classified as c , the goal is to generate a counterfactual instance T' such that the classifier predicts c' instead.

Several approaches have been proposed in the literature. Among them, instance-based methods have shown promising results for time series classification. For example, Delaney, Greene, and Keane (2021) introduce a model-agnostic technique that generates counterfactuals by adapting existing instances from the dataset. Their approach relies on retrieving a *nearest unlike neighbor* (NUN), which belongs to a different class, and modifying the most discriminative subsequences of the original time series to produce a plausible counterfactual.

Unlike traditional feature-based methods, these approaches consider the sequential nature of time series data by focusing on contiguous subsequences rather than independent features. This is important because time series data exhibit strong temporal dependencies, and meaningful explanations must preserve these structures.

Moreover, several key properties of good counterfactual explanations have been identified in this context, including proximity (minimal change from the original instance), sparsity (modification of limited regions), plausibility (remaining within the data distribution), and diversity (providing multiple alternative explanations).

Figure 4.1 illustrates a typical example of a counterfactual explanation for time series classification. The original signal is modified to produce a counterfactual instance that leads to a different predicted class.

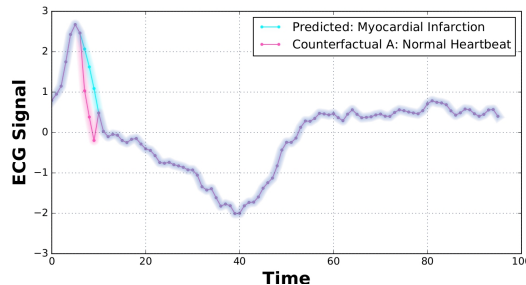


Figure 4.1: Example of a counterfactual explanation for time series classification. The original signal (blue) is modified to generate a counterfactual instance (Red) that changes the predicted class (Delaney, Greene, and Keane (2021)).

Overall, counterfactual explanations for classification provide intuitive and actionable insights by showing how a time series instance can transition from one class to another. However, extending these approaches to forecasting tasks introduces additional challenges due to the continuous nature of predictions and temporal consistency constraints.

4.4.2 Counterfactuals for Forecasting

In time series forecasting, counterfactual explanations consist in generating alternative input sequences that lead to a desired change in the predicted trajectory. This requires not only achieving the target forecasting objective but also ensuring that the generated sequences remain temporally coherent, plausible, and close to the original input. Consequently, counterfactual generation in time series is inherently more complex than in tabular settings due to sequential dependencies and structural constraints (Z. Wang et al. (2023)).

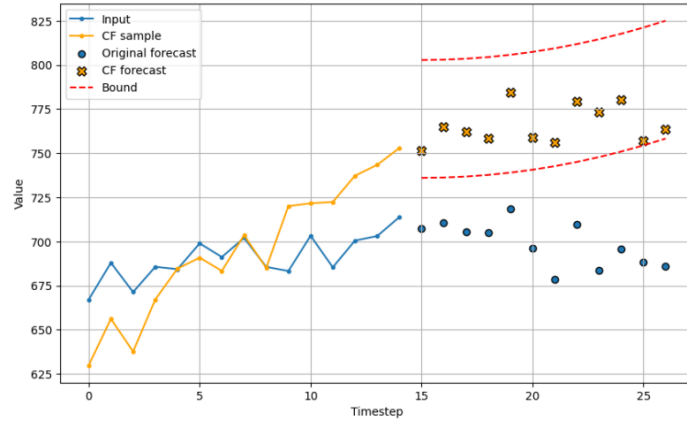


Figure 4.2: Counterfactual forecasting under bounded target constraints

Figure 4.2 illustrates the concept of counterfactual explanations in time series forecasting. The original input sequence is shown along with its corresponding forecasted values. A CF instance is generated by minimally modifying the input sequence, resulting in a new forecast that satisfies a predefined objective.

In this example, the goal is to increase the predicted values within a target range, represented by the dashed bounds. The counterfactual input leads to a shift in the predicted trajectory, while remaining close to the original sequence.

This illustrates the key idea of counterfactual explanations: identifying minimal and plausible changes to the input that produce a desired outcome in the model’s prediction .

4.5 Taxonomy of Counterfactual generation Methods

Counterfactual explanation methods can be broadly categorized into several families based on how counterfactual instances are generated. Following the taxonomy proposed by Verma, Dickerson, and Hines (2020), common categories include optimization-based, generative model-based, and instance-based approaches. In addition, recent works have introduced learning-based methods, such as reinforcement learning, as a promising direction for counterfactual generation.

4.5.1 Optimization-Based Methods

Optimization-based approaches formulate counterfactual generation as a constrained optimization problem, where the objective is to find minimal changes to the input that achieve a desired prediction. These methods are flexible and model-agnostic, but often require solving an optimization problem for each instance, making them computationally expensive and sometimes producing unrealistic samples.

4.5.2 Generative Model-Based Methods

Generative approaches rely on models such as VAEs or GANs to learn the data distribution and generate counterfactuals within a latent space. These methods improve plausibility and realism by constraining generated samples to the data manifold. However, their effectiveness depends on the quality of the generative model and can be challenging for complex time series data.

4.5.3 Instance-Based Methods

Instance-based methods generate counterfactuals by retrieving or adapting existing data points, often using nearest-neighbor techniques. These approaches are computationally efficient and naturally ensure plausibility, but they are limited by dataset diversity and may fail to produce minimal or precise changes.

4.5.4 Learning-Based Methods

More recently, learning-based approaches have emerged, where models learn to directly generate counterfactuals. In particular, reinforcement learning (RL) formulates counterfactual generation as a sequential decision-making problem, enabling efficient exploration of the input space. These methods are well-suited for complex settings such as time series forecasting, but require careful design of reward functions to balance validity, proximity, and plausibility Li et al. (2026).

Taxonomy of Reinforcement Learning

Reinforcement Learning (RL) is a distinct paradigm of Machine Learning where an agent learns to make sequences of decisions by interacting with an environment. The goal of the agent is to maximize a cumulative scalar signal called the *reward*. Unlike supervised learning, RL does not require labeled input/output pairs; instead, it relies on an iterative trial-and-error process to discover which actions yield the highest long-term benefit.

The taxonomy of RL algorithms can be broadly categorized based on the agent’s strategy and its internal representation of the environment Arulkumaran et al. (2017):

- **Model-Based vs. Model-Free RL:**

- *Model-Based*: The agent attempts to learn a transition model of the environment to predict future states and rewards. This allows for planning but often suffers from model bias in complex time series.
- *Model-Free*: The agent learns directly from experience without explicitly modeling the environment’s dynamics. This approach is more robust for high-dimensional data where the underlying rules are unknown.

- **Value-Based vs. Policy-Based Methods:**

- *Value-Based (e.g., Q-Learning)*: The agent learns a value function (Q-function) to estimate the expected return for each action. It then chooses the action with the highest estimated value.
- *Policy-Based (e.g., Policy Gradient)*: The agent directly optimizes the policy $\pi(a|s)$, which is the probability distribution over actions given a state. This is particularly effective for continuous or high-dimensional action spaces.

- **Actor-Critic Methods**: This hybrid approach combines the strengths of both value-based and policy-based methods. The **Actor** proposes an action by following the current policy, while the

Critic evaluates that action by computing a value function. This synergy reduces the variance typically found in pure policy gradient methods and speeds up convergence.

4.6 Evaluation Metrics for Counterfactual Explanations

Evaluating counterfactual explanations (CFs) is a critical yet challenging task in explainable artificial intelligence. While numerous methods have been proposed, the literature highlights significant limitations in their evaluation, including the lack of standardized metrics and limited user-centered validation Keane et al. (2021)

Counterfactual explanations are typically assessed using computational proxy metrics that aim to capture desirable properties such as proximity, sparsity, and plausibility. However, these metrics are often not grounded in human perception, raising questions about their effectiveness in measuring explanation quality. Several core properties have emerged as essential criteria for evaluating counterfactual explanations:

Validity Validity measures whether the generated counterfactual successfully achieves the desired outcome. In the context of time series forecasting, this corresponds to verifying that the counterfactual input leads to a forecast satisfying a predefined objective (e.g., reducing the predicted values by a given percentage).

Proximity Proximity assesses how close the counterfactual instance is to the original input. Most methods rely on distance metrics such as L1 or L2 norms. While proximity is widely used, it does not necessarily guarantee plausibility, as small perturbations may still produce unrealistic samples.

Plausibility Plausibility refers to whether the generated counterfactual is realistic and consistent with the data distribution. Existing methods often approximate plausibility using distributional constraints or generative models. However, the literature highlights that plausibility remains poorly defined and is rarely validated through user studies.

Sparsity Sparsity measures the number of changes between the original instance and the counterfactual. Sparse explanations are generally preferred, as they are easier to interpret. Nevertheless, there is no consensus on the optimal level of sparsity, and empirical evidence suggests that overly minimal changes may not always lead to better explanations .

Coverage Coverage evaluates how often a method is able to generate valid and acceptable counterfactuals across a dataset. This metric is important for assessing the robustness of a method, as some approaches may fail or produce unrealistic counterfactuals in a significant number of cases .

4.7 Challenges and Trade-offs in Counterfactual Explanations

Del Ser et al. (2022) formulate the generation of counterfactual explanations as a multi-objective optimization problem. Indeed, several conflicting objectives must be satisfied simultaneously to produce meaningful and interpretable counterfactual instances. Three key objectives are traditionally considered:

- **Validity:** The counterfactual must successfully change the model’s prediction toward the desired target.
- **Proximity:** The modification applied to the original instance should be minimal to ensure the explanation remains relevant.
- **Plausibility:** The generated counterfactual should remain realistic and lie within the data distribution, respecting physical laws and temporal correlations.

These three objectives are inherently conflicting, and optimizing for one systematically degrades the others:

- **Validity vs. Sparsity:** To guarantee a prediction flip, optimization methods tend to spread perturbations across the entire temporal horizon. The counterfactual becomes valid, but at the cost of modifying every timestamp — making the explanation unreadable and lacking **actionability**.
- **Sparsity vs. Proximity:** Conversely, restricting changes to a small number of time steps forces the method to apply large perturbations at those specific points to still achieve the desired outcome. The explanation is localized, but the magnitude of changes becomes unrealistically large, violating proximity.
- **Proximity vs. Plausibility:** Enforcing small perturbations does not guarantee that the resulting series remains realistic. A sequence of minimal but poorly placed modifications can produce a counterfactual that is close to the original in terms of distance yet lies far outside the data manifold — statistically implausible and physically meaningless.

4.8 Positioning of Counterfactual Explanations within XAI

In high-stakes applications such as energy grid management, financial forecasting, or clinical decision support a passive explanation is often insufficient. An operator facing a predicted power overload cannot act on the information that *“load and temperature were the most important features.”* What they need is a concrete, actionable intervention: *“if the industrial load is reduced between timesteps 60 and 80, the predicted overload disappears.”*

Counterfactual explanations are the only XAI family that provides this level of actionability. Rather than producing a static importance map, they generate a **modified input sequence** that directly encodes the intervention required to steer the model’s forecast toward a desired trajectory.

Beyond actionability, counterfactual explanations satisfy three core properties that make them particularly well-suited for time series. They are **local**, focusing on individual predictions rather than global model behavior. They are **model-agnostic**, applicable to any black-box forecasting architecture. And they are **temporally grounded** — the explanation is expressed in the same sequential language as the data itself, making it directly interpretable by domain experts.

In this sense, counterfactual explanations do not replace attribution-based methods — they complete them. Where SHAP tells the analyst *what mattered*, a counterfactual tells them *what to do about it*.

4.9 Conclusion

This chapter examined counterfactual explanations as a distinct and uniquely actionable family within the XAI landscape. Unlike attribution-based methods that identify which features influenced a prediction, counterfactual explanations answer a fundamentally different question: what minimal and realistic modification to the input would have produced a different outcome? This prescriptive nature makes them particularly well-suited to time series forecasting, where practitioners need not only to understand a prediction but to intervene upon it.

The taxonomy of generation methods optimization-based, generative, instance-based, and learning-based reveals a clear progression toward more scalable and distribution-aware approaches, with reinforcement learning emerging as a promising direction for complex sequential settings. The evaluation framework, structured around validity, proximity, plausibility, sparsity, and coverage, highlights the inherent tension between these criteria: no existing method simultaneously satisfies all of them.

The limitations identified throughout this chapter in plausibility, sparsity, scalability, and the absence of a dedicated forecasting framework are addressed by the contribution presented in Part II.

Part II

Contribution

Chapter 5

Framework Design

Introduction

Having identified the key limitations of existing approaches in Chapter 4, this chapter introduces RL-MCF (Reinforcement Learning for Counterfactual Explanations in Time Series Forecasting), our proposed framework for counterfactual explanations in time series forecasting. We detail the theoretical foundations of the framework, covering the problem formulation, the architecture and role of each component, the mathematical formulations governing the training procedure, and the evaluation metrics used to assess counterfactual quality.

5.1 Problem Formulation

Counterfactual explanations have been extensively studied in classification settings, where the objective is to identify minimal changes to an input instance that alter the predicted class. Formally, given an input $\mathbf{x}$ and a classifier $f(\cdot)$, a counterfactual instance $\mathbf{x}^{cf}$ is sought such that $f(\mathbf{x}^{cf}) \neq f(\mathbf{x})$, while keeping $\mathbf{x}^{cf}$ as close as possible to $\mathbf{x}$. However, extending this formulation to time series forecasting introduces fundamental challenges, as the model output is no longer a discrete label but a continuous trajectory over a future horizon.

Let $\mathbf{x} \in \mathbb{R}^L$ denote a univariate time series of length L , referred to as the look-back window, and let $f : \mathbb{R}^L \rightarrow \mathbb{R}^H$ be a pre-trained black-box forecasting model producing a predicted trajectory:

$$\hat{\mathbf{y}} = f(\mathbf{x}) \in \mathbb{R}^H \quad (5.1)$$

where H denotes the forecasting horizon. A counterfactual explanation is defined as a perturbed input $\mathbf{x}^{cf} \in \mathbb{R}^L$ such that the resulting forecast $f(\mathbf{x}^{cf})$ satisfies a user-defined target constraint, while remaining as close as possible to the original input and preserving temporal realism. The counterfactual generation problem is thus formulated as finding $\mathbf{x}^{cf}$ that simultaneously satisfies three competing criteria:

- **Validity:** the counterfactual forecast must satisfy a predefined target constraint on the predicted trajectory.
- **Proximity:** the perturbation $\|\mathbf{x}^{cf} - \mathbf{x}\|$ must be minimized to ensure the explanation remains actionable and interpretable.
- **Temporal Plausibility:** the generated series must remain realistic and temporally coherent, respecting the underlying data distribution.

These three criteria are inherently conflicting — achieving high validity may require large perturbations that reduce proximity, while enforcing plausibility constrains the search space. This tension defines the core challenge of counterfactual generation for time series forecasting.

5.2 Baseline Approaches and Limitations

Two existing lines of work directly motivate our contribution: gradient-based counterfactual forecasting, and reinforcement learning-based counterfactual generation for time series classification. While each addresses part of the problem, neither is sufficient for our setting, as we detail below.

5.2.1 ForecastCF: Gradient-Based Counterfactual Forecasting

ForecastCF Z. Wang et al. (2023) is, to our knowledge, the first and only dedicated method for counterfactual explanations in time series forecasting. It defines a range-based validity objective requiring the forecasted values to lie within user-specified bounds $[\alpha, \beta]$ at each horizon step:

$$\alpha_t \leq f(\mathbf{x}^{cf})_t \leq \beta_t, \quad \forall t \in \{1, \dots, H\} \quad (5.2)$$

In ForecastCF, the target bounds are centered on the median of the original input sequence $\text{median}(\mathbf{x})$, and counterfactuals are generated via instance-specific gradient-based optimization directly in the raw input space, iteratively adjusting $\mathbf{x}^{cf}$ until the predicted trajectory satisfies the range constraint. Evaluated across multiple real-world datasets and forecasting architectures, ForecastCF demonstrates the feasibility of the counterfactual forecasting paradigm and establishes the range-based objective as a natural and interpretable formulation.

However, despite its pioneering contribution, ForecastCF exhibits three critical limitations that restrict its practical applicability:

- **Distributional Unrealism.** By operating directly in the raw input space without any distributional constraint, ForecastCF provides no guarantee that generated counterfactuals remain within the data manifold. The resulting sequences are frequently temporally incoherent or physically implausible, limiting their interpretability for domain experts.
- **Global Perturbation.** The gradient-based optimization modifies the entire input sequence uniformly, rather than localizing changes to specific actionable temporal windows. From a practical standpoint, an explanation that alters every timestep of a look-back window is neither interpretable nor actionable — a decision-maker cannot realistically intervene at every point in time.
- **Limited Scalability.** ForecastCF requires solving an independent optimization problem for each test instance, with no knowledge transfer across samples. This makes the method computationally prohibitive at scale, with reported runtimes exceeding one hour on standard benchmarks.

5.2.2 Reinforcement Learning for Counterfactual Classification

A second relevant line of work is that of Li et al. (2026), who proposed an actor-critic reinforcement learning framework for counterfactual generation in time series *classification*. In this approach, a Temporal Convolutional Network autoencoder (TCN-AE) is first trained to encode the input time series $\mathbf{x}$ into a compact latent representation $\mathbf{z} = \phi(\mathbf{x})$. A reinforcement learning agent then learns a policy to modify this latent representation in order to achieve a desired prediction outcome. The modified latent vector $\mathbf{z}^{cf}$ is decoded to produce a counterfactual instance $\mathbf{x}^{cf} = \psi(\mathbf{z}^{cf})$.

This latent-space formulation offers two key advantages over gradient-based input-space methods. First, by constraining perturbations to the learned latent space, plausibility is implicitly enforced — the decoder acts as a projection onto the data manifold, ensuring that generated counterfactuals respect the statistical structure of the training distribution. Second, once trained, the policy generates counterfactuals in a single forward pass, enabling efficient and scalable inference across any number of test instances.

However, this framework is inherently designed for classification tasks, where the objective is to switch from one discrete label to another. Such a formulation is not directly applicable to forecasting tasks for two fundamental reasons. First, the model output is no longer a discrete class but a continuous trajectory over a future horizon, making the notion of label flipping meaningless. Second, the reward function is designed around classification objectives and cannot be directly adapted to enforce range-based constraints on predicted trajectories. This limitation motivates the need to extend and adapt this framework to the forecasting setting.

5.3 Contributions

Our main contributions are as follows:

- We propose **RL-MCF**, an actor-critic reinforcement learning framework for counterfactual explanations in time series forecasting that enables scalable single-pass generation through an amortized latent-space perturbation policy.
- We introduce a **latent-space perturbation strategy** via a pre-trained autoencoder that implicitly enforces plausibility, constraining counterfactuals to the learned data manifold without requiring explicit distributional constraints.
- We introduce a **temporal masking mechanism** that enforces sparse and localized perturbations, improving actionability while preserving temporal structure.
- We conduct extensive experiments across three datasets and five forecasting architectures, showing that RL-MCF achieves a strong trade-off between validity, compactness, proximity, and plausibility, while reducing inference time by up to three orders of magnitude compared with instance-wise optimization baselines.

5.4 Framework Overview

The proposed framework, referred to as **RL-MCF** (Reinforcement Learning for Counterfactual Explanations in Time Series Forecasting), is designed to generate counterfactual explanations for time series forecasting in a scalable and principled manner. Unlike gradient-based approaches that solve an independent optimization problem for each instance, RL-MCF learns a *amortized policy* that maps any input to a counterfactual perturbation in a single forward pass.

The overall pipeline, illustrated in Figure 5.1, consists of three main components: two frozen pre-trained models and one trainable agent.

Given an input time series $\mathbf{x}$, the TCN encoder first maps it into a compact latent representation $\mathbf{z} = \phi(\mathbf{x})$. The actor-critic agent receives this latent code concatenated with the original forecast $\hat{\mathbf{y}} = f(\mathbf{x})$ as its state, and outputs a perturbation action $\mathbf{a}$ in the latent space. The perturbed latent vector is decoded back to the input space to produce the candidate $\tilde{\mathbf{x}} = \psi(\mathbf{z}^{cf})$. A temporal mask $\mathbf{m}$ is then applied to blend $\tilde{\mathbf{x}}$ with the original input, yielding the final counterfactual $\mathbf{x}^{cf}$ (Eq. 5.9). The forecasting model f evaluates $\mathbf{x}^{cf}$ to produce $\hat{\mathbf{y}}^{cf}$, used to compute the reward signal that drives the policy update.

The three main components of the framework are detailed in the following section:

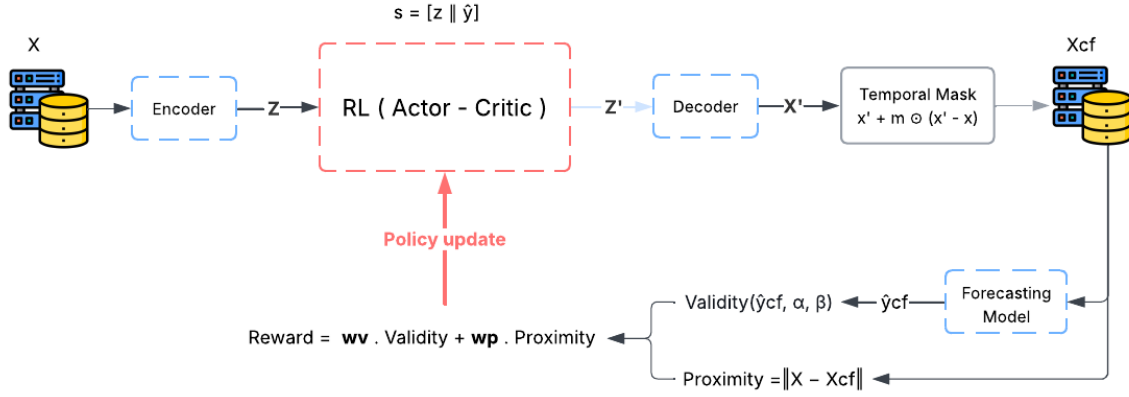


Figure 5.1: Overview of the RL-MCF framework

- **TCN Autoencoder** (ϕ, ψ) : provides a structured latent space that implicitly constrains counterfactuals to the data manifold. It uses 6 dilated causal convolutional layers with exponentially increasing dilation rates $(2^0, \dots, 2^5)$, yielding a receptive field of 127 timesteps that fully covers the look-back window ($L = 96$). A deterministic encoder is preferred over a VAE, as stochastic reparameterisation would destabilise the policy gradient signal during actor-critic training.
- **Forecasting Model** f : serves as a fixed evaluator to compute the validity reward signal.
- **Actor-Critic Agent** π_θ : learns an amortized counterfactual policy enabling single-pass inference at scale.

The temporal masking mechanism, which constrains perturbations to the most recent timesteps of the look-back window, is described in detail in Section 5.5.4. The reward function, the training procedure, and the inference algorithm are presented in Sections 5.5.5, 5.6.1, respectively.

5.5 Framework Components

This section details each component of the RL-MCF framework: the TCN autoencoder, the black-box forecasting model, the actor-critic agent, the temporal masking mechanism, and the reward function.

5.5.1 TCN Autoencoder

To ensure that generated counterfactual instances remain realistic and preserve temporal dependencies, we employ a Temporal Convolutional Network (TCN)-based autoencoder to learn a structured latent representation of the input time series. The autoencoder consists of two components:

- An **encoder** ϕ based on Temporal Convolutional Networks that maps the input time series $\mathbf{x} \in \mathbb{R}^L$ into a compact latent representation $\mathbf{z} \in \mathbb{R}^d$:

$$\mathbf{z} = \phi(\mathbf{x}) \quad (5.3)$$

The latent dimension d must be chosen carefully: too small a value causes the encoder to lose temporal detail; too large a value widens the search space unnecessarily, making the agent’s perturbation task harder. The selection of d is discussed in Chapter 6.

- A **decoder** ψ that reconstructs the time series from the latent representation:

$$\hat{\mathbf{x}} = \psi(\mathbf{z}) \quad (5.4)$$

The TCN encoder leverages dilated causal convolutions to capture both short-term and long-range temporal dependencies efficiently and in parallel, overcoming the sequential bottleneck of recurrent architectures. The autoencoder is trained independently to minimize the mean squared reconstruction loss:

$$\mathcal{L}_{rec} = \frac{1}{L} \|\mathbf{x} - \psi(\phi(\mathbf{x}))\|_2^2 \quad (5.5)$$

Once trained, the encoder and decoder are frozen. The latent space learned by the autoencoder plays a central role in our framework: by constraining all perturbations to this space, the decoder acts as a learned projection onto the data manifold, implicitly enforcing the plausibility of generated counterfactuals. Any modification produced in the latent space is guaranteed to be decoded into a temporally coherent sequence that respects the statistical structure of the training distribution.

5.5.2 Black-Box Forecasting Model

The forecasting model $f : \mathbb{R}^L \rightarrow \mathbb{R}^H$ is a pre-trained black-box predictor that maps any input sequence to a future trajectory. In our main experiments, we adopt the iTransformer Y. Liu et al. (2023) as the primary forecasting backbone, owing to its strong performance on multivariate time series benchmarks. However, the framework is fully model-agnostic: any forecasting architecture can be substituted without modifying the RL agent or the autoencoder.

Within the pipeline, the forecasting model serves as a fixed evaluator. It receives both the original input $\mathbf{x}$ and the counterfactual candidate $\mathbf{x}^{cf}$, producing the forecasts $\hat{\mathbf{y}} = f(\mathbf{x})$ and $\hat{\mathbf{y}}^{cf} = f(\mathbf{x}^{cf})$. These predictions are used to compute the validity component of the reward signal that guides the RL agent. Crucially, no gradient is propagated through the forecasting model during training — the agent interacts with it purely as a black box, querying it for output evaluations.

5.5.3 Actor-Critic Agent

The generation of counterfactual explanations is formulated as a sequential decision-making problem, where a reinforcement learning agent learns to perturb the latent representation of the input time series to satisfy the counterfactual objective. We adopt an actor-critic architecture, which combines the stability of value-based methods with the flexibility of policy gradient approaches.

State. The agent receives as state the concatenation of the latent code and the original forecast:

$$\mathbf{s} = [\mathbf{z} \parallel \hat{\mathbf{y}}] \in \mathbb{R}^{d+H} \quad (5.6)$$

This state representation provides the agent with both a compact encoding of the input history and the forecasting context it needs to assess how far the current prediction is from the target band.

Action. The **actor** network π_θ outputs a Gaussian distribution over perturbation actions $\mathbf{a} \in \mathbb{R}^d$ (the Gaussian is chosen because the latent perturbations are continuous and must be learned smoothly, it supports the reparameterisation trick required for policy gradient updates, and allows controlled exploration via its variance during training). A sample is drawn during training and the mean is used at inference:

$$\mathbf{a} \sim \pi_\theta(\mathbf{s}), \quad \mathbf{z}^{cf} = \text{clip}(\mathbf{z} + \eta \cdot \mathbf{a}, -1, 1) \quad (5.7)$$

where η is a step-size hyperparameter controlling the magnitude of latent perturbations. The clipping operation ensures that the perturbed latent code remains within the range observed during autoencoder training, preventing out-of-distribution decoding artifacts.

Critic. The **critic** network V_ω estimates the expected return from state $\mathbf{s}$:

$$V_\omega(\mathbf{s}) \approx \mathbb{E}[R \mid \mathbf{s}] \quad (5.8)$$

It is used to compute the advantage for policy gradient updates, reducing variance during training and stabilizing convergence. The perturbed latent code is decoded to obtain the counterfactual candidate $\tilde{\mathbf{x}} = \psi(\mathbf{z}^{cf})$, which is then passed through the temporal masking mechanism to produce the final counterfactual $\mathbf{x}^{cf}$.

5.5.4 Temporal Masking Mechanism

A key limitation of existing counterfactual generation methods for time series is the tendency to produce global modifications across the entire input sequence. Such changes are often unrealistic and difficult to interpret, as they imply altering historical observations at every timestep. To address this issue, we introduce a structured temporal masking mechanism that restricts counterfactual modifications to the most recent and relevant region of the look-back window.

Decoding $\mathbf{z}^{cf}$ produces a full-length sequence modification $\tilde{\mathbf{x}}$. Rather than using this directly as the counterfactual, we apply a temporal mask $\mathbf{m} \in [0, 1]^L$ that blends the decoded sequence with the original input:

$$\mathbf{x}^{cf} = \mathbf{x} + \mathbf{m} \odot (\tilde{\mathbf{x}} - \mathbf{x}) \quad (5.9)$$

Figure 5.2 illustrates this masking effect on ETTh1.

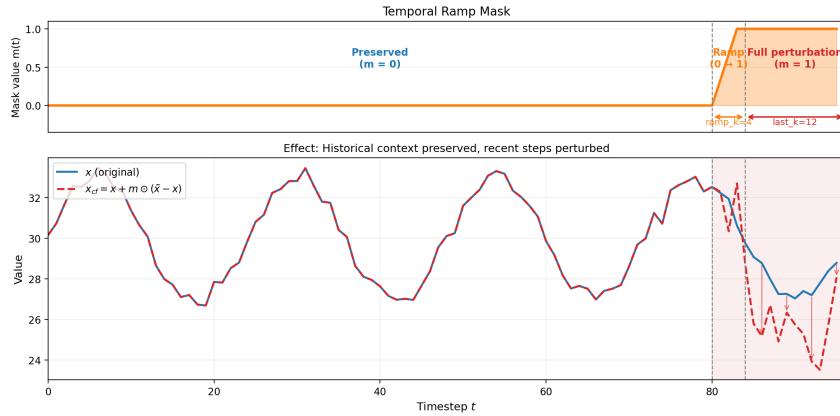


Figure 5.2: Temporal ramp mask illustration .

The mask is defined as a ramp followed by a full-activation window, concentrating modifications on the k most recent timesteps:

$$m_t = \begin{cases} 0 & \text{if } t < L - k - r \\ \frac{t - (L - k - r)}{r} & \text{if } L - k - r \leq t < L - k \\ 1 & \text{if } t \geq L - k \end{cases} \quad (5.10)$$

where k is the number of fully active timesteps and r is the length of the smooth ramp transition. This design ensures that only the k most recent observations are modified, while earlier timesteps are preserved exactly as in the original input. The smooth ramp avoids abrupt transitions that could introduce artificial discontinuities in the generated sequence.

This masking mechanism directly addresses the global perturbation limitation of ForecastCF by producing counterfactuals that are:

- **Sparse:** only a small fraction of timesteps are modified, directly improving the Compactness metric.
- **Localized:** modifications are concentrated on the most recent and forecasting-relevant region of the sequence.
- **Temporally coherent:** the preserved early timesteps maintain the original temporal structure, improving Temporal Consistency.
- **Actionable:** the modified region directly indicates where and when an intervention is needed to steer the forecast toward the target band.

5.5.5 Reward Function

The reward signal combines two terms reflecting the two directly optimised counterfactual criteria: validity and proximity. Plausibility is not included as an explicit reward term — it is implicitly enforced by constraining perturbations to the latent space of the TCN autoencoder.

Target Bounds. To define validity, we first specify the target band $[\alpha, \beta]$ anchored on the original forecast $\hat{\mathbf{y}}$. For a downward target:

$$\beta_t = \hat{y}_t - \rho \cdot \sigma_x, \quad \alpha_t = \beta_t - f_r \cdot \sigma_x \quad (5.11)$$

Validity Term. The validity term r_{valid} measures how close the counterfactual forecast $\hat{\mathbf{y}}^{cf} = f(\mathbf{x}^{cf})$ is to the target band $[\alpha, \beta]$. Rather than using a hard binary indicator, we adopt a soft exponential formulation that provides a smooth gradient signal throughout training:

$$r_{\text{valid}} = \frac{1}{H} \sum_{t=1}^H \exp \left(-2 \cdot \frac{d_t}{\beta_t - \alpha_t} \right) \quad (5.12)$$

where $d_t = \max(0, \alpha_t - \hat{y}_t^{cf}) + \max(0, \hat{y}_t^{cf} - \beta_t)$ is the distance of the counterfactual forecast from the feasible interval at timestep t . When $\hat{y}_t^{cf} \in [\alpha_t, \beta_t]$, we have $d_t = 0$ and $r_{\text{valid}} = 1$. As the forecast moves away from the target band, the reward decays exponentially, providing a continuous incentive for the agent to steer predictions toward the admissible region.

Proximity Term. The proximity term r_{prox} penalizes the distance between the original input and the counterfactual, encouraging minimal and actionable perturbations:

$$r_{\text{prox}} = \exp \left(-\frac{1}{L} \sum_{t=1}^L |x_t^{cf} - x_t| \right) \quad (5.13)$$

The exponential formulation maps the ℓ_1 distance to a reward in $[0, 1]$, with $r_{\text{prox}} = 1$ when $\mathbf{x}^{cf} = \mathbf{x}$ and decaying smoothly as the perturbation grows. This formulation is consistent with the proximity loss used in ForecastCF Li et al. (2026), enabling direct comparison.

5.6 Training and Inference Procedures

5.6.1 Training Procedure

The training process consists of two stages: an offline pre-training phase and a reinforcement learning phase. During pre-training, the forecasting model f and the TCN autoencoder (ϕ, ψ) are trained independently on the target dataset and then frozen. The RL agent is then trained on top of these fixed components, as described in Algorithm 1.

Algorithm 1: RL-MCF: Training Procedure

Input: Training set $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$; Frozen forecaster f ; Frozen autoencoder (ϕ, ψ) ; Actor π_θ ;

Critic V_ϕ ; Hyperparameters w_v, w_p, η, k, r

Output: Trained actor policy π_θ

for $t = 1$ **to** T **do**

 Sample $\mathbf{x} \sim \mathcal{D}$;

$\mathbf{z} \leftarrow \phi(\mathbf{x})$;

$\hat{\mathbf{y}} \leftarrow f(\mathbf{x})$;

$\mathbf{s} \leftarrow [\mathbf{z} \parallel \hat{\mathbf{y}}]$;

$\mathbf{a} \sim \pi_\theta(\mathbf{s})$;

$\mathbf{z}^{cf} \leftarrow \text{clip}(\mathbf{z} + \eta \cdot \mathbf{a}, -1, 1)$;

$\tilde{\mathbf{x}} \leftarrow \psi(\mathbf{z}^{cf})$;

$\mathbf{x}^{cf} \leftarrow \mathbf{x} + \mathbf{m} \odot (\tilde{\mathbf{x}} - \mathbf{x})$;

$\hat{\mathbf{y}}^{cf} \leftarrow f(\mathbf{x}^{cf})$;

$r \leftarrow w_v \cdot r_{\text{valid}}(\hat{\mathbf{y}}^{cf}) + w_p \cdot r_{\text{prox}}(\mathbf{x}^{cf}, \mathbf{x})$;

 Update V_ω : minimize $\mathcal{L}_{\text{critic}} = (r - V_\omega(\mathbf{s}))^2$;

 Update π_θ : maximize $\mathcal{L}_{\text{actor}} = (r - V_\omega(\mathbf{s})) \log \pi_\theta(\mathbf{a} \mid \mathbf{s})$;

return π_θ

5.6.2 Inference Procedure

Once training is complete, the actor policy π_θ generates counterfactuals in a single forward pass for any new input, as described in Algorithm 2.

Algorithm 2: RL-MCF: Inference Procedure

Input: Input time series $\mathbf{x}$; Frozen forecaster f ; Frozen autoencoder (ϕ, ψ) ; Trained actor π_θ ;

Temporal mask $\mathbf{m}$

Output: Counterfactual $\mathbf{x}^{cf}$; Counterfactual forecast $\hat{\mathbf{y}}^{cf}$

$\mathbf{z} \leftarrow \phi(\mathbf{x})$;

$\hat{\mathbf{y}} \leftarrow f(\mathbf{x})$;

$\mathbf{s} \leftarrow [\mathbf{z} \parallel \hat{\mathbf{y}}]$;

$\mathbf{a} \leftarrow \mathbb{E}[\pi_\theta(\mathbf{s})]$;

$\mathbf{z}^{cf} \leftarrow \text{clip}(\mathbf{z} + \eta \cdot \mathbf{a}, -1, 1)$;

$\tilde{\mathbf{x}} \leftarrow \psi(\mathbf{z}^{cf})$;

$\mathbf{x}^{cf} \leftarrow \mathbf{x} + \mathbf{m} \odot (\tilde{\mathbf{x}} - \mathbf{x})$;

$\hat{\mathbf{y}}^{cf} \leftarrow f(\mathbf{x}^{cf})$;

return $(\mathbf{x}^{cf}, \hat{\mathbf{y}}^{cf})$

5.7 Evaluation Metrics

The qualitative definitions of the counterfactual evaluation criteria — validity, proximity, plausibility, sparsity, and coverage — were introduced in Section 4.6 of Chapter 4. Here, we provide their precise mathematical formulations as instantiated in the RL-MCF framework.

We evaluate counterfactual quality along six complementary axes: validity ratio, stepwise validity AUC, proximity, compactness, temporal consistency, and plausibility.

5.7.1 Validity Ratio

The validity ratio directly operationalizes the range-based validity objective introduced by Z. Wang et al. (2023), adapted here to the multi-step, multi-instance evaluation setting. For each instance k and timestep t , we define a binary validity indicator:

$$v_{k,t} = \begin{cases} 1 & \text{if } \alpha_{k,t} \leq f(\mathbf{x}_k^{cf})_t \leq \beta_{k,t} \\ 0 & \text{otherwise} \end{cases} \quad (5.14)$$

The validity ratio is then the proportion of valid timesteps across all instances and horizon steps:

$$\text{Validity} = \frac{1}{K \cdot H} \sum_{k=1}^K \sum_{t=1}^H v_{k,t} \quad (5.15)$$

where K is the number of test instances and H the forecasting horizon. A value of 1 indicates that all predicted steps satisfy the range constraint for all instances.

5.7.2 Stepwise Validity AUC

The standard validity ratio aggregates all timesteps into a single scalar, obscuring whether a counterfactual satisfies the constraint uniformly across the horizon or only partially. To capture this finer-grained information, we introduce the **Stepwise Validity AUC** as an original contribution of this work.

For each instance k , we first compute the number of valid timesteps using the indicator $v_{k,t}$ defined in Section 5.7.1:

$$c_k = \sum_{t=1}^H v_{k,t} \in \{0, 1, \dots, H\} \quad (5.16)$$

We then define the cumulative validity curve $\phi(\tau)$ as the fraction of instances for which at least τ timesteps satisfy the constraint:

$$\phi(\tau) = \frac{1}{K} \sum_{k=1}^K \mathbf{1}[c_k \geq \tau], \quad \tau \in \{0, 1, \dots, H\} \quad (5.17)$$

By construction, $\phi(0) = 1$ (all instances have at least 0 valid steps) and $\phi(H) \leq 1$ (only fully valid instances reach $\tau = H$). The Stepwise Validity AUC is the normalized area under this curve:

$$\text{AUC} = \frac{1}{H} \sum_{\tau=0}^H \phi(\tau) \quad (5.18)$$

Note that the discrete summation replaces the continuous integral, since τ takes integer values in $\{0, \dots, H\}$. A higher AUC indicates that counterfactuals are valid across a larger fraction of the horizon per instance, penalizing methods that achieve high average validity by concentrating valid steps on easy timesteps.

5.7.3 Proximity

Proximity is a standard metric in the counterfactual explanation literature Wachter, Mittelstadt, and Russell (2017) and Z. Wang et al. (2023), measuring how much the counterfactual deviates from the original input:

$$\text{Proximity} = \frac{1}{K} \sum_{k=1}^K \left\| \mathbf{x}_k^{cf} - \mathbf{x}_k \right\|_2 \quad (5.19)$$

Lower values indicate more actionable and minimal explanations.

5.7.4 Compactness

Compactness is an adaptation of the sparsity criterion defined qualitatively in Section 4.6, reformulated here as a continuous proportion rather than a raw count. For each instance k and timestep t , we define a binary unchanged indicator:

$$u_{k,t} = \begin{cases} 1 & \text{if } |x_{k,t}^{cf} - x_{k,t}| \leq \epsilon \\ 0 & \text{otherwise} \end{cases} \quad (5.20)$$

where $\epsilon = 10^{-3}$ by default. A timestep is considered effectively unchanged when its perturbation falls below this tolerance threshold. Compactness is then the fraction of unchanged timesteps across all instances and the full look-back window of length L :

$$\text{Compactness} = \frac{1}{K \cdot L} \sum_{k=1}^K \sum_{t=1}^L u_{k,t} \quad (5.21)$$

This formulation directly quantifies the effect of the temporal masking mechanism introduced in Section 5.5.4: a higher compactness score reflects more localized perturbations, confirming that modifications are concentrated on the most recent actionable timesteps rather than spread across the full look-back window.

5.7.5 Temporal Consistency

Temporal consistency is an original metric introduced in this work to address a limitation specific to the time series setting: existing counterfactual evaluation frameworks do not assess whether generated sequences preserve the directional dynamics of the original input. For each instance, we compute the Pearson correlation between the first-order differences of both sequences:

$$\text{TC}_k = \frac{1}{2} \left(1 + \text{Pearson}(\Delta \mathbf{x}_k, \Delta \mathbf{x}_k^{cf}) \right) \quad (5.22)$$

where $\Delta \mathbf{x}_t = x_{t+1} - x_t$ denotes the first-order difference operator. The $\frac{1}{2}(1 + \cdot)$ normalization maps the Pearson correlation from $[-1, 1]$ to $[0, 1]$, where 1 indicates that the counterfactual follows the same local trends as the original series and 0 indicates a fully inverted dynamics. This metric is particularly relevant for the temporal masking mechanism, which is designed to preserve early timesteps and concentrate modifications on the recent window.

5.7.6 Plausibility

Plausibility operationalizes the distributional realism criterion introduced in Section 4.6. Following Li et al. (2026), who employ an ensemble of anomaly detectors to assess the realism of generated counterfactuals

in the classification setting, we adopt the same three detectors — Isolation Forest (IF), Local Outlier Factor (LOF), and One-Class SVM (OC-SVM) — and adapt their ensemble scoring to the forecasting setting. Each detector is trained on the training set and produces a normalized anomaly score in $[0, 1]$, where lower values indicate that the counterfactual is closer to the training distribution. The ensemble score is computed as their arithmetic mean:

$$\text{Plausibility} = \frac{1}{3} (s_{\text{IF}} + s_{\text{LOF}} + s_{\text{OC-SVM}}) \quad (5.23)$$

The ensemble design mitigates the risk of any single detector failing to characterize the training distribution boundary, particularly for datasets with high residual variance such as Weather. Since plausibility is implicitly encouraged by constraining perturbations to the latent space of the autoencoder, we expect our method to produce lower anomaly scores than gradient-based approaches operating directly in the input space.

5.8 Conclusion

This chapter presented the complete formulation of the RL-MCF framework. Starting from the limitations of gradient-based and instance-specific approaches, we defined a counterfactual forecasting objective that jointly targets validity and proximity through a composite reward, while implicitly promoting compactness, temporal consistency, and distributional plausibility through architectural design choices rather than explicit optimisation terms. The proposed framework addresses each identified limitation through three complementary components: a TCN-based autoencoder that confines all modifications to the learned data manifold, implicitly enforcing plausibility without requiring explicit distributional constraints; a temporal masking mechanism that enforces sparse and localised perturbations on the most recent actionable timesteps, directly improving compactness and temporal consistency; and an actor-critic agent that learns an amortized counterfactual policy, enabling efficient single-pass generation and eliminating the need for instance-specific optimisation. The following chapter details the full implementation and independent validation of each pipeline component.

Chapter 6

Implementation and Experimental Setup

6.1 Introduction

With the methodology established in the previous chapter, this chapter turns to its concrete realization. It covers the full implementation of the **RL-MCF** framework from the experimental environment and benchmark datasets to the independent training and validation of each pipeline component. Each module is frozen once validated before being integrated into the end-to-end counterfactual generation pipeline.

Three benchmark datasets were selected to cover a diverse range of temporal behaviors and signal characteristics, ensuring that the framework is evaluated under genuinely varied conditions. For each pipeline component, we detail its configuration, implementation, and provide guidelines to reproduce the results from our public repository detailed results and analysis for each component are presented in Chapter 7.

The chapter is organized as follows. Section 6.2 describes the development environment and software stack. Section 6.3 presents the three benchmark datasets and their temporal structure as revealed by STL decomposition analysis. Section 6.4 details the training and validation of the five forecasting architectures. Section 6.5 covers the TCN autoencoder, which provides the latent space in which counterfactual perturbations are performed. Section 6.6 describes the ensemble anomaly detectors used to assess plausibility at evaluation time. Finally, Section 6.7 presents the full RL-MCF pipeline — from hyperparameter sensitivity analysis and configuration to implementation and reproduction guidelines — with the corresponding results also reported in Chapter 7.

6.2 Development Environment

The **RL-MCF** framework was implemented in Python 3.10 and developed using Visual Studio Code. All deep learning components were built on top of PyTorch, while data manipulation, statistical analysis, and anomaly detection relied on the standard scientific Python ecosystem. Table 6.1 summarises the full software stack. Computationally intensive experiments were executed on Google Colaboratory using an NVIDIA Tesla T4 GPU (15 GB VRAM), which provided sufficient memory for training all five forecasting architectures as well as the TCN autoencoder and the reinforcement learning agent. Version control was managed via Git, and the full codebase is publicly available at:

<https://github.com/bouchrabenkhelifa/XAI-Counterfactual-TS-Forecasting-RLMCF>

Table 6.1: Libraries and frameworks used throughout the project.

Library	Version	Role
PyTorch	$\geq 2.0.0$	Deep learning framework
NumPy	$\geq 1.24.0$	Numerical computation
Pandas	$\geq 1.5.0$	Data loading and preprocessing
Scikit-learn	$\geq 1.2.0$	Anomaly detectors, scaling
SciPy	$\geq 1.9.0$	Signal processing utilities
Matplotlib	$\geq 3.6.0$	Visualisation
Einops	$\geq 0.6.0$	Tensor manipulation for Transformers
Reformer	$\geq 1.4.4$	Efficient attention implementation
tqdm	$\geq 4.64.0$	Progress tracking

6.3 Datasets

The experimental evaluation is conducted on three publicly available benchmark datasets spanning two distinct domains: energy systems and meteorology. They were selected to reflect a diversity of temporal structures, sampling frequencies, and signal complexities, ensuring that the RL-MCF framework is evaluated under genuinely varied conditions. Table 6.2 summarises their key characteristics.

Table 6.2: Summary of datasets used in the experiments.

Dataset	Frequency	Length	Target variable	Domain
ETTh1	Hourly	17,420	OT (oil temperature)	Energy
ETTh2	Hourly	17,420	OT (oil temperature)	Energy
Weather	10-minute	52,696	WetBulbCelsius	Meteorology

To inform the configuration choices made in the next sections, each dataset is subjected to an STL (Seasonal and Trend decomposition using Loess) decomposition that separates the signal into its trend, seasonal, and residual components. This analysis directly guides the selection of key hyperparameters notably the look-back window L , the forecasting horizon H , the band width f_r , and the number of active mask steps k — by revealing the dominant temporal structure of each dataset. The results are presented below, one dataset at a time.

6.3.1 ETTh1

ETTh1¹ records oil temperature from an electrical transformer station in China at hourly resolution over approximately two years. It is the primary dataset used throughout this work and serves as the reference configuration for all ablation and sensitivity analyses.

Figure 6.1 shows the STL decomposition with period = 24 hours. The series is strongly trend-dominated: the trend component alone accounts for $\approx 86\%$ of the total variance, reflecting a slow and persistent downward drift over the observation window. The seasonal component ($\approx 7\%$) captures a stable daily cycle driven by the 24-hour operational rhythm of the transformer station, while the residual ($\approx 6\%$) is the smallest among the three datasets — confirming that the signal is well-explained by its structural components with minimal unexplained noise.

From a modeling perspective, the low residual variance implies that a well-trained forecaster should achieve low prediction error, a necessary condition for reliable counterfactual validity signals during RL training. A look-back window of $L = 96$ (four complete daily cycles) and a forecasting horizon of $H = 48$ (two cycles ahead) are directly motivated by this structure, as derived in Section 6.4.2.

¹ETTh1 Dataset



Figure 6.1: STL decomposition of the ETTh1 target variable (OT, period = 24 h).

6.3.2 ETTh2

ETTh2² shares the same schema and recording period as ETTh1 but originates from a different transformer station, exhibiting meaningfully different temporal dynamics.

Figure 6.2 shows the STL decomposition under the same configuration (period = 24 hours). The most striking difference from ETTh1 is the seasonal component, which accounts for $\approx 20\%$ of the total variance — nearly three times its contribution in ETTh1 — indicating that daily cycles play a far more prominent role in driving the signal. The trend component ($\approx 65\%$) remains dominant but less overwhelming, and the residual ($\approx 10\%$) is noticeably larger, reflecting greater short-term variability and structural complexity.

These differences have direct implications for counterfactual generation. A stronger seasonal component means the forecasting model must track more pronounced oscillations, making it harder to steer predictions into a target band through localised input perturbations. Together with the larger residual, these factors explain why ETTh2 is systematically harder than ETTh1.

6.3.3 Weather

The Weather dataset³ records 21 meteorological variables at 10-minute intervals across a full year, with wet-bulb temperature (**WetBulbCelsius**) as the forecasting target. Its higher sampling frequency, larger variate count, and distinct signal structure make it a more challenging and complementary testbed.

Figure 6.3 shows the STL decomposition with period = 24 h (= 144 steps at 10-minute resolution). The Weather dataset exhibits a fundamentally different profile from both ETT datasets. While the trend component remains dominant ($\approx 76\%$), reflecting the gradual seasonal evolution of atmospheric temperature across the year, the periodic component is almost negligible ($\approx 2.3\%$): daily cycles contribute minimally to the overall dynamics, a direct consequence of the physical nature of wet-bulb temperature, which is governed by synoptic-scale weather patterns rather than fixed daily rhythms. The residual

²ETTh2 Dataset

³Weather Dataset

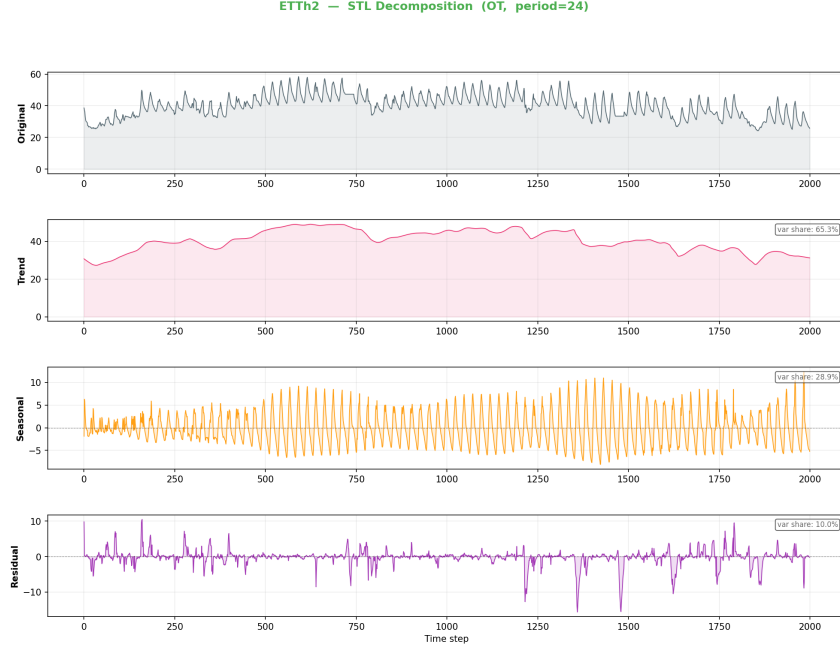


Figure 6.2: STL decomposition of the ETTh2 target variable (OT, period = 24 h).

($\approx 13\%$) is the largest among the three datasets, indicating that a substantial fraction of the signal is driven by irregular atmospheric events that neither trend nor seasonality can capture.

This structural profile — trend-dominated, seasonality-poor, and noise-rich — makes the Weather dataset a genuinely distinct testbed and justifies dedicated hyperparameter configurations, particularly regarding the band width f_r and the number of active mask steps k .

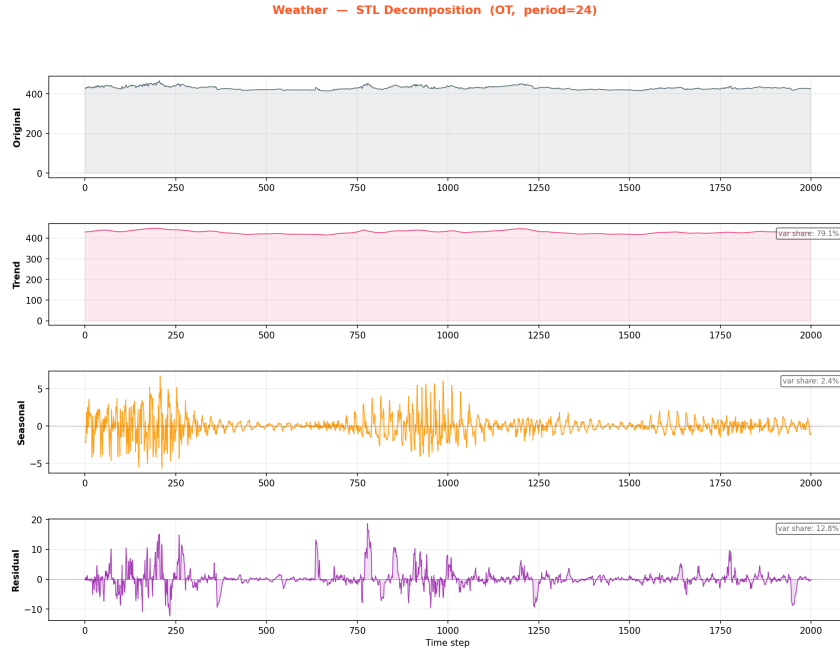


Figure 6.3: STL decomposition of the Weather target variable.

6.3.4 Synthesis

The three datasets collectively span a broad spectrum of temporal complexity. Table 6.3 consolidates the key structural characteristics extracted from the STL decompositions and the configuration choices they motivate. While the core architecture and hyperparameters of RL-MCF remain fixed across all datasets, the STL analysis reveals structural differences that justify minimal but targeted adjustments — specifically the forecasting horizon H and the band width f_r — to account for each dataset’s distinct temporal regime. All configuration files are available in the repository under **assets/configs/**.

Table 6.3: Synthesis of STL decomposition results and modeling implications.

Dataset	Trend (%)	Seasonal (%)	Residual (%)	Dominant structure	Difficulty	Key implication
ETTh1	≈ 86	≈ 7	≈ 6	Trend	Low	$L=96$, $k=12$, standard config
ETTh2	≈ 65	≈ 20	≈ 10	Trend + Seasonal	Medium	Larger perturbations needed
Weather	≈ 76	≈ 2.3	≈ 13	Trend + Residual	High	Higher f_r , adjusted k

6.3.5 Data Preparation

Raw data is stored as CSV files containing a date column and one column per feature variable. In univariate mode (`features='S'`), only the target column is retained:

```

1 if self.features == 'S':
2     df_data = df_raw[[self.target]]
3     # "OT" for ETT, "T (degC)" for Weather

```

Listing 6.1: Univariate target selection.

Chronological split. Splits are strictly chronological to preserve temporal causality — no shuffling is applied at any stage. The partition boundaries for ETTh1/ETTh2 are fixed as follows:

```

1 # 12 months train / 4 months val / 4 months test

```

```

2 border1s = [0,
3             12*30*24 - seq_len,
4             12*30*24 + 4*30*24 - seq_len]
5 border2s = [12*30*24,
6             12*30*24 + 4*30*24,
7             12*30*24 + 8*30*24]

```

Listing 6.2: Chronological split boundaries (ETTh1/ETTh2).

Table 6.4: Dataset split sizes after windowing ($L = 96$, $H = 48$).

Dataset	Train	Validation	Test
ETTh1 / ETTh2	8 640 samples (70%)	2 880 samples (10%)	2 880 samples (20%)
Weather	36 696 samples (70%)	5 242 samples (10%)	10 444 samples (20%)

Normalisation. A `StandardScaler` is fitted exclusively on the training split and then applied to the full dataset, preventing any data leakage:

```

1 scaler = StandardScaler()
2 scaler.fit(train_data.values)           # mu, sigma computed on train only
3 data = scaler.transform(df_data.values) # applied to train + val + test

```

Listing 6.3: Leak-free normalisation pipeline.

Each feature is zero-centred ($\mu = 0$) and unit-scaled ($\sigma = 1$). The fitted scaler is saved alongside the autoencoder checkpoint to allow de-normalisation at inference time.

Sliding window. The normalised series is segmented into overlapping windows with a stride of 1:

```

1 seq_x = data[i : i + seq_len]           # input: (96, 1)
2 seq_y = data[i + seq_len - label_len :
3             i + seq_len + pred_len]     # target: (96, 1)

```

Listing 6.4: Sliding window segmentation.

Temporal encoding. Two encoding schemes are used depending on the model family:

```

1 # timeenc=1: frequency encoding for iTransformer and PatchTST
2 data_stamp = time_features(dates, freq='h')
3 # -> [hour_sin, hour_cos, day_sin, day_cos, ...]
4
5 # timeenc=0: raw encoding for GRU, TimesNet, DLinear
6 data_stamp = [month, day, weekday, hour]

```

Listing 6.5: Temporal encoding schemes.

DataLoader. Training batches are shuffled; validation and test sets are evaluated in order:

```

1 DataLoader(dataset,
2             batch_size=32,
3             shuffle=(flag == 'train'),
4             drop_last=True)

```

Listing 6.6: DataLoader configuration.

6.4 Forecasting Model Training

The counterfactual generation framework requires a frozen, reliable forecaster to evaluate whether a perturbed input produces a forecast that falls within the target band $[\alpha, \beta]$. The quality of this validity signal depends directly on the forecaster’s accuracy: a poorly calibrated model would produce noisy or misleading rewards during RL training, undermining the entire pipeline. Five architectures are therefore trained and evaluated independently on each dataset before being frozen and integrated into the RL loop.

6.4.1 Model Overview

The five architectures span the main families of deep time series models surveyed in Chapter 2, and were selected to cover a representative range of inductive biases — from attention-based to convolutional, recurrent, and linear models. Table 6.5 summarises their key characteristics.

Table 6.5: Summary of the five forecasting architectures trained in this work.

Model	Family	Tokenisation	Key mechanism
iTransformer	Transformer	Variable-wise	Cross-variate attention; primary RL-MCF backbone
PatchTST	Transformer	Time patch	Patch self-attention over local segments
TimesNet	CNN	1-D $\rightarrow$ 2-D	2-D temporal convolution
GRU	Recurrent	Timestep-wise	Gated recurrent units
DLinear	Linear	Trend/Residual	Decomposed linear projection

Each model is trained independently on each of the three datasets using a common configuration ($L = 96$, $H = 48$, univariate target — justified in Section 6.4.2) and then frozen. The iTransformer serves as the primary backbone for RL-MCF, as its inverted tokenisation is particularly well-suited to the trend-dominated structure shared by all three datasets (Section 6.3.4).

6.4.2 Observation Window and Forecasting Horizon Selection

The choice of look-back window L and forecasting horizon H is grounded directly in the temporal structure revealed by the STL and ACF analyses. As established in Section 6.3, the dominant periodicity is 24 steps for ETTh1 and ETTh2 (hourly resolution) and 144 steps for Weather (10-minute resolution), both corresponding to a 24-hour daily cycle.

Step 1 — Dominant period. The ACF analysis presented in Figure 6.4 confirms a clear periodic peak at lag ≈ 24 for the ETT datasets and at lag ≈ 144 for Weather. This daily structure is physically consistent with the nature of the data: ETTh1 and ETTh2 follow the industrial load profile of transformer stations H. Zhou et al. (2021), while wet-bulb temperature is governed by the diurnal solar cycle. Importantly, the near-absent seasonality of the Weather dataset ($\approx 2.3\%$ of variance, Section 6.3.3) means that this dominant period is far weaker in practice — a limitation acknowledged in the dedicated configuration for that dataset.

Step 2 — Observation window. We adopt the standard rule of thumb that the look-back window should span at least four complete periods of the dominant cycle, giving the model sufficient context to identify the periodic structure and separate it from noise:

$$L = 4 \times T_{\text{dominant}} \quad (6.1)$$

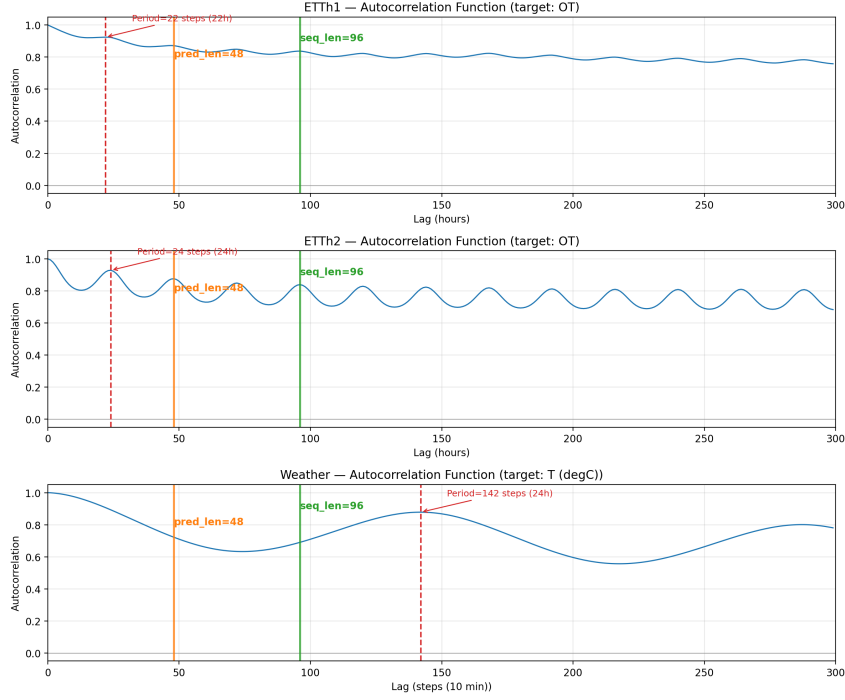


Figure 6.4: Autocorrelation function (ACF) of the target variable for each dataset .

For ETTh1 and ETTh2, this yields $L = 4 \times 24 = 96$ steps — four complete daily cycles, consistent with the strong and stable seasonality identified for these datasets ($\approx 7\%$ and $\approx 20\%$ of variance respectively). The same value $L = 96$ is applied to Weather for cross-dataset comparability: it is the standard benchmark configuration used by iTransformer Y. Liu et al. (2023), PatchTST Nie et al. (2023), and TimesNet. For Weather, this covers only $96/144 \approx 0.67$ daily cycles — a known limitation given its negligible seasonality, but one that is offset by the trend-dominated nature of the signal (Section 6.3.3).

Step 3 — Forecasting horizon. For ETTh1 and ETTh2, the prediction horizon is set to two dominant periods:

$$H = 2 \times T_{\text{dominant}} = 2 \times 24 = 48 \text{ steps} \quad (6.2)$$

This corresponds to a 48-hour (two-day) forecast — a practically meaningful horizon for energy management applications and one that enables direct comparison with ForecastCF Z. Wang et al. (2023), which was evaluated under identical conditions. For Weather, the horizon is extended to $H = 96$ steps, consistent with the standard benchmark configuration used by iTransformer Y. Liu et al. (2023) and PatchTST Nie et al. (2023), and to capture a full daily cycle given the 10-minute resolution of the dataset.

Table 6.6: Observation window and forecasting horizon for each dataset.

Dataset	Resolution	Dominant period	In hours	seq_len	pred_len
ETTh1	60 min	24 steps	24 h	96	48
ETTh2	60 min	24 steps	24 h	96	48
Weather	10 min	144 steps	24 h	96	96

6.4.3 Configuration

All five models share the same experimental protocol. Model-specific hyperparameters (model dimension, number of layers, attention heads) were selected via grid search and stored in the configuration files under `assets/configs/{dataset}_dataset/forecasters/`. The shared training configuration is reported in Table 6.7.

Table 6.7: Shared training configuration for all forecasting models.

Hyperparameter	Value	Applies to
Look-back window L	96	All models
Forecasting horizon H	48/96	ETTh1, ETTh2/Weather
Target variable	Univariate	All models
Optimiser	Adam	All models
Loss function	MSE	All models
Early stopping patience	10	All models
Batch size	32	All models
Learning rate	1×10^{-4}	All models
Max epochs	50	All models
Normalisation	RevIN	iTransformer, PatchTST

RevIN is applied only to iTransformer and PatchTST, whose attention mechanisms are sensitive to distribution shift across windows — a particularly relevant consideration for ETTh1, whose strong downward trend ($\approx 86\%$ of variance) means that the mean of the series drifts substantially over the observation period.

6.4.4 Implementation

All five forecasters share a single training entry point, `src/experiments/forecasting/run.py`, with a unified `GenericForecasterTrainer` that is instantiated dynamically from a JSON configuration file — only the configuration file changes between runs, with no modification to the training code itself.

```
1 configs = load_config(args.config)
2 model_type = configs.model_type.lower()
3 trainer = GenericForecasterTrainer(configs)
4 trainer.train()
```

Listing 6.7: Automatic trainer routing based on `model_type`.

```

1 class GenericForecasterTrainer:
2     def __init__(self, configs):
3         self.model = _build_model(configs, device)
4         self.optimizer = Adam(self.model.parameters(),
5                               lr=configs.learning_rate)
6         self.criterion = MSELoss()
7
8     def train(self):
9         early_stopping = EarlyStopping(patience=configs.patience)
10        for epoch in range(configs.train_epochs):
11            for batch_x, batch_y, batch_x_mark, batch_y_mark in train_loader:
12                outputs = self.model(batch_x, batch_x_mark)
13                loss = self.criterion(outputs[:, -pred_len:],
14                                     batch_y[:, -pred_len:])
15                loss.backward()
16                self.optimizer.step()
17            val_loss = self.validate(val_loader)
18            early_stopping(val_loss, self.model, checkpoint_path)
19            if early_stopping.early_stop:
20                break

```

Listing 6.8: GenericForecasterTrainer — common training loop.

6.4.5 Results

All five forecasting architectures achieve sufficient predictive accuracy to serve as reliable black-box evaluators for the RL training loop. The iTransformer results across all three datasets are reported in Chapter 7, Section 7.3.1, where a detailed analysis of the performance is also provided.

6.4.6 Reproducing the Results

Each of the three datasets is paired with five forecasting architectures, yielding fifteen independent training runs in total. Each run is fully reproducible via the following command:

```

1 python src/experiments/forecasting/run.py \
2     --config assets/configs/{DATASET}_dataset/forecasters/
3     {MODEL}/{DATASET}_96_48_S.json

```

Listing 6.9: Training command.

where $\{\text{DATASET}\} \in \{\text{etth1}, \text{etth2}, \text{weather}\}$ and $\{\text{MODEL}\} \in \{\text{itransformer}, \text{patchtst}, \text{timesnet}, \text{gru}, \text{dlinear}\}$.

6.5 TCN Autoencoder Training

The TCN autoencoder serves as the backbone of the latent-space perturbation strategy introduced in Chapter 5. Rather than modifying the raw input series directly, the RL agent operates in the compact, bounded latent space $\mathcal{Z} = [-1, 1]^d$ learned by the autoencoder — a design choice that implicitly constrains generated counterfactuals to remain close to the training distribution, without requiring any explicit plausibility term in the reward function. The autoencoder is trained independently on each dataset and then frozen before RL training begins.

6.5.1 Architecture and Implementation

The encoder φ maps an input time series $\mathbf{x} \in \mathbb{R}^{T \times C}$ to a latent vector $\mathbf{z} \in [-1, 1]^d$ via a stack of N dilated temporal convolutional blocks. Exponentially increasing dilation rates give the encoder a receptive field of $2^N - 1$ time steps, sufficient to capture long-range dependencies across the full input window of $T = 96$ — a window size motivated by the dominant 24-hour periodicity identified in Section 6.3.

```

1 class TCNEncoder(nn.Module):
2     def __init__(self, seq_len, n_features=1, hidden_dim=64,
3                   latent_dim=64, n_blocks=6, kernel_size=3,
4                   dropout=0.1):
5         super().__init__()
6         self.proj = nn.Conv1d(n_features, hidden_dim, 1)
7         self.blocks = nn.ModuleList([
8             TCNBlock(hidden_dim, kernel_size, 2**i, dropout)
9             for i in range(n_blocks)
10        ])
11        self.fc = nn.Linear(hidden_dim, latent_dim)
12        self.tanh = nn.Tanh()
13
14    def forward(self, x):
15        out = self.proj(x.transpose(1, 2))
16        for b in self.blocks:
17            out = b(out)
18        return self.tanh(self.fc(out.mean(2)))

```

Listing 6.10: TCN encoder architecture.

The `Tanh` activation at the output bounds the latent vector in $[-1, 1]^d$, which is critical for the RL agent: the actor samples perturbations within this bounded space, and the proximity penalty $\rho \cdot \sigma_{\text{global}}$ provides a principled clipping radius that prevents the agent from escaping the decoder’s training distribution.

6.5.2 Configuration

Training minimises the mean squared reconstruction loss over windows of length $L = 96$:

$$\mathcal{L}_{\text{rec}} = \frac{1}{L} \|\mathbf{x} - \psi(\varphi(\mathbf{x}))\|_2^2 \quad (6.3)$$

The main configuration choices are driven by the temporal characteristics established in Section 6.3:

- **Latent dimension d :** selected via grid search to balance reconstruction quality and the compactness of the search space available to the RL agent. A latent space that is too small loses the temporal detail needed to reconstruct the seasonal oscillations present in ETTh2 ($\approx 20\%$ seasonal variance); one that is too large makes the agent’s perturbation task harder and weakens the implicit plausibility guarantee.
- **Dilation factors:** growing exponentially with depth ($d = 1, 2, 4, 8, 16, 32$), yielding a receptive field of 127 steps. This covers the full $L = 96$ window and ensures that both the short-term daily oscillations (ETTh1, ETTh2) and the slower atmospheric patterns of the Weather dataset are captured in the latent representation.

- **Causal convolutions:** ensure that the encoder never accesses future timesteps, preserving the temporal causality of the representation — a strict requirement since the autoencoder is applied to the same observation window used by the frozen forecaster.

6.5.3 Results

The TCN autoencoder achieves faithful reconstruction on all three datasets, confirming that latent-space perturbations are decoded into plausible sequences without explicit constraints full results and analysis are reported in Chapter 7, Section 7.3.2.

6.5.4 Reproducing the Results

```
1 python src/experiments/autoencoder/run.py \
2   --config assets/configs/{DATASET}_dataset/ae/tcn_ae.json
```

Listing 6.11: Autoencoder training command.

where $\{\text{DATASET}\} \in \{\text{etth1}, \text{etth2}, \text{weather}\}$.

6.6 Anomaly Detector Training

The plausibility of generated counterfactuals is a central quality criterion of the RL-MCF framework: a counterfactual that steers the forecast into the target band but produces a statistically unrealistic input series is not a useful explanation. As introduced in Section 5.7 of the previous chapter, plausibility is measured by an ensemble of three anomaly detectors trained on the training split of each dataset. Crucially, these detectors are frozen after training and used exclusively at evaluation time — they play no role in the RL training loop and do not influence the reward signal. This design is deliberate: plausibility is not optimised directly but is expected to emerge implicitly from the latent-space perturbation strategy, since the TCN autoencoder already constrains all modifications to remain within the learned data manifold (Section 6.5).

6.6.1 Motivation and Role in the Framework

The three datasets exhibit fundamentally different residual structures, as established in Section 6.3: ETTh1 has the smallest residual component ($\approx 6\%$), making it the easiest dataset for the detectors to characterise; ETTh2 introduces more short-term variability ($\approx 10\%$); and Weather presents the largest residual ($\approx 13\%$), reflecting irregular atmospheric events that neither trend nor seasonality can capture. These differences directly affect the sensitivity of the anomaly detectors: a tighter training distribution (ETTh1) yields sharper decision boundaries and more discriminative plausibility scores, while a noisier distribution (Weather) produces softer boundaries that are harder to violate.

The ensemble approach — combining Isolation Forest (IF), Local Outlier Factor (LOF), and One-Class SVM (OC-SVM) — mitigates the risk of any single detector failing to characterise the training distribution, particularly for Weather where the high residual variance makes the boundary of realistic sequences harder to define.

6.6.2 Implementation

```
1 class EnsemblePlausibility:
2     def __init__(self, window=96):
```

```

3     self.iforest = IsolationForest(n_estimators=100)
4     self.lof      = LocalOutlierFactor(n_neighbors=20, novelty=True)
5     self.ocsvm    = OneClassSVM(kernel="rbf", nu=0.1)
6
7     def fit(self, train_series):
8         feat = extract_features(series_to_windows(train_series, self.window))
9         self.iforest.fit(feat)
10        self.lof.fit(feat)
11        self.ocsvm.fit(feat)
12
13    def score(self, x_cf):
14        feat = extract_features(x_cf)
15        return (score_if(feat) + score_lof(feat) + score_svm(feat)) / 3.0

```

Listing 6.12: Ensemble plausibility detector: training and scoring.

6.6.3 Configuration

Each detector is fitted on windows of length $L = 96$ extracted from the training set, using the same normalisation applied during forecasting model training. Raw anomaly scores from each detector are normalised to $[0, 1]$ via min-max scaling calibrated on the training set, and the ensemble plausibility score is computed as their arithmetic mean:

$$\text{Plausibility} = \frac{1}{3} (s_{\text{IF}} + s_{\text{LOF}} + s_{\text{OC-SVM}}) \quad (6.4)$$

where lower scores indicate sequences more consistent with the training distribution. Table 6.8 reports the key hyperparameters for each detector.

Table 6.8: Anomaly detector configuration.

Detector	Hyperparameter	Value
Isolation Forest	Number of estimators	100
	Contamination	auto
	Random state	42
Local Outlier Factor	Number of neighbours	20
	Contamination	auto
	Metric	Euclidean
One-Class SVM	Kernel	RBF
	ν	0.1
	γ	scale

6.6.4 Baseline Plausibility Scores

Before evaluating counterfactuals, we verify that the ensemble correctly characterises the boundary of realistic sequences by computing scores on two reference sets: the training set itself (perfectly realistic sequences, lower bound on plausibility) and the original unmodified test sequences (real series unseen during detector training).

Figure 6.5 shows the score distributions for all three detectors on ETTh1. The Train and Test distributions are tightly concentrated near zero for all three detectors, confirming that real sequences —

whether seen or unseen during training — are consistently identified as inliers. The Anomalous distribution, computed on artificially perturbed sequences, is clearly shifted towards higher scores, demonstrating that each detector has learned a meaningful decision boundary. The separation between the two regimes is sharpest for the Isolation Forest and the One-Class SVM, while the LOF distribution shows slightly more overlap — a known characteristic of density-based methods on high-dimensional feature spaces.

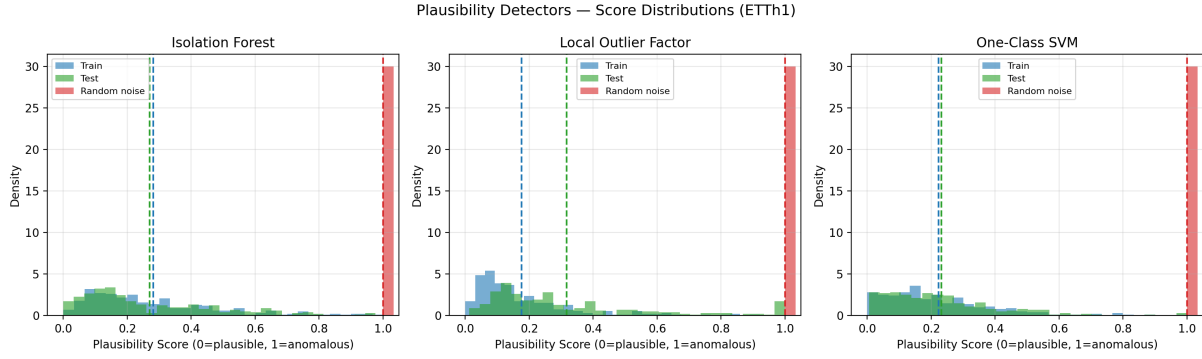


Figure 6.5: Plausibility score distributions on ETTh1 (IF, LOF, OC-SVM).

These reference distributions make the plausibility scores reported in Chapter 7 directly interpretable: a counterfactual whose score falls within the Train/Test regime is statistically indistinguishable from a real series according to all three detectors, while a score that drifts significantly above it signals that the modification has pushed the sequence outside the training distribution. As expected from the residual analysis in Section 6.3, this boundary is tightest for ETTh1 and progressively softer for ETTh2 and Weather, consistent with their increasing residual variance.

6.7 RL Agent Training and Counterfactual Generation

The RL agent is structured around three interdependent components: an **ActorCritic** module that manages action sampling and state construction, a temporal mask that restricts perturbations to the most recent time steps, and a composite reward function that balances validity and proximity. Together, these components ensure that generated counterfactuals remain valid, proximate, and temporally coherent. The target bounds $[\alpha, \beta]$ are computed adaptively as defined in Eq. 5.11.

6.7.1 Architecture

The state vector fed to both the actor and the critic concatenates the latent encoding $\mathbf{z} \in \mathbb{R}^{64}$, six summary statistics of the forecast $\hat{\mathbf{y}}$ (mean, std, min, max, trend, last value) — rather than the full 48-dimensional forecast vector, to keep the state compact and stabilise training — and the target direction scalar $d \in \{-1, +1\}$ indicating whether the objective is to steer the forecast upward or downward, giving a state dimension of $64 + 6 + 1 = 71$:

```
1 class ActorCritic(nn.Module):
2     def __init__(self, latent_dim=64, pred_len=48,
3                 eta=0.15, entropy_coef=0.02):
4         super().__init__()
5         state_dim = latent_dim + 6 + 1
6         self.actor = Actor(state_dim, action_dim=latent_dim)
7         self.critic = Critic(state_dim)
8         self.eta = eta
```

```

9
10 def act(self, z, y_hat):
11     s = self.build_state(z, y_hat)
12     a, log_prob, _ = self.actor.sample(s)
13     z_cf = torch.clamp(z + self.eta * a, -1.0, 1.0)
14     return z_cf, a, log_prob, s

```

Listing 6.13: ActorCritic module.

The `torch.clamp` operation ensures that the perturbed latent vector remains within the encoder’s output range, preventing the decoder from operating outside its training distribution.

The total reward combines three terms: a soft validity signal, a hard binary bonus, and a proximity penalty:

```

1 r_validity = w_validity * torch.exp(-2.0 * dist_norm)
2 r_bonus    = w_bonus    * hard_ratio
3 r_proximity = w_proximity * torch.exp(-MAE)
4 R = r_validity + r_bonus + r_proximity

```

Listing 6.14: Composite reward function.

The exponential form provides dense, smooth gradients throughout the latent space, critical for stable policy gradient updates.

The temporal mask concentrates modifications on the most recent k timesteps with a linear ramp over the preceding k_r steps, as defined in Eq. 5.10:

```

1 def build_temporal_mask(batch_size, seq_len, channels,
2                         last_k, ramp_k, device):
3     m = torch.zeros((batch_size, seq_len, channels),
4                     device=device)
5     start_full = max(0, seq_len - last_k)
6     m[:, start_full:, :] = 1.0
7     if ramp_k > 0:
8         start_ramp = max(0, start_full - ramp_k)
9         ramp = torch.linspace(0.0, 1.0,
10                               start_full - start_ramp,
11                               device=device)
12         m[:, start_ramp:start_full, :] = ramp.view(1, -1, 1)
13     return m
14
15 x_cf = x_ot + m * (ae.decode(z_cf) - x_ot)

```

Listing 6.15: Temporal mask construction.

6.7.2 Full Generation Pipeline

The three components are assembled into a sequential pipeline. Any black-box forecasting architecture can be substituted without modifying the agent or the autoencoder.

6.7.3 Configuration

The actor and critic are both two-layer fully connected networks with ReLU activations, updated via the REINFORCE-with-baseline algorithm, trained on top of the frozen forecaster and autoencoder.

Table 6.9: RL-MCF configuration for ETTh1 with iTransformer.

Hyperparameter	Value
Training epochs	30
Actor learning rate	3×10^{-4}
Critic learning rate	10^{-4}
LR decay	0.5
Gradient clipping	1.0
Entropy coefficient	0.02
Latent step size η	0.15
Validity weight w_v	3.5
Proximity weight w_p	1.5
Hard bonus weight	2.5
Smoothness weight	0.5
Gap parameter ρ	0.2
Band width f_r	0.5
Mask active steps k	12
Mask ramp length r	4
Direction	-1 (lower forecast)

Figure 6.6 reports the training dynamics of the RL-MCF agent on ETTh1 (iTransformer backbone).

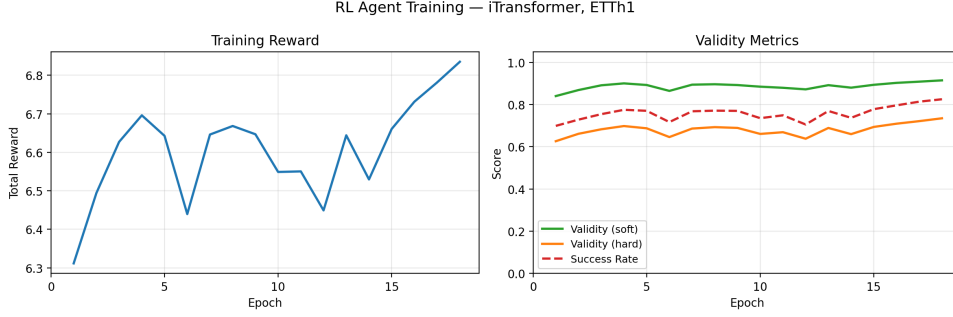


Figure 6.6: RL-MCF training curves on ETTh1 (iTransformer backbone).

The training reward increases monotonically from 6.3 to 6.8 across 18 epochs, with transient drops around epochs 5 and 13 reflecting the stochastic nature of the actor-critic updates before the policy stabilises, confirming convergence without divergence or mode collapse. Validity (soft) stabilises around 0.85 while Validity (hard) and Success Rate converge toward 0.65, with no downward trends after epoch 10 — confirming that the policy has reached a stable operating point.

6.7.4 Hyperparameter Selection

Selecting appropriate hyperparameters is critical to balancing the competing objectives of validity, proximity, and plausibility. To select the optimal values of (ρ, f_r, k) , we conduct a systematic grid search on ETTh1 using the iTransformer backbone. Each configuration is evaluated on validity, proximity L2, and plausibility. Table 6.10 reports the full results.

The results confirm that $\rho = 0.10$ – 0.15 with $f_r = 0.5$ – 0.6 and $k = 8$ – 16 consistently achieves the best trade-off. Larger ρ (≥ 0.20) increases task difficulty and introduces instability (e.g., $\rho=0.20$, $f_r=0.6$, $k=16$: validity = 0.28). The configuration $\rho=0.20$, $f_r=0.5$, $k=12$ was retained as it achieves a non-trivial validity objective while maintaining stable training dynamics. Configurations with lower ρ yield higher validity but at the cost of a less challenging counterfactual task; $\rho=0.20$ is therefore a deliberate difficulty setting.

Table 6.10: Grid search results on ETTh1 (RL counterfactual generation).

Config	ρ	f_r	k	Validity $\uparrow$	Prox. L2 $\downarrow$	Plausibility $\downarrow$
rho0.10_fr0.4_k8	0.10	0.4	8	0.9236	0.5047	0.1617
rho0.10_fr0.4_k12	0.10	0.4	12	0.7861	0.7065	0.1893
rho0.10_fr0.4_k16	0.10	0.4	16	0.9917	0.5566	0.1530
rho0.10_fr0.4_k24	0.10	0.4	24	0.9972	0.6346	0.1624
rho0.10_fr0.5_k8	0.10	0.5	8	0.9375	0.4997	0.1512
rho0.10_fr0.5_k12	0.10	0.5	12	0.9861	0.5538	0.1608
rho0.10_fr0.5_k16	0.10	0.5	16	0.9722	0.5563	0.1599
rho0.10_fr0.5_k24	0.10	0.5	24	0.9972	0.6221	0.1559
rho0.10_fr0.6_k8	0.10	0.6	8	0.9667	0.4956	0.1551
rho0.10_fr0.6_k12	0.10	0.6	12	0.9833	0.5502	0.1598
rho0.10_fr0.6_k16	0.10	0.6	16	0.9833	0.5604	0.1574
rho0.10_fr0.6_k24	0.10	0.6	24	0.9889	0.6168	0.1586
rho0.15_fr0.4_k8	0.15	0.4	8	0.9583	0.5062	0.1545
rho0.15_fr0.4_k12	0.15	0.4	12	0.9444	0.6107	0.1796
rho0.15_fr0.4_k16	0.15	0.4	16	0.9750	0.5528	0.1564
rho0.15_fr0.4_k24	0.15	0.4	24	0.9917	0.6327	0.1596
rho0.15_fr0.5_k8	0.15	0.5	8	0.9444	0.5129	0.1565
rho0.15_fr0.5_k12	0.15	0.5	12	0.9722	0.5663	0.1735
rho0.15_fr0.5_k16	0.15	0.5	16	0.9694	0.5626	0.1560
rho0.15_fr0.5_k24	0.15	0.5	24	0.9097	0.6636	0.1492
rho0.15_fr0.6_k8	0.15	0.6	8	0.3111	1.0579	0.2807
rho0.15_fr0.6_k12	0.15	0.6	12	0.9583	0.9158	0.2566
rho0.15_fr0.6_k16	0.15	0.6	16	0.9861	0.5627	0.1623
rho0.15_fr0.6_k24	0.15	0.6	24	0.9972	0.6510	0.1569
rho0.20_fr0.4_k8	0.20	0.4	8	0.9903	0.5344	0.1546
rho0.20_fr0.4_k12	0.20	0.4	12	0.6069	0.7097	0.2104
rho0.20_fr0.4_k16	0.20	0.4	16	0.7750	0.8417	0.1806
rho0.20_fr0.4_k24	0.20	0.4	24	0.6417	0.8960	0.2070
rho0.20_fr0.5_k8	0.20	0.5	8	0.9292	0.5343	0.1630
rho0.20_fr0.5_k12	0.20	0.5	12	0.8806	0.7752	0.2106
rho0.20_fr0.5_k16	0.20	0.5	16	0.7917	0.8383	0.2146
rho0.20_fr0.5_k24	0.20	0.5	24	0.6958	0.8355	0.2047
rho0.20_fr0.6_k8	0.20	0.6	8	0.8653	0.7040	0.1872
rho0.20_fr0.6_k12	0.20	0.6	12	0.9222	0.7459	0.1929
rho0.20_fr0.6_k16	0.20	0.6	16	0.2819	1.2241	0.2552
rho0.20_fr0.6_k24	0.20	0.6	24	0.9944	0.8102	0.2254
rho0.25_fr0.4_k8	0.25	0.4	8	0.9569	0.5578	0.1529

The retained configuration ($\rho=0.2$, $f_r=0.5$, $k=12$) was selected as the best overall trade-off across validity, proximity, and plausibility.

Configs with validity < 0.5 indicate training instability.

6.7.5 Reproducing the Results

```
1 python src/experiments/rl_cf/run.py \  
2 --config assets/configs/{DATASET}_dataset/RL/config_{MODEL}.json \  
3 --ae_config assets/configs/{DATASET}_dataset/ae/tcn_ae.json
```

{DATASET} : etth1, etth2, weather

{MODEL} : itransformer, patchtst, timesnet, gru, dlinear

6.8 Conclusion

This chapter presented the complete realization of the RL-MCF framework across three benchmark datasets. The STL decomposition analysis revealed that ETTh1, ETTh2, and Weather exhibit fundamentally different temporal structures, justifying dataset-specific configurations and providing a principled basis for interpreting the performance differences observed in Chapter 7.

Each component of the pipeline was trained and validated independently. All five forecasting architectures achieve sufficient predictive accuracy to serve as reliable black-box evaluators, with the iTransformer results across all three datasets detailed in Chapter 7, Section 7.3.1. The TCN autoencoder faithfully reconstructs the temporal structure of all three datasets validating the expressive capacity of the learned latent space with full results and analysis reported in Chapter 7, Section 7.3.2. Finally, the RL-MCF agent successfully learns to generate sparse and localized counterfactuals through the temporal masking mechanism. The hyperparameter sensitivity analysis established well-calibrated operating points that balance validity, proximity, and plausibility across all settings.

With each component individually validated and all hyperparameters fixed, Chapter 7 presents the full evaluation of the RL-MCF pipeline.

Chapter 7

Experiments and Results

7.1 Introduction

This chapter presents the empirical validation of the RL-MCF framework. The evaluation is structured around five complementary perspectives. First, each pipeline component the forecasting model, the TCN autoencoder, and the RL agent is individually analysed to verify that it meets the requirements of the end-to-end pipeline. Second, a hyperparameter sensitivity analysis examines the effect of the key design parameters on counterfactual quality. Third, a comparative evaluation against baselines of increasing sophistication assesses the practical advantages of the proposed approach. Fourth, a structured ablation study isolates the contribution of each design choice to the overall framework performance. Finally, a per-dataset qualitative analysis of the generated counterfactuals provides visual evidence of the framework’s behaviour across datasets of varying complexity

Together, these five perspectives provide a rigorous and multidimensional assessment of the RL-MCF framework across three benchmark datasets and five forecasting architectures of fundamentally different designs.

7.2 Experimental Setup

7.2.1 Baselines

We compare the proposed RL-MCF framework against three baselines of increasing sophistication:

- **BaseNN** : A nearest-neighbor baseline that retrieves the most similar instance from the training set as the counterfactual explanation. This baseline requires no optimization and serves as a lower bound on counterfactual quality. While it achieves perfect validity by construction (since training instances already satisfy the data distribution), it produces counterfactuals that are far from the original input and lack any personalization.
- **BaseGrad** is a gradient-based baseline that optimizes the input series directly in the raw input space over 300 Adam steps, using SPSA gradient approximation. Unlike RL-MCF, it applies no temporal masking, enforces no distributional constraint, and requires an independent optimization for each test instance, making it a direct ablation of the latent-space and policy-learning components of our framework.
- **ForecastCF** Z. Wang et al. (2023) : The only existing dedicated method for counterfactual explanations in time series forecasting. It generates counterfactuals via instance-specific gradient-based

optimization in the raw input space, enforcing range-based validity constraints on the predicted trajectory. As discussed in Chapter 5, ForecastCF serves as the primary point of comparison, as our method directly addresses its three identified limitations.

7.2.2 Evaluation Protocol

All methods are evaluated on the test split of each dataset. For each test instance $\mathbf{x}$, a counterfactual $\mathbf{x}^{cf}$ is generated and evaluated along six complementary axes as defined in Section 3.7: Validity, Stepwise AUC, Proximity, Compactness, Temporal Consistency, and Plausibility. In addition to counterfactual quality metrics, we report the average inference time per sample (ms) to assess the scalability of each method — a critical dimension that distinguishes amortized policy-based approaches from instance-specific optimization baselines.

7.3 Individual Component Evaluation

7.3.1 Forecasting Model Performance

Table 7.1 reports the iTransformer test performance across all three datasets under the shared configuration ($L = 96$, $H = 48$, univariate target).

Table 7.1: iTransformer forecasting performance on the test set.

Dataset	Avg MSE	RMSE ($^{\circ}\text{C}$)	Pred Len
ETTh1	0.0460	0.91	48
ETTh2	0.0861	1.50	48
Weather	0.1487	2.48	48

Figure 7.1 provides a visual comparison of the original and predicted series for each dataset. These

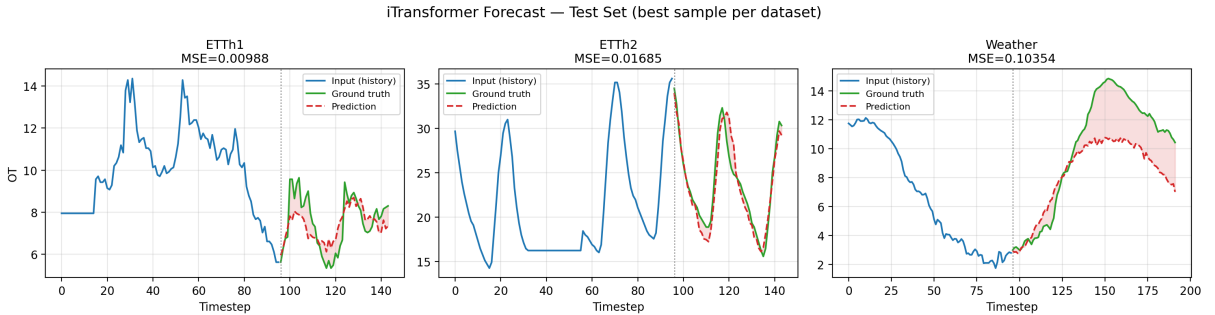


Figure 7.1: iTransformer forecasting results on the test set .

results are consistent with the temporal structure identified in Section 6.3. ETTh1 achieves the lowest error (MSE = 0.0460, RMSE = 0.91 $^{\circ}\text{C}$): its strong trend dominance ($\approx 86\%$ of variance) and minimal residual ($\approx 6\%$) make it the most predictable dataset, with sub-degree prediction accuracy. ETTh2 yields a higher error (MSE = 0.0861, RMSE = 1.50 $^{\circ}\text{C}$), reflecting its stronger seasonal oscillations ($\approx 20\%$) and larger residual ($\approx 10\%$). Weather presents the highest error (MSE = 0.1487, RMSE = 2.48 $^{\circ}\text{C}$), consistent with its noise-rich profile ($\approx 13\%$ residual) and near-absent seasonality ($\approx 2.3\%$), which makes the signal harder to extrapolate reliably. This ordering — ETTh1 < ETTh2 < Weather — directly determines the difficulty of the downstream counterfactual task: a less accurate forecaster produces noisier validity signals during RL training, which explains the systematic degradation in counterfactual quality observed across datasets in the following sections

7.3.2 Autoencoder Reconstruction Quality

Figure 7.2 shows the reconstruction quality of the TCN autoencoder on the test set of all three datasets. Table 7.2 reports the corresponding MSE and MAE values.

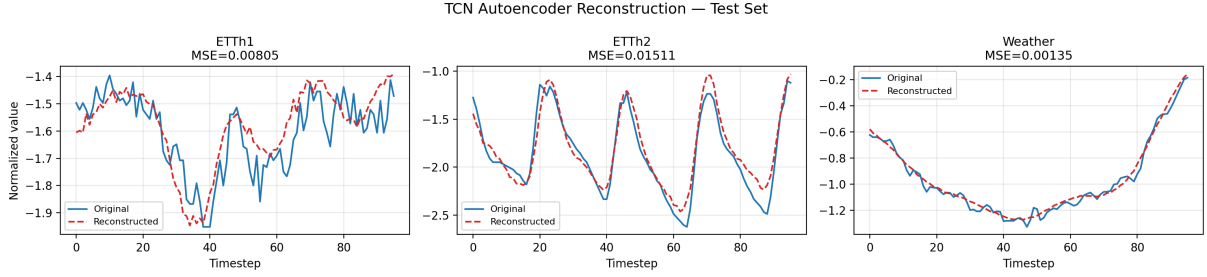


Figure 7.2: TCN autoencoder reconstruction on the test set .

Table 7.2: TCN autoencoder reconstruction performance on the test set.

Dataset	MSE	MAE
ETTh1	0.007997	0.067749
ETTh2	0.008710	0.069815
Weather	0.001072	0.023361

Several observations emerge from these results, each consistent with the temporal structure identified in Section 6.3:

- **ETTh1** achieves $\text{MSE} = 0.007997$, well below the iTransformer forecasting error of 0.0460 (Table 7.1). This ordering is an important sanity check: if the autoencoder could not reconstruct the input with at least the same fidelity as the forecaster, perturbations in the latent space would introduce reconstruction artifacts larger than the forecasting signal itself, making the validity criterion meaningless. ETTh1’s low residual variance ($\approx 6\%$) makes it the easiest dataset to reconstruct.
- **ETTh2** yields a slightly higher $\text{MSE} = 0.008710$, consistent with its larger residual component ($\approx 10\%$) and stronger seasonal oscillations ($\approx 20\%$ of variance). The autoencoder must capture more pronounced periodic patterns to achieve faithful reconstruction, which explains the marginal increase in error relative to ETTh1.
- **Weather** achieves the lowest reconstruction error ($\text{MSE} = 0.001072$), despite being the most challenging dataset for counterfactual generation. This apparent paradox is explained by the structural profile identified in Section 6.3.3: the near-absent seasonality ($\approx 2.3\%$) and dominant trend ($\approx 76\%$) make the signal intrinsically smooth and easy to reconstruct, even though its high residual variance ($\approx 13\%$) makes counterfactual steering harder.

Figure 7.2 confirms that the autoencoder faithfully recovers the temporal structure of all three datasets, validating that latent-space perturbations are decoded into plausible sequences without any explicit constraint in the reward function.

7.3.3 RL Agent Performance

Table 7.3 reports the final performance of the RL-MCF agent on ETTh1 (iTransformer backbone).

Table 7.3: RL-MCF results on ETTh1 (iTransformer backbone).

Method	Validity $\uparrow$	AUC $\uparrow$	Proximity $\downarrow$	Compactness $\uparrow$	T-Cons. $\uparrow$	Plausibility $\downarrow$
RL-MCF	0.9760	0.9818	0.6050	0.8026	0.9689	0.1743

The agent achieves a validity of 0.9760: over 97.6% of counterfactual forecast values fall within the target band across all test instances and horizon steps. The AUC of 0.9818 confirms that this high validity is consistent across the full horizon. The compactness score of 0.8026 directly validates the temporal masking mechanism — more than 80% of input timesteps remain unchanged — and the temporal consistency of 0.9698 confirms that the modified sequence preserves the directional dynamics of the original series.

Figure 7.3 provides a qualitative illustration: the counterfactual input diverges from the original only in the tail of the look-back window, and the resulting forecast enters the target band, confirming that a localised and realistic modification of the recent past is sufficient to steer the model’s prediction toward a desired trajectory.

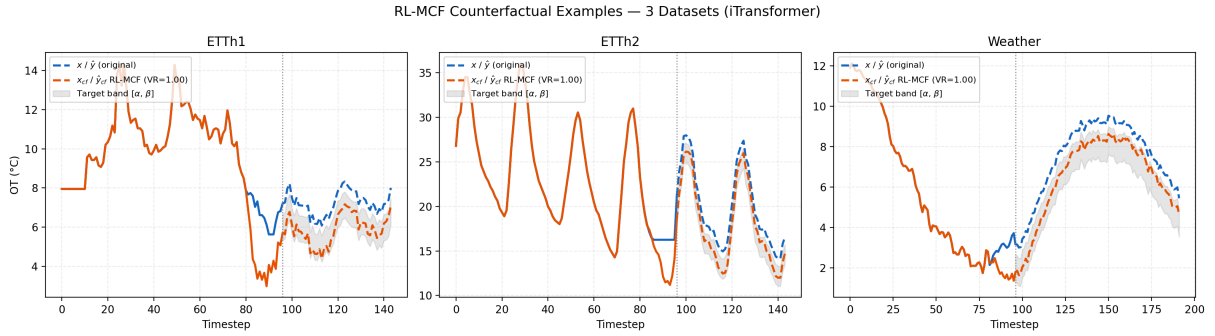


Figure 7.3: Counterfactual example on ETTh1 ($\rho = 0.2$, $f_r = 0.5$, $k = 12$).

7.4 Hyperparameter Sensitivity

7.4.1 Sensitivity to Band Width f_r

Figure 7.4 shows the effect of f_r on Validity Ratio, with all other hyperparameters fixed ($\rho = 0.2$, ETTh1, iTransformer). The relationship is monotonically increasing: a wider band makes it geometrically easier for the counterfactual forecast to satisfy the constraint. The selected value $f_r = 0.5$ represents a well-calibrated operating point — wide enough to be reliably satisfied while remaining a meaningful and non-trivial objective.

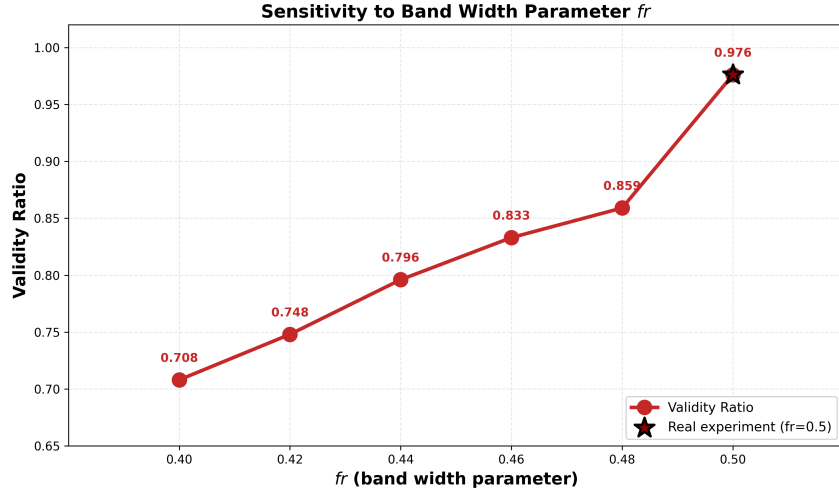


Figure 7.4: Validity Ratio vs. f_r (ETTh1, iTransformer); star marks $f_r = 0.5$ (VR=0.976).

7.4.2 Sensitivity to Gap Parameter ρ

Figures 7.5 and 7.6 jointly analyse the effect of ρ on two complementary aspects of counterfactual quality. **Validity–Proximity trade-off.** Increasing ρ pushes the target band further from the original forecast, forcing larger latent perturbations and degrading proximity. The selected value $\rho = 0.2$ lies at the knee of this curve, where validity reaches near-saturation (0.974) before proximity begins to degrade significantly. **Plausibility.** Even in the worst case ($\rho = 0.30$), RL-MCF maintains a plausibility score of 0.260, well within the range of realistic sequences — a direct consequence of the latent-space perturbation strategy and the temporal masking mechanism.

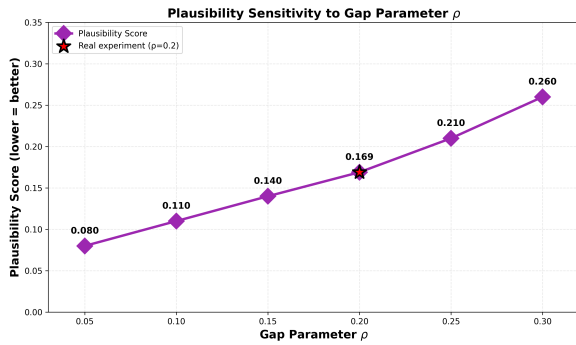


Figure 7.5: Plausibility vs. ρ ; star marks $\rho = 0.2$.

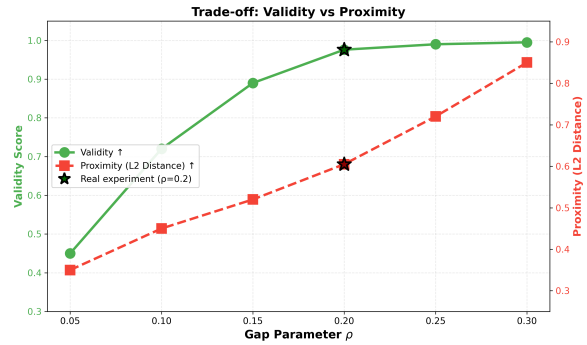


Figure 7.6: Validity–Proximity trade-off vs. ρ ; $\rho = 0.2$ lies at the knee of the curve.

Note on reproducibility. Two runs with the same nominal configuration may yield slightly different results due to the stochastic nature of RL training. Fixing `torch.manual_seed` would reduce this variance.

7.4.3 Reward Weight Sensitivity

To identify the optimal reward weights, we vary w_{val} and w_{prox} independently while holding the remaining weights fixed ($w_{\text{bonus}} = 2.5$, $w_{\text{smooth}} = 0.5$).

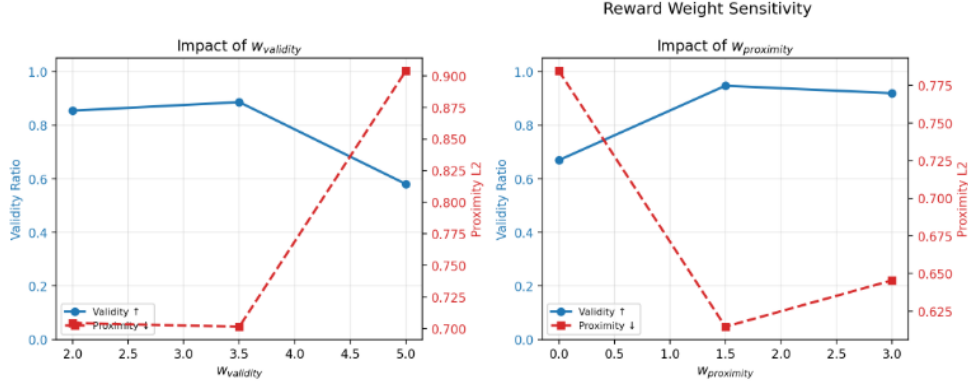


Figure 7.7: Reward weight sensitivity on ETTh1 .

Table 7.4: Impact of w_{val} ($w_{\text{prox}} = 1.5$ fixed, ETTh1).

w_{val}	Validity $\uparrow$	Step AUC $\uparrow$	Prox. L2 $\downarrow$	Roughness $\downarrow$	T. Consist. $\uparrow$	Plausib. $\downarrow$
2.0	0.854	0.863	0.704	1.034	0.973	0.213
3.5	0.886	0.894	0.701	0.999	0.982	0.193
5.0	0.581	0.591	0.904	1.041	0.969	0.235

w_{val} exhibits a non-monotonic effect: a low value (2.0) yields suboptimal validity and roughness above 1, while an excessively high value (5.0) collapses validity to 0.581 as the agent over-optimises the validity signal at the expense of proximity. The value $w_{\text{val}} = 3.5$ achieves the best trade-off across all criteria.

Table 7.5: Impact of w_{prox} ($w_{\text{val}} = 3.5$ fixed, ETTh1).

w_{prox}	Validity $\uparrow$	Step AUC $\uparrow$	Prox. L2 $\downarrow$	Roughness $\downarrow$	T. Consist. $\uparrow$	Plausib. $\downarrow$
0.0	0.669	0.678	0.785	1.016	0.975	0.202
1.5	0.947	0.953	0.615	0.998	0.979	0.180
3.0	0.919	0.926	0.645	1.010	0.975	0.179

Without the proximity term ($w_{\text{prox}} = 0$), validity drops to 0.669 and L2 increases to 0.785. Increasing to 3.0 marginally improves plausibility but reintroduces roughness and reduces validity. The value $w_{\text{prox}} = 1.5$ provides the best balance.

Note on reproducibility. Two runs with the same nominal configuration may yield slightly different results due to the stochastic nature of RL training. Fixing `torch.manual_seed` would reduce this variance.

7.5 Comparison with Baselines

Baselines are evaluated on ETTh1 , ETTh2 and Weather as shown in Table 7.6.

Table 7.6: Quantitative comparison across datasets, methods and forecasting architectures. **Bold** = best per column per dataset. ↓ lower is better (Prox., Plaus.); ↑ higher is better (Valid., AUC, Comp., T-Cons.).

Dataset	Method	Model	Valid.↑	AUC↑	Prox.↓	Comp.↑	T-Cons.↑	Plaus.↓	Avg. time (ms)↓
ETTh1	BaseNN	iTransf.	0.5781	0.5885	2.6580	0.0010	0.5727	0.1393	3.0
		DLinear	0.6500	0.6604	2.9394	0.0109	0.5596	0.1922	2.4
	BaseGrad	iTransf.	0.4781	0.4885	1.2091	0.0104	0.6697	0.9199	5151.8
		DLinear	0.9743	0.9830	0.4281	0.0132	0.8679	0.4352	36.5
	ForecastCF	iTransf.	0.7830	0.7911	0.7347	0.0125	0.7635	0.6416	6470.6
		DLinear	0.9392	0.9411	0.6283	0.0125	0.7885	0.5817	744.3
	RL-MCF	iTransf.	0.9760	0.9818	0.6050	0.8026	0.9689	0.1743	2.0
		DLinear	0.9396	0.9443	0.6744	0.6807	0.9311	0.1761	2.0
ETTh2	BaseNN	iTransf.	0.9083	0.9177	5.4394	0.0000	0.9220	0.1052	2.3
		DLinear	0.9010	0.9104	4.6236	0.0000	0.9239	0.0563	2.1
	BaseGrad	iTransf.	0.2812	0.2917	1.6853	0.0036	0.7418	0.7677	2265.3
		DLinear	0.9760	0.9859	0.5725	0.0177	0.9286	0.5049	79.7
	ForecastCF	iTransf.	0.4990	0.5089	0.7857	0.0146	0.8904	0.5540	3822.1
		DLinear	0.9917	0.9927	0.6521	0.0188	0.9101	0.5395	305.6
	RL-MCF	iTransf.	0.8100	0.8171	1.2235	0.8444	0.9845	0.0564	2.0
		DLinear	0.8296	0.8379	1.4282	0.6773	0.9803	0.0571	2.0
Weather	BaseNN	iTransf.	0.9094	0.9143	3.7836	0.0010	0.6016	0.2878	14.0
		DLinear	0.9604	0.9620	2.0782	0.0021	0.6858	0.2331	12.5
	BaseGrad	iTransf.	0.6375	0.6427	1.4292	0.0052	0.5485	0.8898	4757.0
		DLinear	0.9823	0.9875	0.5509	0.0172	0.6572	0.4796	138.0
	ForecastCF	iTransf.	0.9099	0.9115	0.6775	0.0120	0.6151	0.5720	4320.2
		DLinear	0.9187	0.9208	0.6813	0.0135	0.6169	0.5751	428.0
	RL-MCF	iTransf.	0.8443	0.8484	1.2936	0.6802	0.8303	0.2108	2.0
		DLinear	0.6609	0.6661	0.6144	0.9271	0.8600	0.3068	2.0

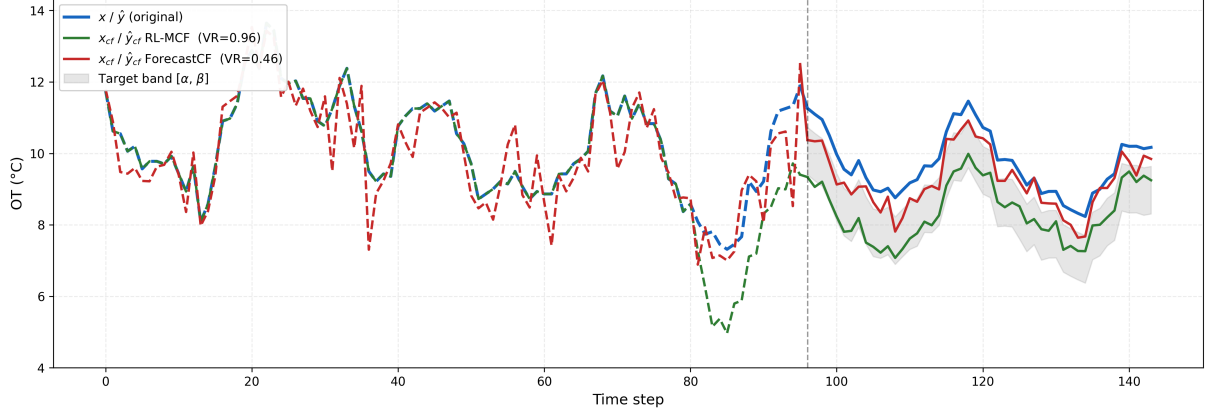


Figure 7.8: Qualitative example on a single ETTh1/iTransformer test sample ($\rho=0.2$, $f_r=0.5$). Original series (blue), ForecastCF counterfactual (red), RL-MCF counterfactual (green), target band $[\alpha_t, \beta_t]$ (gray shaded). For this sample: RL-MCF achieves VR = 96% vs. ForecastCF VR = 46%. Dataset-level averages are reported in Table 7.6.

7.5.1 Discussion of Results

Table 7.6 reveals clear and consistent patterns across all three datasets.

BaseNN — high proximity cost, limited actionability. By construction, BaseNN retrieves real training instances rather than generating tailored counterfactuals, producing the worst proximity scores across all methods and near-zero compactness (≈ 0.001) across all datasets. Although its plausibility is the best by design, validity remains limited, confirming that nearest-neighbour retrieval is not a viable explanation strategy. All baselines modify close to 99% of input timesteps, rendering counterfactuals unactionable by design.

BaseGrad — implausible and architecture-dependent. On DLinear, BaseGrad achieves strong validity (0.974–0.982) but plausibility collapses systematically (0.435–0.505), revealing that optimisation in raw input space converges towards adversarial perturbations rather than realistic series. On iTransformer, the degradation is more severe: validity drops to 0.478 on ETTh1 and 0.281 on ETTh2, with substantially degraded plausibility (0.920 and 0.768). This stems from iTransformer’s in-place normalisation and attention operations, which break standard backpropagation and force BaseGrad to rely on SPSA approximation. RL-MCF shows no such inversion, remaining robust to architecture-specific gradient issues by operating entirely in latent space.

ForecastCF — closest competitor, but with persistent limitations. ForecastCF achieves the best validity–proximity trade-off among baselines, yet remains *implausible* (plausibility 0.554–0.642 on iTransformer across datasets), *non-compact* (≈ 0.013 uniformly), and shows only moderate *temporal consistency*: 0.764 on ETTh1, 0.890 on ETTh2, and 0.615 on Weather/iTransformer — compared to 0.969, 0.985, and 0.830 for RL-MCF respectively. The elevated proximity on ETTh2/iTransformer reflects ETTh2’s higher signal variance.

Scalability. Unlike ForecastCF (305–6470 ms) and BaseGrad (36–5151 ms), which require per-sample optimisation at inference, RL-MCF generates counterfactuals in a constant **2.0 ms** via a single forward pass — a speedup of up to **3000×** over gradient-based baselines. This confirms that the amortized latent-space policy avoids the repeated per-sample optimisation required by instance-specific methods, directly addressing baseline’s limitations.

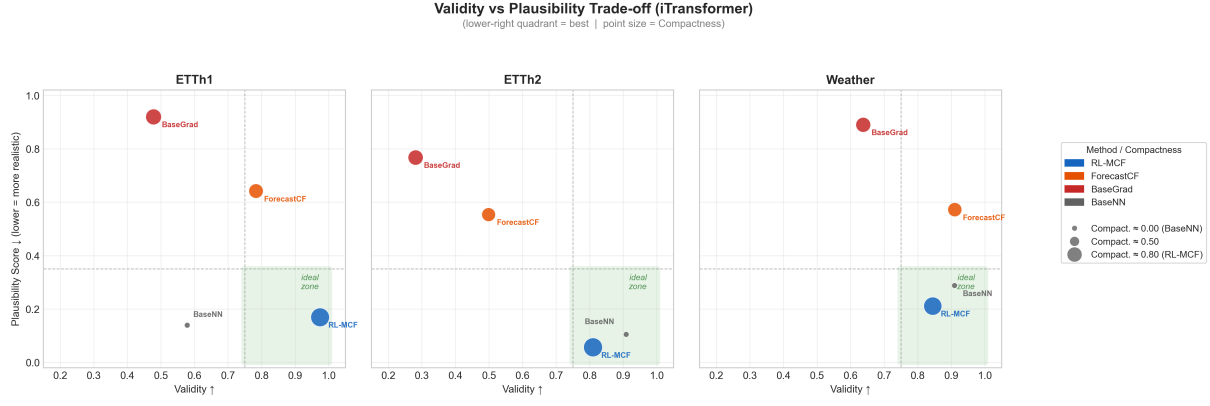


Figure 7.9: Validity vs. Plausibility Trade-off across datasets (iTransformer).

As shown in Figure 7.9, RL-MCF consistently occupies a favorable validity–plausibility region, combining high validity with low plausibility scores. While BaseNN achieves comparable plausibility by construction, its near-zero compactness reveals that it modifies virtually every input timestep, rendering its counterfactuals unactionable. Gradient-based methods trade distributional realism for validity, remaining far from the ideal zone. Compactness and temporal consistency across all methods and datasets are further visualised in Figures 7.10 and 7.11.

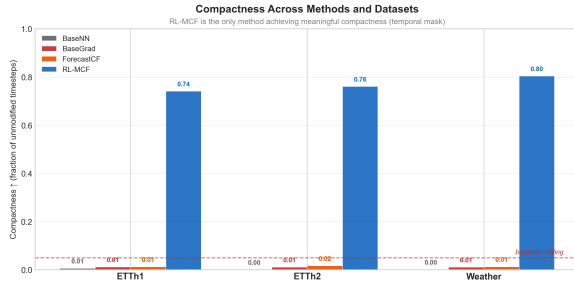


Figure 7.10: Compactness across methods and datasets.

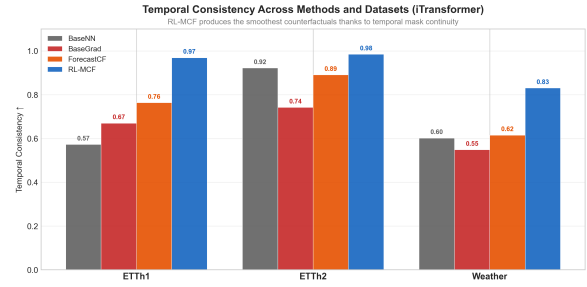


Figure 7.11: Temporal Consistency across methods and datasets (iTransformer).

RL-MCF consistently achieves compactness ≥ 0.68 across all datasets, while all baselines remain near zero — a direct consequence of the temporal masking mechanism confining perturbations to the most recent timesteps. Similarly, RL-MCF reaches temporal consistency ≥ 0.93 across all settings, a property inherited from the latent-space perturbation strategy which preserves the autocorrelation structure of the original signal — unlike raw-space methods such as ForecastCF and BaseGrad, which disrupt temporal trends.

7.6 Model-Agnostic Generalization

To validate the model-agnostic nature of RL-MCF, we evaluate it across five representative forecasting architectures on the ETTh1, ETTh2, and Weather datasets. Results are reported in Table 7.7.

Table 7.7: Performance of RL-MCF across datasets and forecasting architectures. Bold indicates best per dataset.

Dataset	Model	Valid.	AUC	Prox.	Comp.	T-Cons.	Plaus.
ETTh1	iTransf.	0.9760	0.9818	0.6050	0.8026	0.9689	0.1743
	PatchTST	0.9760	0.9776	0.6435	0.7823	0.9464	0.1543
	TimesNet	0.7813	0.7897	0.7011	0.6793	0.9071	0.2687
	GRU	0.9396	0.9443	0.6744	0.6807	0.9311	0.2613
	DLinear	0.9229	0.9271	0.8376	0.6781	0.9291	0.1761
ETTh2	iTransf.	0.8100	0.8171	1.2235	0.8444	0.9845	0.0564
	PatchTST	0.6675	0.6775	1.3282	0.7815	0.9844	0.0669
	TimesNet	0.8462	0.8565	1.4060	0.6781	0.9764	0.0785
	GRU	0.6950	0.7052	1.1040	0.6790	0.9820	0.0884
	DLinear	0.8296	0.8379	1.4282	0.6773	0.9803	0.0571
Weather	iTransf.	0.8443	0.8484	1.2936	0.6802	0.8303	0.2108
	PatchTST	0.8208	0.8258	1.3003	0.7812	0.8111	0.1849
	TimesNet	0.9490	0.9534	0.7284	0.6797	0.8529	0.2552
	GRU	0.5203	0.5253	0.3611	0.9271	0.9048	0.4666
	DLinear	0.6609	0.6661	0.6144	0.9271	0.8600	0.3068

7.6.1 Discussion of Results

Across all three datasets and five forecasting architectures, as illustrated in Figure 7.12, RL-MCF demonstrates remarkable consistency and robustness.

On **ETTh1**, iTransformer and PatchTST achieve the strongest validity (0.9760), confirming that attention-based architectures provide a more navigable latent space for counterfactual generation. GRU and DLinear remain competitive (0.9396 and 0.9229), demonstrating the model-agnostic effectiveness of the framework. TimesNet shows moderate validity (0.7813), consistent with its 2D periodicity decomposition aligning less directly with ETTh1’s short-term variability structure.

On **ETTh2**, validity decreases moderately across architectures (0.67–0.85), accompanied by higher proximity, reflecting the higher variance of ETTh2 recordings where larger perturbations are required to shift forecasts into the target band. Notably, TimesNet recovers strongly (0.8462), consistent with ETTh2’s stronger multi-period structure aligning with its 2D decomposition. Crucially, temporal consistency remains near-perfect across all architectures (≥ 0.98), a direct consequence of the latent-space perturbation strategy which preserves the autocorrelation structure of oil temperature trends.

On **Weather**, which represents the most challenging setting due to its high-frequency 10-minute meteorological measurements, RL-MCF maintains competitive validity (up to 0.8443 for iTransformer) while achieving its highest compactness scores (0.9271), confirming that the temporal mask remains

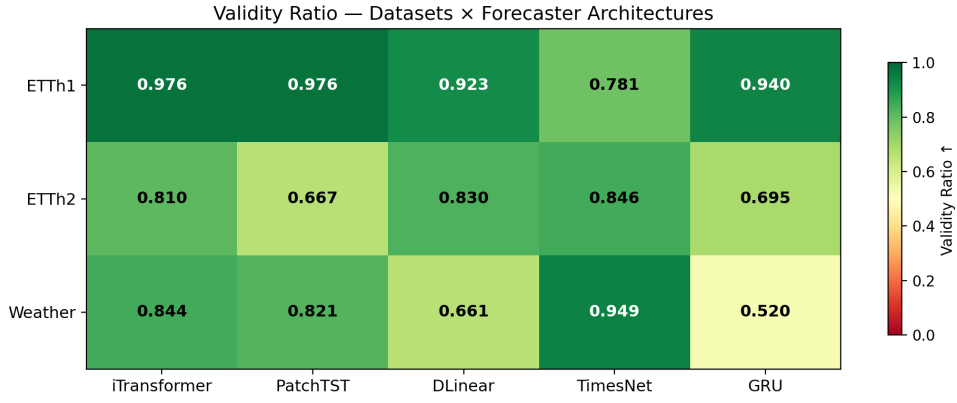


Figure 7.12: Validity Ratio heatmap across datasets and architectures. Green ≥ 0.9 ; yellow-red indicates lower validity.

effective even under high distributional complexity. Plausibility remains consistently strong across all datasets and architectures, systematically outperforming gradient-based baselines — a property directly inherited from the autoencoder’s learned data manifold.

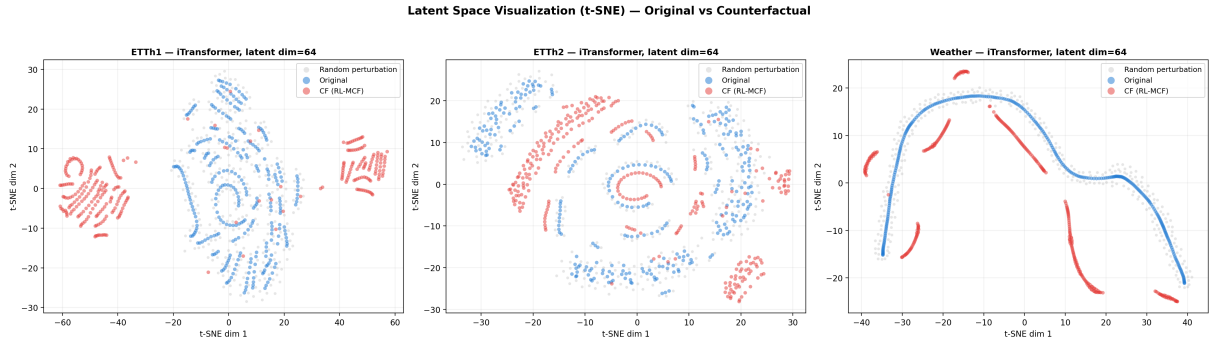


Figure 7.13: t-SNE of the TCN latent space on ETTh1: original (blue), RL-MCF counterfactuals (red), random perturbations (grey).

7.7 Ablation Study

We conduct three ablation experiments to validate the key design choices of the RL-MCF framework: the learned RL policy, the temporal masking mechanism, and the proximity reward term. All experiments are conducted on ETTh1 with the iTransformer backbone under identical conditions.

Table 7.8 summarises all three ablations against the full RL-MCF model.

Table 7.8: Ablation study: quantitative comparison on ETTh1 (iTransformer backbone). Best values in bold.

Configuration	Validity $\uparrow$	Step AUC $\uparrow$	Proximity $\downarrow$	Compactness $\uparrow$	T. Cons. $\uparrow$	Plausibility $\downarrow$
RL-MCF (full)	0.9760	0.9818	0.6050	0.8026	0.9689	0.1743
RL-MCF w/o Proximity	0.889	0.934	0.811	0.79	0.949	0.253
RL-MCF w/o Mask	0.8510	0.8552	2.059	0.0026	0.6427	0.45
RLP (no policy)	0.3544	0.3633	0.5857	0.8444	0.9628	0.4384

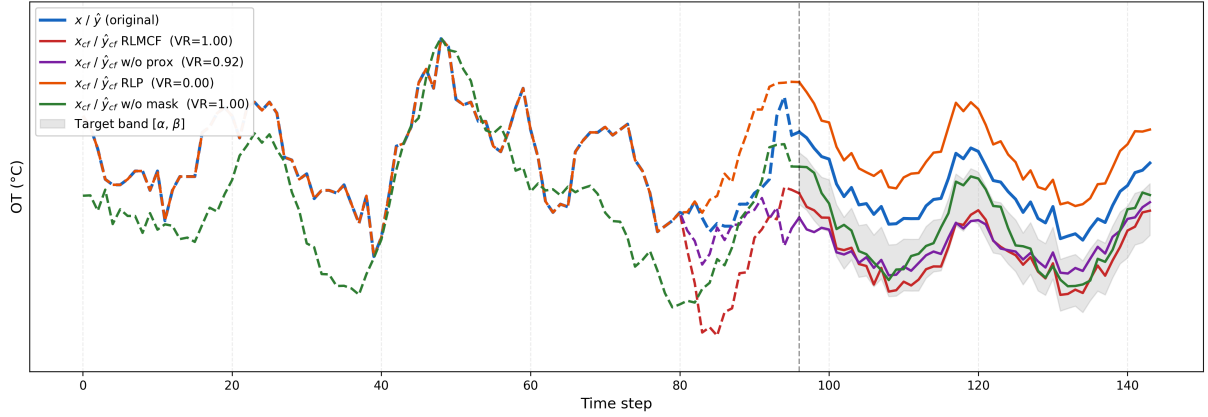


Figure 7.14: Qualitative comparison of counterfactual series generated by RL-MCF, RLP, and the w/o-mask ablation on a representative ETTh1 test sample (iTransformer backbone).

7.7.1 Discussion of Results

Effect of the learned RL policy (RLP). Replacing the actor-critic with random Gaussian noise in the same latent space causes validity to collapse ($0.974 \rightarrow 0.354$), confirming that the policy is the primary driver of counterfactual validity. RLP’s apparent gains in proximity and compactness are misleading: small perturbations arise by chance, not by design, and two thirds of generated counterfactuals never reach the target band.

Effect of temporal masking (w/o Mask). Disabling the mask causes near-complete compactness collapse ($0.802 \rightarrow 0.002$), confirming it is solely responsible for sparse, localised perturbations. Validity and proximity also degrade ($0.974 \rightarrow 0.851$; $0.605 \rightarrow 2.059$): without the mask, perturbations spread across the full input horizon, rendering counterfactuals unactionable.

Effect of the proximity reward (w/o Proximity). Setting $w_{\text{prox}} = 0$ degrades validity (-8.5 pp), proximity ($+34.0\%$), and plausibility ($+49.2\%$), revealing that proximity regularisation implicitly promotes distributional realism — a non-obvious but structurally grounded side effect.

These ablations confirm that each component makes a distinct and necessary contribution: the learned policy drives validity, the temporal mask enforces compactness, and the proximity reward preserves realism.

7.8 Qualitative Evaluation Across Datasets

7.8.1 ETTh1

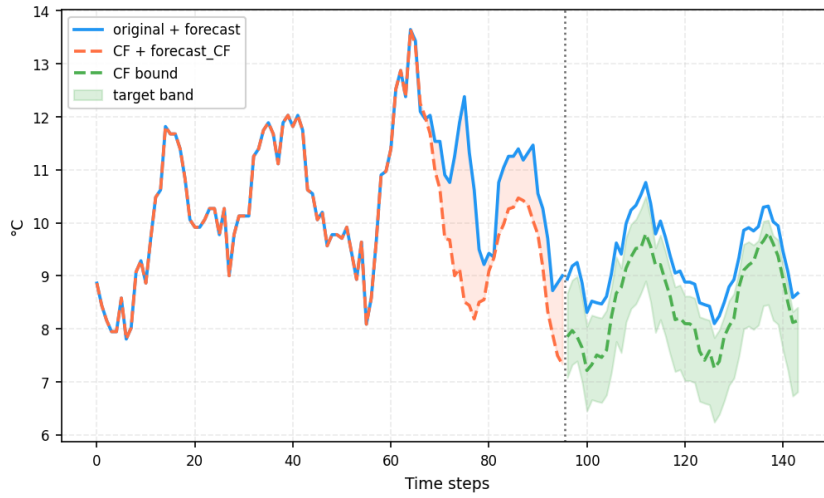


Figure 7.15: Qualitative examples on ETTh1. Per-sample validity: 0.94, 0.98, 1.00, 0.94.

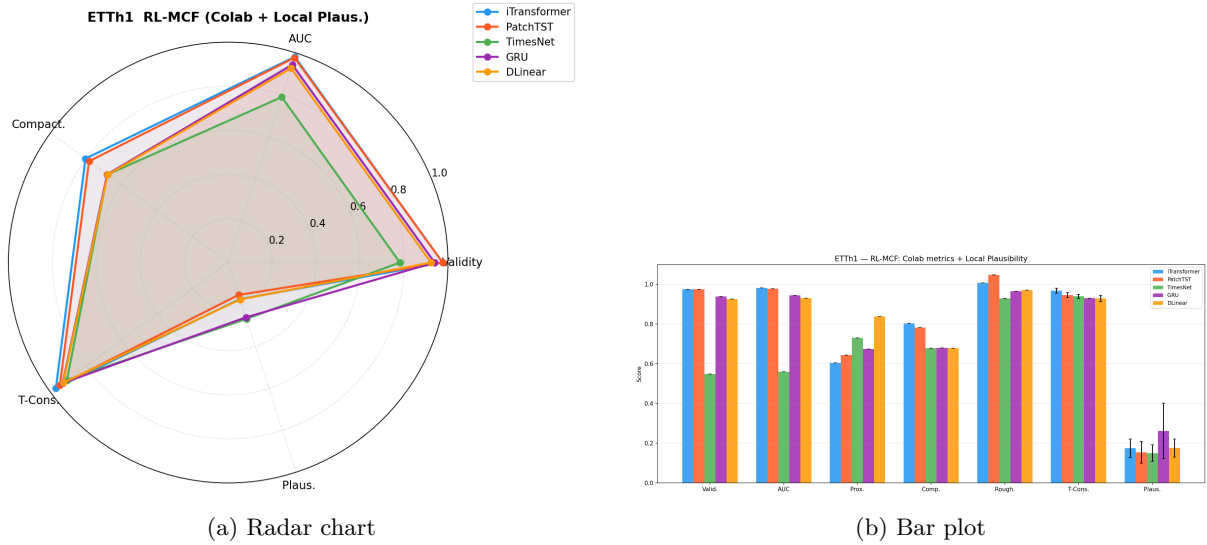


Figure 7.16: ETTh1: RL-MCF across 5 forecasting architectures.

The qualitative examples confirm that RL-MCF successfully steers the forecast into the target band with high per-sample validity (0.94–1.00) while preserving the natural shape of the series. The radar chart shows that iTransformer and PatchTST achieve the most balanced profiles, dominating on validity, compactness, and temporal consistency simultaneously. The bar plot confirms that plausibility remains uniformly low across all architectures, approaching BaseNN levels and confirming on-manifold generation.

7.8.2 ETTh2

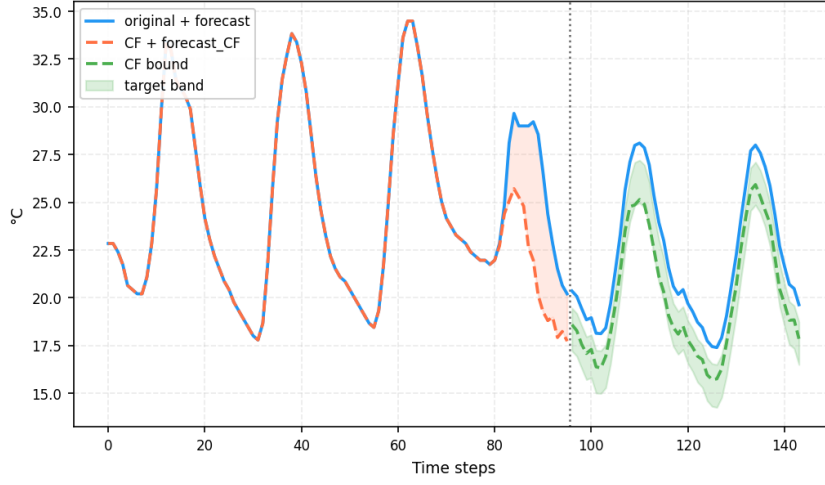


Figure 7.17: Qualitative examples on ETTh2. Per-sample validity: 0.96, 0.88, 0.69, 0.17.

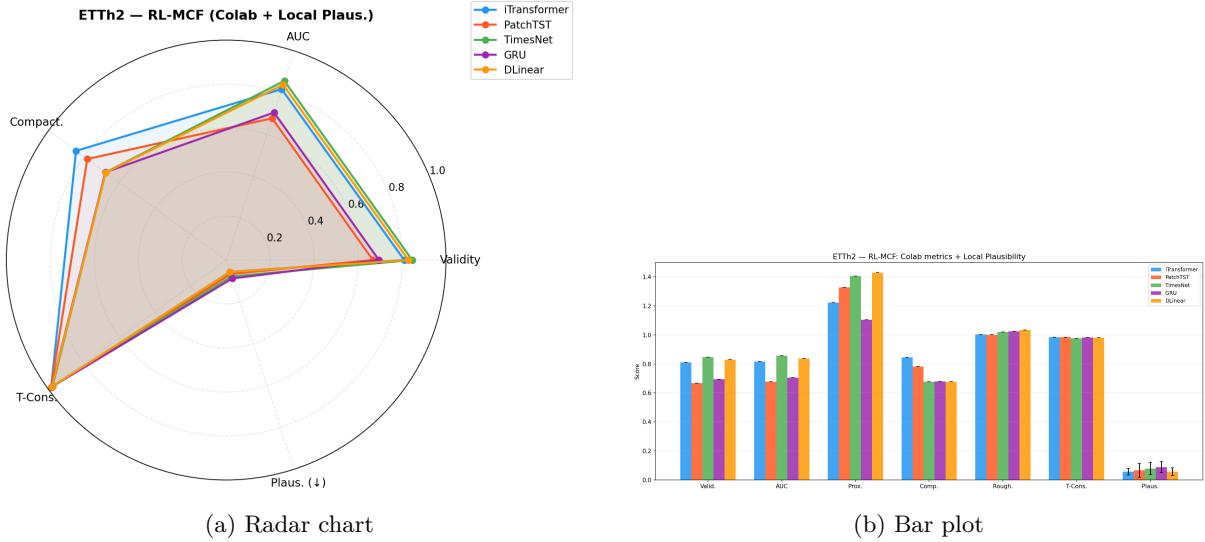


Figure 7.18: ETTh2: RL-MCF across 5 forecasting architectures.

The qualitative examples illustrate the increased difficulty of ETTh2. The fourth sample (validity 0.17) represents an extreme case where the target band lies beyond the agent’s reachable latent region — a direct consequence of ETTh2’s high-amplitude oscillations placing the validity boundary at an unusually large latent distance from the original encoding. The radar chart shows a globally reduced validity axis compared to ETTh1, with a shrunk proximity dimension reflecting higher L2 distances. Despite this, temporal consistency remains near-perfect across all architectures, confirming that latent perturbations preserve autocorrelation structure regardless of magnitude. The bar plot highlights that plausibility scores remain the lowest of all datasets, a direct consequence of the manifold constraint.

7.8.3 Weather

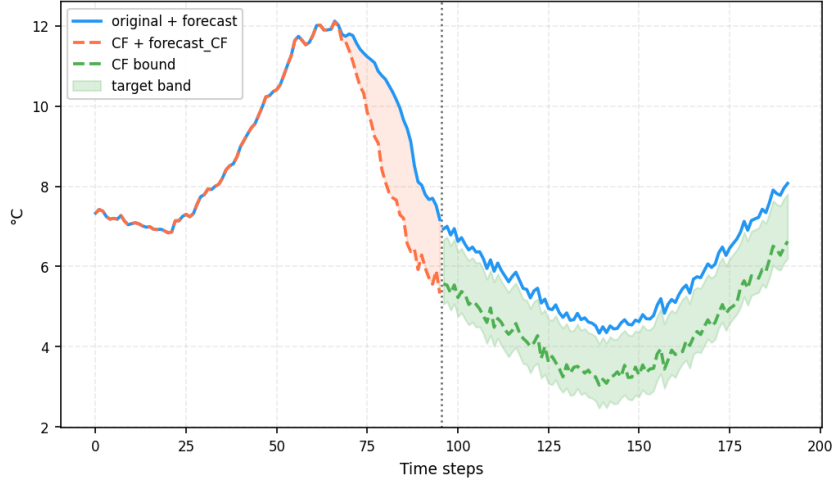


Figure 7.19: Qualitative examples on Weather. Per-sample validity: 1.00, 1.00, 1.00, 1.00.

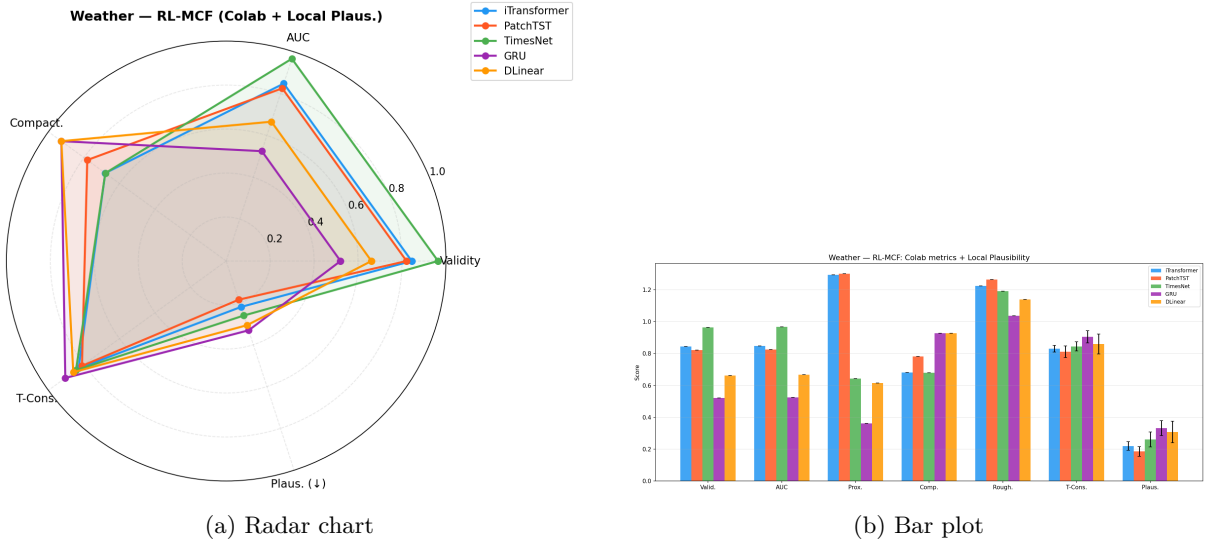


Figure 7.20: Weather: RL-MCF across 5 forecasting architectures.

The qualitative examples show perfect validity (1.00) on all four iTransformer samples, confirming reliable target band steering despite the high-frequency nature of the dataset. The radar chart exposes the sharpest architecture-level disparity of all three datasets: TimesNet achieves the widest area — consistent with Weather’s rich multi-periodic structure aligning with its 2D inductive bias, as discussed in Section 7.6.1 — while GRU collapses on the validity axis, confirming Weather/GRU as the most challenging configuration in the entire evaluation. The bar plot shows that compactness scores are the highest across all datasets (up to 0.9271), confirming that the temporal mask remains effective even under high distributional complexity.

7.9 Conclusion

This chapter presented a comprehensive empirical evaluation of the RL-MCF framework across three benchmark datasets and five forecasting architectures. The individual component evaluation confirmed that each pipeline component meets its functional requirements: the iTransformer achieves sufficient predictive accuracy across all three datasets, the TCN autoencoder faithfully reconstructs temporal structure, and the RL agent successfully generates sparse and localised counterfactuals. The hyperparameter sensitivity analysis established well-calibrated operating points for the key design parameters, confirming that the selected configuration achieves a robust balance between validity, proximity, and plausibility.

The quantitative comparison with baselines demonstrated that RL-MCF is the only method that jointly satisfies all desirable properties of a counterfactual explanation validity, compactness, temporal coherence, and distributional realism a combination that no baseline achieves simultaneously. While individual baselines may outperform RL-MCF on isolated metrics, they systematically fail on the remaining dimensions: BaseNN sacrifices proximity and compactness, BaseGrad collapses on plausibility and is architecture-dependent, and ForecastCF remains non-compact and temporally inconsistent. Beyond counterfactual quality, RL-MCF generates explanations via a single forward pass, achieving a constant inference time that is orders of magnitude faster than any gradient-based baseline a critical advantage for deployment at scale.

The ablation study confirmed that each design component makes a distinct and necessary contribution: the learned RL policy is the primary driver of validity, the temporal masking mechanism is solely responsible for compactness and perturbation locality, and the proximity reward term simultaneously promotes validity, proximity, and distributional realism. Removing any single component degrades performance across multiple dimensions simultaneously, confirming that no element is redundant.

Finally, the per-dataset qualitative evaluation confirmed that RL-MCF generalises robustly across datasets of varying complexity and across forecasting architectures of fundamentally different designs. Together, these results validate RL-MCF as a principled, scalable, and model-agnostic framework for counterfactual explanation in deep time series forecasting.

General Conclusion

The growing deployment of deep learning models for time series forecasting in high-stakes environments has created a pressing need for explanation methods that go beyond passive feature attribution. Practitioners facing an unexpected forecast need to answer a fundamentally different question: *what minimal, realistic modification to the observed input would have been sufficient to produce a different forecast?* Counterfactual explanations provide a direct, actionable answer, yet no existing method simultaneously satisfies the four desiderata of a high-quality counterfactual: validity, proximity, compactness, and distributional realism.

This work addressed this gap through the design, implementation, and empirical validation of **RL-MCF** (Reinforcement Learning Masked Counterfactual Explanations for time series forecasting). The framework rests on three tightly integrated components. First, a TCN autoencoder constrains the search to the data manifold in latent space, providing a distributional plausibility guarantee that gradient-based methods cannot offer. Second, a learnable temporal mask concentrates perturbations on the most recent input timesteps, producing causally localised and actionable explanations — the ablation study confirmed that removing this mechanism causes a near-complete collapse of compactness while simultaneously degrading validity and proximity. Third, an actor-critic agent learns a global generation policy that balances validity and proximity through a composite reward function, enabling efficient single-pass generation validated across five architectures and three benchmark datasets of increasing complexity. The six-dimensional evaluation protocol introduced in Chapter 5 further constitutes a methodological contribution in its own right, as no prior work had proposed such a comprehensive and formally grounded framework for this task.

The practical implications are concrete. In energy forecasting, an operator can immediately identify which historical load patterns, if slightly modified, would have led to a lower-risk forecast. In predictive maintenance, an engineer can determine which sensor readings, adjusted within a realistic range, would have predicted normal operation. The three components of RL-MCF are independently trained and interchangeable, significantly reducing integration cost in production environments.

Limitations and Future Perspectives

Despite these contributions, several directions remain open for future investigation.

Extension to multivariate settings. The current framework operates on univariate target series. Extending RL-MCF to fully multivariate settings requires counterfactual perturbations to simultaneously respect inter-variable dependencies and temporal autocorrelation, demanding a rethinking of both the latent perturbation strategy and the validity constraint.

Adaptive temporal masking. The temporal mask is currently fixed by design parameters (k, r) . An adaptive mask jointly optimised by the agent could yield more instance-specific actionability, concentrat-

ing perturbations on the timesteps most relevant to each individual counterfactual objective.

Integration of domain knowledge constraints. The reward function is currently domain-agnostic. Incorporating physical, business, or regulatory constraints as hard penalties or projection operators on the action space would make RL-MCF directly deployable in constrained operational environments.

Cross-dataset transferability. Evaluating the transferability of the learned perturbation policy across datasets without retraining remains an important direction, particularly for settings where labelled historical data is scarce.

Human-centred evaluation. A formal user study involving domain experts would complement the quantitative results with human-centred evidence an essential step toward deploying explainable AI systems in real-world decision-making contexts.

In summary, this dissertation demonstrated that reinforcement learning, combined with a structured latent space and a principled temporal masking mechanism, provides a powerful paradigm for generating counterfactual explanations for deep time series forecasting models. **RL-MCF** represents a significant step toward making black-box forecasting models genuinely interpretable not merely explained, but actionably understood.

Bibliography

- Abnar, Samira and Willem Zuidema (2020). “Quantifying Attention Flow in Transformers.” In: *arXiv preprint arXiv:2005.00928*.
- Alsudani, Rana Sabeeh Abboud and Jicheng Liu (2017). “The Use of Some of the Information Criterion in Determining the Best Model for Forecasting of Thalassemia Cases Depending on Iraqi Patient Data Using ARIMA Model.” In: *Journal of Applied Mathematics and Physics* 5.3, pp. 667–679.
- Arrieta, Alejandro Barredo et al. (2020). “Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI.” In: *Information Fusion* 58, pp. 82–115.
- Arulkumaran, Kai et al. (2017). “A Brief Survey of Deep Reinforcement Learning.” In: *arXiv preprint arXiv:1708.05866*.
- Bodria, Francesco et al. (2023). “Benchmarking and Survey of Explanation Methods for Black Box Models.” In: *Data Mining and Knowledge Discovery* 37, pp. 1719–1778. DOI: 10.1007/s10618-023-00933-9.
- Bruckert, Sebastian, Bettina Finzel, and Ute Schmid (2020). “The Next Generation of Medical Decision Support: A Roadmap Toward Transparent Expert Companions.” In: *Frontiers in Artificial Intelligence* 3, p. 507973.
- Cheng, Qisen et al. (2025). “Unifying Prediction and Explanation in Time-Series Transformers via Shapley-Based Pretraining.” In: *Proceedings of the 21st IEEE International Colloquium on Signal Processing and Its Applications (CSPA)*. IEEE, pp. 325–330.
- Chudo, Solomon Buke and Gyorgy Terdik (2025). “Modeling and Forecasting Time-Series Data with Multiple Seasonal Periods Using Periodograms.” In: *Econometrics* 13.2, p. 14.
- Crabbé, Jonathan and Mihaela Van Der Schaar (2021). “Explaining Time Series Predictions with Dynamic Masks.” In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. Vol. 139. Proceedings of Machine Learning Research. PMLR, pp. 2166–2177.
- Del Ser, Javier et al. (2022). “Exploring the Trade-Off Between Plausibility, Change Intensity and Adversarial Power in Counterfactual Explanations Using Multi-Objective Optimization.” In: *arXiv preprint arXiv:2205.10232*.
- Delaney, Eoin, Derek Greene, and Mark T. Keane (2021). “Instance-Based Counterfactual Explanations for Time Series Classification.” In: *Proceedings of the International Conference on Case-Based Reasoning (ICCBR)*. Springer, pp. 32–47.
- Dickey, David A. and Wayne A. Fuller (1979). “Distribution of the Estimators for Autoregressive Time Series with a Unit Root.” In: *Journal of the American Statistical Association* 74.366, pp. 427–431. DOI: 10.2307/2286348.
- Keane, Mark T. et al. (2021). “If Only We Had Better Counterfactual Explanations: Five Key Deficits to Rectify in the Evaluation of Counterfactual XAI Techniques.” In: *arXiv preprint arXiv:2103.01035*.

- Kreuzer, Tim, Jelena Zdravkovic, and Panagiotis Papapetrou (2025). “Unpacking the Trend: Decomposition as a Catalyst to Enhance Time Series Forecasting Models.” In: *Data Mining and Knowledge Discovery* 39, p. 54.
- Li, Peiyu et al. (2026). “Counterfactual Explanations for Time Series Data via Reinforcement Learning.” In: *arXiv preprint*. Submitted to ICLR 2026.
- Liu, Yong et al. (2023). “iTransformer: Inverted Transformers Are Effective for Time Series Forecasting.” In: *arXiv preprint arXiv:2310.06625*.
- Lopes, Pedro et al. (2022). “XAI Systems Evaluation: A Review of Human and Computer-Centred Methods.” In: *Applied Sciences* 12.19, p. 9423. DOI: 10.3390/app12199423.
- Lundberg, Scott M. and Su-In Lee (2017). “A Unified Approach to Interpreting Model Predictions.” In: *Advances in Neural Information Processing Systems* 30.
- Nie, Yuqi et al. (2023). “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers.” In: *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin (2016). ““Why Should I Trust You?” Explaining the Predictions of Any Classifier.” In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1135–1144.
- Rojat, Thomas et al. (2021). “Explainable Artificial Intelligence (XAI) on Time Series Data: A Survey.” In: *arXiv preprint arXiv:2104.00950*. DOI: 10.48550/arXiv.2104.00950.
- Schlegel, Udo et al. (2019). “Towards a Rigorous Evaluation of XAI Methods on Time Series.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pp. 4197–4201. DOI: 10.1109/ICCVW.2019.00516.
- Schwalbe, Gesina and Bettina Finzel (2023). “A Comprehensive Taxonomy for Explainable Artificial Intelligence: A Systematic Survey of Surveys on Methods and Concepts.” In: *Data Mining and Knowledge Discovery* 38, pp. 3043–3101. DOI: 10.1007/s10618-022-00867-8.
- TransferLab AppliedAI Institute (2025). *Interpretability by Design vs Post-Hoc Methods*. <https://transferlab.ai/series/explainable-ai/>. Accessed: 2025.
- Vaswani, Ashish et al. (2017). “Attention Is All You Need.” In: *Advances in Neural Information Processing Systems* 30.
- Verma, Sahil, John Dickerson, and Keegan Hines (2020). “Counterfactual Explanations for Machine Learning: A Review.” In: *arXiv preprint arXiv:2010.10596*.
- Vielhaben, Johanna et al. (2024). “Explainable AI for Time Series via Virtual Inspection Layers.” In: *Pattern Recognition* 150, p. 110309.
- Wachter, Sandra, Brent Mittelstadt, and Chris Russell (2017). “Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR.” In: *Harvard Journal of Law & Technology* 31, p. 841.
- Wan, Renzhuo et al. (2019). “Multivariate Temporal Convolutional Network: A Deep Neural Networks Approach for Multivariate Time Series Forecasting.” In: *Electronics* 8.8, p. 876.
- Wang, Yuxuan et al. (2024). “Deep Time Series Models: A Comprehensive Survey and Benchmark.” In: *arXiv preprint arXiv:2407.13278*.
- Wang, Zhendong et al. (2023). “Counterfactual Explanations for Time Series Forecasting.” In: *arXiv preprint arXiv:2310.08137*. Accepted at ICDM 2023. DOI: 10.48550/arXiv.2310.08137.
- Wen, Qingsong et al. (2022). “Transformers in Time Series: A Survey.” In: *arXiv preprint arXiv:2202.07125*.
- Wu, Haixu et al. (2021). “Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting.” In: *Advances in Neural Information Processing Systems* 34, pp. 22419–22430.
- Yan, Ruixuan et al. (2022). “Neuro-Symbolic Models for Interpretable Time Series Classification Using Temporal Logic Description.” In: *Proceedings of the 2022 IEEE International Conference on Data Mining (ICDM)*. IEEE, pp. 618–627.

- Zhang, Yunhao and Junchi Yan (2023). “Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting.” In: *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Zhang, Yuyi et al. (2024). “ShapTime: A General XAI Approach for Explainable Time Series Forecasting.” In: *Proceedings of the Conference on Intelligent Systems and Applications*. Springer, pp. 659–673.
- Zhou, Haoyi et al. (2021). “Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting.” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 12, pp. 11106–11115.
- Zhou, Tian et al. (2022). “FEDformer: Frequency Enhanced Decomposed Transformer for Long-Term Series Forecasting.” In: *Proceedings of the 39th International Conference on Machine Learning (ICML)*. PMLR, pp. 27268–27286.